

Daily Notes - 2025 June



Jun 30., Mon
Chaps Team



Jun 27., Fri
Chaps Team



Jun 26., Thu
Chaps Team



Jun 25., Wed
Chaps Team



Jun 24., Tue
Chaps



Jun 23., Mon
Chaps



Jun 20., Fri
Chaps Team



Jun 19., Thu
Chaps Team



Jun 18., Wed
Chaps team



Jun 17., Tue
Chaps



Jun 16., Mon
Chaps



Jun 13., Fri
Chaps



Jun 12., Thu
Chaps Team



Jun 11., Wed
Chaps Team



Jun 10., Tue
Chaps



Jun 6., Fri
Chaps



Jun 5., Thu
Chaps



Jun 4., Wed
Chaps Team



Jun 3., Tue
Chaps



Jun 2., Mon
Chap Team



Chaps Team

Balint Bartfai · Auth · I finished & deployed to sandbox moving Auth's persistence layer to Postgres. Wrote a compre...

↑ Jun 30., Mon

Chaps Team

 Balint Bartfai

Auth

I finished & deployed to sandbox moving Auth's persistence layer to Postgres. Wrote a comprehensive test suite to ensure functionality is identical to the previous Redis-based one. Token handling and read-heavy operations are still in Redis, with most DB ops being cached as well.

Ran into a bunch of edge cases.

- (Soft-) deleted users Redis - Chaps - DB sync
- User logins after deletion hanging around. New users with the same login as deleted ones were not able to re-register

For each edge-case found, I added tests to ensure the functionality works. Auth has become larger than just token handling. A bunch of code had to be refactored from the initial built-in-1-day for it to be properly testable.

For new code in critical systems like Auth: please ensure **automated tests are included for PRs**.

Tests now run on pushes and PRs in auth.

Ironically, the latest tests run locally but not on GH actions 🤖 I'll fix this Monday, not sure why module resolution is different.

Chap creation via Auth is wired up with K8s on sandbox! Caveats we'll need to solve with Levy:

- first chap creation call fails (while chaplet is spun up) - this is expected but not great behaviour
- second call can take ~8sec, after which the chaplet is running

I'm thinking that we can start a chaplet the moment a user registers - and with the sleeping logic coming, it'll basically stop itself if there's no chaps in use. The only "downside" to having more chaplets running is they take up some resources on the cluster - but since they will sit around and go to sleep, they won't consume much CPU or memory on the host node. We're paying for the resource pool, so there's no real additional costs involved either.

Mastra planning

Went through Mastra, wrote down my thoughts and plan for evaluation for Monday's workshop.

CRAFT

Chaps on Mastra thinking

On the Agent level · We'll need to take a closer look at Workflows, as these form the heart of Mastr...



Last Edited 6/27/2025

Thoughts for the workshop in general and about Mastra

we have 4 weeks left before going beta → I feel we need to be quite operational / delivery focused going forward (and also aim for releasing to staff as soon as possible). I'd look at Mastra migration also from this perspective.

Let's write up what needs to be definitely solved from agent fwk point of view:

blocker eliminate any delays before response starts generating

- eliminate chaplet startup time → warmup support and integrating it into native app
- eliminate executor process startup time (needs a single, already running executor process that picks up requests within chaplet)
- eliminate MCP server startup time (MCP servers of warmed up chap are started immediately on warmup)
- MCP call caching layer (as agent's needs results of some MCP calls in their context)

prio ? typed SDK to author chap configs

- not sure how seriously we think about this, and what experience we think should be there in beta.
- Mastra can be a quick win here, with type enforced steps in TS and its local dev server testing capability. Comes at the cost of vendor lock-in, and loss of portability, and loss of (much harder) security sandboxing for the chap (3rd party chap can execute any code).
- YAML authoring is a bit (quite) cumbersome, and enforcing types and testing configs likely needs more custom development than with Mastra, but super portable across platforms, also guarantees secure execution of 3rd party chap.

low Pre-emptive migration to avoid later increased costs

- Mastra has some general advantages: being on TS avoids a hybrid runtime (we can throw out Python), it is more modern regarding LLMs and tool call handling. Even for these reasons alone, we may want to migrate sooner or later, and the more code we add to Python, the more it will cost to migrate.
- From beta operational perspective I don't think this is a high prio in itself though (the value of getting to beta faster may be higher than extra time needed later to migrate)

low Features. Regarding feature set of Mastra, I don't think Mastra is a big win, at least not with current chaps beta focus. Rather a handy execution tool. I agree to move towards *capabilities* - that was exactly focus of recent weeks on our agent dev side: we now have support for modular configs, that are more than tools and instructions → but self contained capabilities (see eg [this](#)). We even implemented a simple sequential workflow steps execution based on yaml config, see eg [here](#). I think Mastra itself does not address *modularity* of these capabilities and also does not support agent *handoff*, which is my candidate model for orchestration (and also [OpenAIs](#)) (though maybe can be implemented as a workflow).

So I don't really believe in Mastra controlling the *high level agentic/lifecycle execution* of Chaps (or at least don't not find it that interesting now), but seems to be a useful tool as the *low level procedural executor* - to produce agent context, UI descriptors and other integration points of chaps, which are all typically typed configs - and as the config SDK experience.

↑ Daily Notes - 2025 June

Jun 27., Fri



Chaps Team

Peter Sipos · merged REST api for managing chap files · feat: add REST api for managing chap files · GET /api/chaps...

↑ Jun 27., Fri

Chaps Team

 Peter Sipos

merged REST api for managing chap files



feat: add REST api for managing chap files

#71 · Opened by Schipy · Merged by Schipy

Merged

GET `/api/chaps/:chapId/files?path=docs` ([Postman](#))

GET `/api/chaps/:chapId/files/content?name=docs/_user_learnings.md` ([Postman](#))

POST `/api/chaps/:chapId/files/content?name=docs/_user_learnings.md` ([Postman](#))

DELETE `/api/chaps/:chapId/files/content?name=docs/_user_learnings.md` ([Postman](#))

manipulation outside docs folder is not recommended except for testing, it's behind an additional query param (`iKnowWhatImDoing=true`)

merged Chap settings enhancements

chap settings is ~instance K-V store that can be used for static preferences type configuration, eg apiKey, enable/disable features, chap modes etc. Settings are manageable from API (also from magic link) and can also be exposed to LLMs as needed.



Chap settings enhancements

#70 · Opened by Schipy · Merged by Schipy

Merged

A couple of instance settings related improvements in this PR:

- **rebranded old "instance config" to settings** in code and also on endpoint change to `/api/chaps/:chapId/settings` (but `config` is supported for backward compa)
- also moved storage from `/meta/instance-config.json` to `/instance/settings-store.json` (with backward compa, so trying to read old location first, and we keep using them if present)
- added settings load/save helper to python code as well
- added new tools that expose reading/writing settings to LLMs
- added a demo feature to Linear chap, which allows it save a list of project ids based on usage as ones whose data (id) should be cached (prepopulated in its context)
- as a side improvement for above, added support for a batch of MCP calls as input in context rules

in-progress Some GCal chap improvements

- creating a proper private fork
- merging in upstream changes with improved timezone handling etc
- upstream it also improves list-calendars tool

- neededcas I would like to add calendars of the user to the agent context (boost perf if it does not always have to query the calendar ids, which is primary etc)

Also couple of discussion with BalintO regarding agent SDK

Plus had check-in with Zita (general, happens like every half year or so)

 2025. Jun 27., Fri

ooo (Friday)

Next week:

- Mastra?

also on the menu

- Agent SDK: support a scope dropdown menu in chat (eg for Linear teams)
- Agent SDK: control layer in agent config (ifs and includes in agent configs, eg turn off a feature based on a chat toggle)
- Agent SDK: agent communication style & rhythm? (long postponed)

 **Levy Melnikov**

 2025. Jun 26., Thu

Chaplets on K8S

Finished CI\CD and documentation. We tested k8s chaplets, and both for mine and Balint B's chaplets it worked.

Started preparing for moving our sandbox chaplets into k8s infra, but didn't add all of the secret keys **yet**.

 2025. Jun 27., Fri

Chaplets on K8s

Finalize our move to k8s.

 State of chaplets on sandbox will be lost.

Wired together auth and new supervisor - we had a first successful test run - new chaplets are being created on the new kubernetes cluster! 🎉 calls are proxied to the correct chaplet! 🎉 **Levy has made amazing progress with the cluster.**

Spent most of the day on Auth. Auth calls supervisor, which has now been swapped to the new cluster.

- rewrite to save to postgres instead of Redis for persistence
 - this requires splitting the DB schema between 2 repos. I'm not convinced this is the best solution, so I'm going to spend a bit more time to iron it out
- sped up local dev - vercel dev was cumbersome to set vars, etc, now it's one command

Tomorrow

- Finish **Auth redis → postgres**
- I'll be working with Levy if any **integration issues** crop up during the tests with K8s cluster
- **Prepare for workshop:** If nothing else comes up, I'll start to prepare Mastra and how we can PoC it with the current chap configs - which is scheduled for next week

↑ Daily Notes - 2025 June

Jun 26., Thu



Chaps Team

Peter Sipos · Server side tools · Learnings about server side (built-in) LLM tools after plumbing together server side...

↑ Jun 26., Thu

Chaps Team

Server side tools

Learnings about server side (built-in) LLM tools after plumbing together server side `web_search`

My takeaways:

- a minus point for Orchestra, as its own tool calling loop interferes with server side tool calling (see below)
- we have inferior control over server side tools, from SDK and user user experience perspective I don't like it that much:
 - we have no progress message or any visibility about it, only until it happened (cannot show feedback to user)
 - we cannot implement eg a per web search user authorization in the chat (like how Cursor asks for confirm before any web search or file delete). Web search is extra cost for user (eg [Anthropic pricing](#)), so user may want to tie it to extra confirm.
- also Exa.ai seem quite good actually, Cursor (and OpenRouter, Orchestra) also uses exa
GPT o3 on the topic: · Consensus among AI builders: Exa is very good—often the top choice for RAG pipelines beca...

via Orchestra→OpenRouter: `does not work to access server tools` . Eg for OpenAI it does not speak OpenAI specific response API, for Anthropic the needed `provider` routing is flawed, and it won't route our request to Anthropic provider.

- needed Orchestra fork tweaks to forward props
- issue 1: if we require OpenRouter to forward us to only those Anthropic providers that support the extra `web_search` tool → it says it does not have any that supports it
- if we just pin to "anthropic" provider, it still routes us eg to Amazon Bedrock (provider name dbl check and good)

via Orchestra→Anthropic: `kindof works`, with caveats:

- it seems flaky: Orchestra's own tool calling loop - where it inserts user messages that describes client side tools and the `tool_call` format the model has to use to call those tools - *interferes* with using the server side tools. When having both in play, behavior becomes chaotic with the model sometimes usign Orchestra syntax to call `web_search`, or returns plain text instead of `no_tool_calls` json response etc. I prompt against it (part of the feature snippet), so it should mostly work, but not a hard guarantee as I see.
- we don't have tracing on Anthropic API calls, I drafted a [PR](#), but I think it does not work yet
- providing my user location also did not work (`tools.0.web_search_20250305: Country code HU is not supported`)

(this whole thing also needed Orchestra fork tweaks to forward props, and also to handle multiple content part responses coming from Anthropic API)

I did have successful runs: eg adding link to a calendar event with actual search:

User: Can you search for a single web link for Interfood .hu order page and add it to my Saturday Interfood event?

Agent:

(it searched on server side:)



Sign in | Better Stack

logged the server side tools for us [LLM] API Response:

```
[{"type": "server_tool_use", "raw": "TextBlock(citations=None, text=None, type='server_tool_use', id='srvtoolu_01Gju2ZfHytv1shVkGcnVpB3', name='web_search', input={'query': 'Interfood.hu order page'})"}, {"type": "web_search_tool_result", "raw": "TextBlock(citations=None, text=None, type='web_search_tool_result', tool_use_id='srvtoolu_01Gju2ZfHytv1shVkGcnVpB3', content=[{'type': 'web_search_result', 'title': 'F\u00f90151oldal - angol | Interfood', 'url': 'https://www.interfood.hu/en/ ...'}])"]
```

Searching for the Saturday Interfood event to update

Searching for Interfood events in the calendar

Adding the [Interfood.hu](https://www.interfood.hu) order page link to the Saturday June 28th Interfood event

✓ The [Interfood.hu](https://www.interfood.hu) order page link (<https://www.interfood.hu/etlap-es-rendeles/>) has been successfully added to your Saturday Interfood event on June 28, 2025. You can now easily access the menu and place your order directly from your calendar event. \n\nIf you need the link added to more Interfood events or have any other requests, just let me know!

How to test **web_search**?

Enable web search by this chaplet api call **on a specific chap:**

luki-labs.postman.co

POST on `{{ROOT}}/api/chaps/:chapId/dynamic-agent-config?overwrite=true&name=web-search`

body:

```
1  setups:
2    default:
3      agents:
4        main_agent:
5          llm: AnthropicModels.custom_model(claude-sonnet-4-20250514)
6          llm_tools:
7            web_search:
8              type: web_search_20250305
9              name: web_search
10             max_uses: 5
11
```

```

12     context:
13     web_search_tool_use:
14         sort_index: 200
15     steps:
16     - output: |-
17         You have two categories of tools:
18     - Internal tool like **web_search** (type
19       `web_search_20250305`). You invoke it the way you're instructed to use server
20       tools (emit "server_tool_use" json); do NOT output any `tool_call` JSON string
       for web_search that the user later describes (web_search is not a user-defined
       external tool)
20     - External tools, described later in user messages. For these
       you must output a `tool_call` JSON block exactly as requested, so the framework
       calling you can execute the function.

```

Disable: DELETE on same route

merged enable user learnings for all chaps (applies to newly created chaps)



enable user learnings for all chaps

Control Layer for agent configs (`if` s and `include` s basically)

A few use cases I think the Agent SDK should support towards agent developers:

- have a chap setting that turns on-off a feature, eg web search for the chap
- implementing flexible data caching, eg Linear chap should be able to request data caching for teams and projects it works with all the time (where by "data caching" we mean injecting to its context for all request - as long as the setting says so of course - and then there's real data caching topic of when do we actually refetch that data)

Technically what we need:

- provide LLMs access to settings - this is just tool and prompting
- however as setting-exposition is a recurring pattern, it should be a reusable component → need an `include`, or rather a `constructor(args)` capability for config snippets
- be able to exclude a config snippet from merge ("turn off")

What crystallizes is a control layer that dynamically generates the actual list of configs that need to be merged to get the final agent configuration. Basically **we keep the current config merge logic**, but instead of statically merging all agent*.yaml configs in the chap, we define additional primitives and meta-templating, which allows as to conditionally include and argument-parametrized-include snippets multiple times to get the agent config snippets (which are then merged like now)

Draft syntax:

web search with an enabled setting (which can be set eg on UI by user)

```
1 use:
2   - localdef: web_search_anthropic
3     enabled: ${settings.enableWebSearch}
4
5 defs:
6   web_search_anthropic:
7     agents:
8       main_agent:
9         llm: AnthropicModels.custom_model(claude-sonnet-4-20250514)
10        llm_tools:
11          web_search:
12            type: web_search_20250305
13            name: web_search
14            max_uses: 5
15          context:
16            web_search_tool_use:
17              sort_index: 200
18              steps:
19                - output: |-
20                  You have two categories of tools:
21                    - Internal tool like **web_search** (type `web_search_20250305`).
22                    You invoke it the way you're instructed to use server tools (emit "server_tool_use"
23                      json); do NOT output any `tool_call` JSON string for web_search that the user later
24                      describes (web_search is not a user-defined external tool)
25                    - External tools, described later in user messages. For these you
26                      must output a `tool_call` JSON block exactly as requested, so the framework calling
27                      you can execute the function.
```

Linear's smart caching of data (where cache setting is controlled by the agent itself as it totally depends on user to user, whether certain teams, or projects or both is used frequently as scope). Here only illustrating for *projects*, but straightforward to extend to teams (and to project statuses within teams)

```
1 use:
2   # defined in an agent-expose-setting.yaml
3   - component: expose-setting
4     args:
5       setting_description: |-
6         linear_cache_project: array of often used project names, whose data should
7         be cached and provided for you for each user request
8   - localdef: linear_cached_data
9
10 defs:
11   linear_cached_data:
12     agents:
13       main_agent:
```

```

14     context:
15     linear_cached_data:
16         sort_index: 100
17     steps:
18         - output: |-
19             {% for result in get_project.results %}
20             The id of project {{ result.name }} is {{ result.id }}.
21             {% endfor %}
22     input:
23         - tool_call:
24             name: get_project
25             json_props: |-
26                 [
27                     {% for name in settings.linear_cache_project %}
28                     { "query": {{ name | tojson }} }{{ "," if not loop.last
29
30                     {% endfor %}
31

```

 2025. Jun 26., Thu

implement above use cases

Balint Bartfai

We discussed with Levy how to proceed in the morning, and agreed the Chaplet kubernetes cluster work can't really be split up into K8s - supervisor, they're interwoven. Levy will be continuing with that layer of the infra, I am moving one layer up to Platform.

- Fixed the chaplet build issue - tokens were messed up on Github.
- [#chaps-ops](#) set up with alerts from DigitalOcean. Will integrate more - BetterStack - later.

Name	Applied to
 Memory Utilization Percent is running high Memory Utilization Percent is above 85% for 5 min	All Droplets
 CPU Utilization Percent is running high CPU Utilization Percent is above 80% for 10 min	All Droplets
 5 Minute Load Average is running high 5 Minute Load Average is above 80 for 5 min	All Droplets

Platform on Cloudflare

Platform (API, sockets) is currently deployed on Fly.io as a container. [Yesterday cloudflare announced containers](#), today I implemented Platform on Cloudflare.

Initially, I managed to get the original platform container to run on Cloudflare. There were a couple of caveats and behaviours I ran into:

- CF containers are always behind Workers, and run on a durable object. Requests hit the worker first, which proxies the request to "a" container. Worker, DO, and container need to have env vars and secrets set
- Containers can fetch their env from an internal endpoint on startup (very useful for if we want to run chaplets) - and this is the solution that worked
- Because we can only use one instance of a container (due to Socket.io connections), we have to address the container (ie. tell CF which named instance we would like to use) in the worker. But by addressing Containers, that also means the container must be started by the Worker explicitly
- Containers can take a while to spin up. Cloudflare claims 2-3 seconds, my initial tests with the whole API running on the container were 10-11 sec. Later this did reduce to 3-4 sec after fixing all env vars, so connections don't fail and hang around on startup

Sockets work with the DO. Socket.io connections are routed through to the container.

This can be found on the branch <https://github.com/lukilabs/chaps-api/tree/cf-containers>

All-in-all, my experience with the product was very good. Logs and general observability is pretty good. Workers can respond to events happening, containers can be individually addressed or autoscale depending on the application. A quick calculation - if we run one container for 30 days nonstop, it would cost us 52USD. Containers spin down after a set timeout, so if we get to the point of having one running 0-24, we have a very successful product 🎉😊

I took the concept one step further, and decoupled routes which require sockets for broadcasts (messaging), and ones which don't. So I implemented Platform with the CF worker handling routes without sockets.

This can be found on the branch <https://github.com/lukilabs/chaps-api/tree/cf-platform-edge>

It's deployed to `https://platform-ng.chapsdev.com`.

There's quite a few changes that had to be made to make this work, including using a different framework (Hono). Let's discuss if we want to go down this route - it's a different concept to use an edge worker vs. running our own server on the container, but the upside is it's very scalable.

I haven't implemented this, but to go fully edge-capable, we can further decouple the DOs which are used for messaging:

- Instead of using one DO for all connections, we can have a DO per user
- Worker would then do the same as it does now with the socket.io connections - sends events based on eg. participants in a thread to the correct DO
- This way, multiple containers can be running, and we wouldn't need to address a specific one - state is handled by the DO, transparently to the clients

Platform on DigitalOcean

I'm going to move platform to DigitalOcean first, until we can fully test Cloudflare - containers are a very new product. This gives us the same flexibility as Fly.io, moves us off flyio, and onto the same infra as chaplets.

- Sandbox: <https://cloud.digitalocean.com/apps/a50c5742-4b51-44ba-a91f-59bde2633518?i=f0c88c>

Deployed to <https://chaps-platform-sandbox-5r8oj.ondigitalocean.app/>

Branch can be found here <https://github.com/lukilabs/chaps-api/tree/digitalocean>

Auth on Cloudflare

Auth is already edge-capable as it's running on Vercel, so it's relatively simple to adapt to the worker concept. Static hosting is handled by Pages, API by workers. Deploy is as easy as with Vercel.

Didn't finish this, and it's lower-pri, but I have a WIP branch for this as well <https://github.com/lukilabs/chaps-auth/tree/cloudflare>

Next steps

- Sync up on chaplet work - I'll wire up supervisor with platform
 - If there's anything here we can split up (supervisor) after the k8s work, I'll help with that

- Platform sandbox move. I'd like to at least have it off fly.io and on DO, then we can make the call if it makes sense to go further with Cloudflare. If sandbox goes well, I'm happy to move prod off Fly as well.
- Lower prio: move auth's persistence of chaps and users to neon. Currently auth is on persistent redis (which we use for tokens), but this carries the risk of desync with platform.

 Levy Melnikov

 2025. Jun 25., Wed

Chaplets on K8S

I've finished somewhat production ready setup. We now have:

- prod and sandbox chaplet clusters with supervisor. Cluster re-creation requires manual steps, I ditched the idea to make everything fully IaC.
- Chaplet images pushed to digital ocean and new supervisor can pull them
- New supervisor can create \ delete \ get chaplets. Redeployment is also available.
- Supervisor can also proxy requests. If chaplet is not created it will be created. Requests will fail until chaplet is created. Effectively if your chaplet is not started you can just keep sending requests until eventually one of them will return a response from chaplet.

 2025. Jun 26., Thu

Chaplets on K8S

One important thing remaining is CI\CD for new chaplet infra & documentation.

After that I will start moving chaplets from fly.io to digital ocean. I will start with sandbox and then proceed with prod.

If I have time I will also look into edge-cases & hardening e.g. restoring unit tests, what happens if a chaplets is relocated by kubernetes to a different node due to resources constraints etc.

↑ Jun 26., Thu · Chaps Team

also Exa.ai seem quite good actually, Cursor (and OpenRouter, Orchestra) also uses exa

GPT o3 on the topic:

- **Consensus among AI builders:** Exa is *very good*—often the top choice for RAG pipelines because its vector-first stack returns cleaner, easier-to-parse results.
- **Consensus among everyday web searchers:** “Promising, almost there.” It feels snappy and uncluttered, but you’ll still open Google for some generic or hyper-recent stuff.
- **Cursor community:** No systemic complaints about Exa; frustrations lie elsewhere. Installing the Exa MCP server is actually a *popular fix* when Cursor’s built-in search option glitches.

Short answer up-front

- **For AI and dev tooling** (RAG pipelines, agent-style searches, Cursor’s “web-search” tool) Exa consistently ranks in—or near—the lead on recent third-party and self-published retrieval benchmarks.

- **For everyday, consumer-style searching** Google still wins on breadth, recency and “long-tail” coverage, but many testers like Exa’s low-noise, ad-free results for focused, research-y prompts.
- **Inside Cursor** people rarely complain *about Exa itself*—most forum threads are about Cursor UI changes or agent quirks. A few users even install an Exa MCP server as a workaround when Cursor’s own web-search toggle disappears.

Below are the details, with links to the key data points.

1 How Exa differs from Google

Aspect	Google Search	Exa Search
Retrieval signal	Primarily keyword + PageRank, ads, freshness	Dense-vector + rerank models trained <i>for</i> LLM use
Output	Mixed: ads, Knowledge Panels, AI Overviews	Plain JSON or “Websets” UI—just titles, URLs, snippets (no ads/AI prose)
API focus	Meant for humans, gated commercial API for LLM grounding	Meant for LLMs/agents first; fine-grained filters (domain, embedding similarity, recency)
Index size	Vast (>100 B pages)	Smaller, but optimized to drop SEO spam/noise

Source: Exa product FAQs and docs

2 Quality benchmarks & independent tests

Study (date)	Metric	Winner
Exa blog SimpleQA benchmark (1 000 factual Qs, GPT-4o judge, Jan 2025)	Avg. answer score	Exa 4.18 vs Google Search Grounding 3.92, Bing 3.77, Perplexity 3.71
Galaxy.ai comparative eval (200 eclectic queries, LLM judge, Dec 2024)	“Fully correct” retrieval rate	Google 72 %, Exa 69 % , Perplexity 67 %, Tavily 63 % (difference <3 pp)
Patronus AI case study (semantic product-research queries, Apr 2025)	Relevance@3	Exa 81 % vs Bing API 54 %

Interpretation:

- Exa edges ahead on *semantic* or RAG-style tasks where embeddings matter.
- Google still has a slight edge on ultra-fresh or broad “head” queries.

- Across studies, gap is small enough that prompt quality and post-processing often matter more than engine choice.

3 What human testers say

- **Less junk, more sources.** Bloggers who are frustrated with Google's new AI Overviews praise Exa for "surfacing passages directly from trusted pages" and hiding SEO listicles .
- **Learning curve.** Several reviewers note you get better results once you phrase queries in a slightly more "database-like" way ("companies building open-source dev tools 2021-2025") .
- **Coverage gaps.** On HN some early users reported Exa "isn't even DuckDuckGo quality" for ordinary web searches—usually when the query relied on Google's long-tail index .
- **Cost sensitivity.** There are mild grumbles that Exa's paid tier now matches Perplexity/Gemini pricing .

4 Cursor-specific sentiment

- When Cursor briefly hid its native web-search toggle (v0.46), one user installed an **Exa MCP server as a workaround**—they complained about the missing button, not about Exa's quality .
- Community projects (e.g., "Cursor Rules CLI") use Exa for semantic look-ups and receive neutral-to-positive feedback; no one blames Exa for bad results—they focus on rule quality or agent memory .
- The loudest Cursor complaints today target agent loopiness, token costs, or model selection (Anthropic vs. others) rather than search results .

Bottom line: inside Cursor forums Exa is treated as a handy plug-in, not a pain-point.

5 When to choose Exa (vs Google/Bing/Perplexity)

Choose Exa if...	Stick with Google/Bing/Perplexity if...
You're building an LLM app or using Cursor's agent and need JSON, embeddings, and no-ads snippets.	You need exhaustive recall on obscure niches or ultra-fresh news minutes after it breaks.
Semantic filters ("all YC S21 robotics startups in Europe") matter more than keyword recall.	You can't tolerate occasional 404s or missing pages due to a smaller crawl.
You want programmatic pricing (per-token) and plan to deduplicate/rerank yourself.	You rely on Google's ranking signals (Backlinks, freshness) or want AI Overviews.

6 Verdict

- **Consensus among AI builders:** Exa is *very good*—often the top choice for RAG pipelines because its vector-first stack returns cleaner, easier-to-parse results.
- **Consensus among everyday web searchers:** “Promising, almost there.” It feels snappy and uncluttered, but you’ll still open Google for some generic or hyper-recent stuff.
- **Cursor community:** No systemic complaints about Exa; frustrations lie elsewhere. Installing the Exa MCP server is actually a *popular fix* when Cursor’s built-in search option glitches.

So, for “eventual human end-users” the current view is: Google remains the blanket tool, but Exa is carving out a loyal niche among power users who value semantic precision and ad-free snippets—and it integrates smoothly with Cursor without drawing notable ire.

Jun 25., Wed



Chaps Team

Peter Sipos · ~3 hr meetings/events today: monthly pres, Web Guild (basically only real touch point with Web Team...

Chaps Team

 Peter Sipos

~3 hr meetings/events today: monthly pres, Web Guild (basically only real touch point with Web Team since weeks aside from some daily note comments, so wanted to attend), sundae, so bit less progress on Agent SDK today

Proceedings

merged modularised agent configuration (and start introducing `/instance` folder)



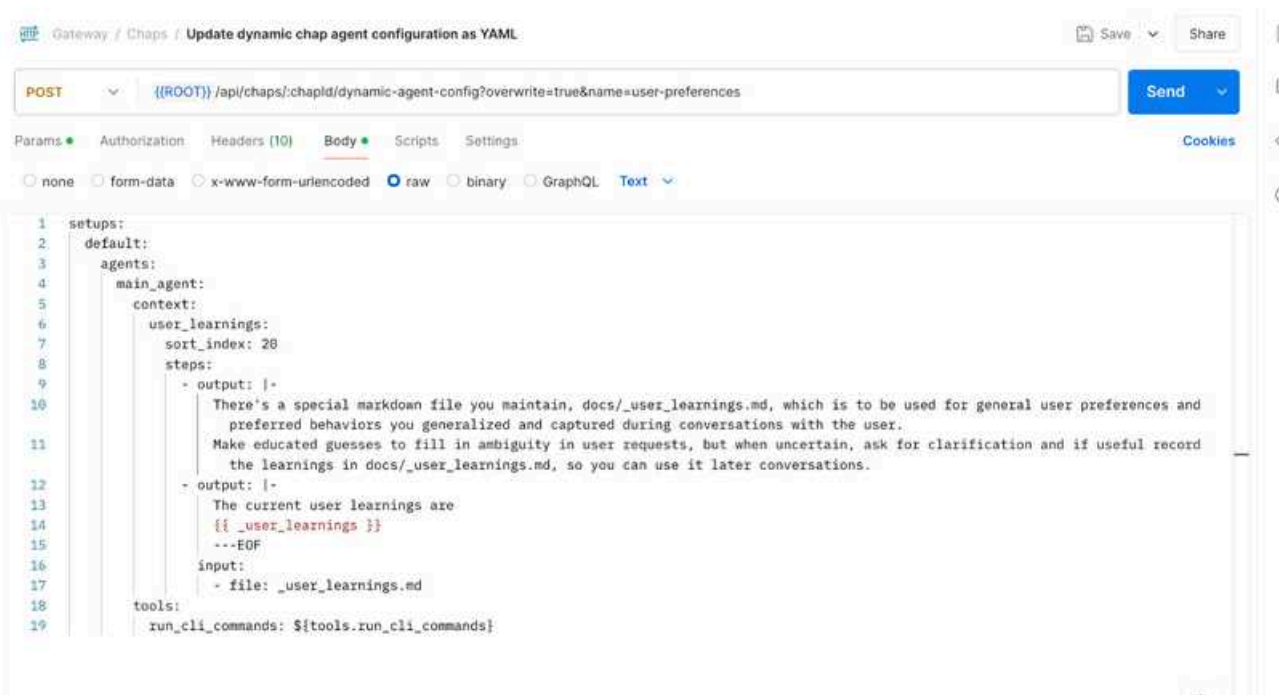
feat(agent-config): modularize agent configuration

#63 · Opened by Schipy · Merged by Schipy

Merged

introduced new top level chap folder: `/instance` (next to `/meta` and `/docs`), an intended grouping place for all runtime/instance specific non user data (settings, configs, later even tokens etc). This decouples them from "static" config which comes with chap installation (and is upgraded by chap update). As a start `instance` holds the dynamic agent components that are injected to a chap.

introduced **support for merging all `agent*.yaml`** files, both from static config path (`/meta` by default) and new dynamic config path `/instance`. This allows defining (or runtime injecting) self-contained agent component snippets, eg a user preferences agent component



`/dynamic-agent-config` and `/agent-config` gateway endpoints updated accordingly



did few test runs, and it really used the saved preferences in conversation

Web Search tool

Was trying to make native Anthropic web_search tool work.

I did set everything up to pass the tool to Anthropic via OpenRouter

- patched our Orchestra fork to forward extra params to OpenRouter call
 - this even had some hoops, as OpenAI SDK extra params (OpenRouter API is based on OpenAI API) need to be passed directly (like `tools`), but other extra params, like `provider` needs to be passed in an `extra_body`
- added support in our agent config to define `llm_tools` (for built-in provider side tool defs) which are forwarded to llm model factory calls of Orchestra, eg:

```
1 setups:
2   default:
3     agents:
4       main_agent:
5         llm: OpenrouterModels.custom_model(anthropic/claude-sonnet-4)
6         llm_tools:
7           web_search:
8             type: web_search_20250305
9             name: web_search
10            max_uses: 5
```

but it did not work

if we tell OpenRouter to only route us to providers that support the config we pass with

`"require_parameters": True` (see [OpenRouter routing](#)) then we get

```
Error code: 404 - {'error': {'message': 'No endpoints found that can handle the requested
parameters. To learn more about provider routing, visit: https://openrouter.ai/docs/provider-routing',
'code': 404}}
```

without enforcing `require_parameters` we can get routed to wherever eg `Amazon Bedrock` as model provider, which does not support `web_search` (not even on provider side), only `function` type tools.

```
{'error': {'message': 'Provider returned error', 'code': 400, 'metadata': {'raw':
'{"message": "tool_choice may only be specified while providing tools.", 'provider_name': 'Amazon
Bedrock'}}
```

So we do have to ask for fixed anthropic routing, however none of the slugs work, tried with

- `claude-sonnet-4-20250514`, as defined by [Anthropic docs](#) - not a valid OpenRouter slug
- the valid OpenRouter slug `anthropic/claude-sonnet-4`, which it is [properly aware](#) that it is labeled `claude-sonnet-4-20250514` on Anthropic so forwards it to the proper model name at Anthropic, still does not work (above 404)

I think OpenRouter actually does not know and support this tool parameter for Anthropic (it's own metadata bookkeeping does not contain that Anthropic supports this).

Next: one more try, see below

📅 2025. Jun 25., Wed

I'm having another quick go at native web search by directly using Anthropic models from Orchestra. This needs:

- patching now the same arg forwarding for AnthropicModels in our Orchestra fork (different model factory and implementation)
- make sure Anthropic API key is configured

Above should be < hour (either works or not), then next thing I'd like to look at is iterate a small bit on **chap settings**:

goal: be able to define a "setting" in the agent config, which is

- visible and manageable on trad APIs (eg how currently api key can be set in chap from a magic link)
- also exposed to LLMs via tools (eg support agent can manage it based in user request)
- and can be used to control behavior in agent config

something like (syntax may change):

```
1  main_agent:
2    llm_tools:
3      web_search: ${if: settings.is_web_search_enabled, web_search, None}
4
```

Additional next:

- want to check out monaco-yaml as a potential playground to edit agent configs
- Think how it fits to add tool call caching, eg its nice to remember in user prefs that I'm using a particular Linear project, but now agent always has to fetch the project id, synamic workflow status ids etc

Ephemeral chaplets / state management

Finished and deployed state management to sandbox chaplets currently running on the old fly infra.

New chaplets will not have volumes mounted. All chaplet state (chaps, data, etc) is archived into an R2 bucket as a single `zstd` archive, and synced up/down.

This is key to be able to run chaplets on new infra, as we cannot mount volumes.

State download is initiated when the chaplet starts up. Currently, state upload is triggered:

- After chap creation, deletion, update
- After config save
- Periodically - now set to 5mins, parameterized

Transition

During the transition phase to ephemeral chaplets, on our current **sandbox** chaplets:

- old volumes are mounted to `/data-old` - if you need anything ssh into the chaplet via `flyctl` and copy over files, then initiate sync up - `/data` is still available - see below
- upload is initiated automatically on changes (git commits called via service in TS), so a non-graceful shutdown will not save state - currently all chaplet stops are non-graceful. On the new kubernetes infra, the control plane handles graceful shutdowns, and uploads state automatically
- sync up can be initiated manually either from gateway's `StateManagementService` or `/app/state-upload.sh` (and `-download.sh` for the other direction as well)
- new chaplets do not have volume mounts at all. `/data` is still available but symlinked to the state extract dir (currently `/tmp/data`)

Completely new chaplets will be ephemeral **on sandbox, after the pool has been rotated**, and will be deploying to prod after a couple of days testing.

Some challenges along the way were the `zstd` library not working with `bun` - this has to do with how the library is built and linked on install with `bun`. We could either change the whole chaplet to use `node` instead, or the solution I chose, redo how archiving is done to call shell scripts from TS. Some file handling issues for how our current `/data` dir is structured and then `tar`-d and zipped up. Solving old mount points, getting the user IDs to `env`, etc.

Performance

Archive and upload for 600 files in a lot of directories (testing with `node_modules`) was 1-2sec.

Download was 700ms, on chaplet start. Archived state was 85kB, from source of 2,5Mb. This was from a chaplet running on fly in `FRA` to an R2 bucket in `us-east`. Same-region up/down should be significantly faster.

State versioning

Periodic sync up is set up on the chaplets. If for some reason this fails or corrupts state, we need to have a way to restore, otherwise a users chaps and data is lost when state is overwritten. R2 does not support object versioning.

For this, I set up an archive R2 bucket, `chaps-state-sandbox-versions` which gets state objects uploaded when a new state object is added to the original `chaps-state-sandbox` R2 bucket. It does this via event notifications to the state bucket, which invokes a CF Worker. The worker hashes the state, and if it differs from the previous state, uploads it as a version to the archive. Archived versions expire after 7 days.

 **lukilabs / chaps-workers**
CF workers for chaps

 lukilabs TypeScript Stars 0

Tomorrow

[Updated the delivery roadmap](#). I'd like to start working on Agent SDK next week after we finish the hardcore part of infra this week. We'll cut scope to deprio new socket comms on threads, as this is the largest change after chaplet infra, needs client changes as well, and has lower risk/reward - for now.

- chaplet state management done → I will be moving on to the control plane, will be syncing with Levy during standup, and planning the next couple of days. He's currently working on the cluster itself, we need to move the supervisor to be the actual K8s control plane
- chaplet proxying via supervisor still needs to be tested on the new cluster. To get to an E2E run for agents on new infra, I'll be
 - temporarily deploying a new version of supervisor to fly which calls the current, simplified supervisor on new infra → proxies to the new chaplets
 - this way we can start E2E runs

If this temporary new supervisor can start chaplets and route requests to them, we can switch over the fly VMs to K8s **on sandbox**. I am anticipating this work to finish ~EOD tomorrow. This means we should be able to start using them with Chaps Thursday/EOW.

 Levy Melnikov

 2025. Jun 24., Tue

Chaplets on Kubernetes

I made chaplets work on Kubernetes. Supervisor API created a container on our kubernetes cluster and it was reachable provided that the request had right auth.

 2025. Jun 25., Wed

Chaplets on Kubernetes - continue

The current state is that I have a github branch with code that can start a cluster(with manual steps) and launch supervisor there that works(can create a chaplet and authorize & proxy requests to it)

~ Wednesday

- Supervisor APIs
 - When a request to a chaplet is proxied it should be automatically launched if it was not launched before
 - Admin APIs: redeploy my\this\all chaplets, status of my\this chaplet
- Reproducible cluster - currently creating a kubernetes cluster requires manual steps. It would be nice to have a single script that provisions a new cluster so that we are sure that everything reproducible\close to IaC style. (Although full IaC probably won't be possible).
- Production cluster - I built sandbox cluster next to production one. I will need to "productionize" the sandbox one & ensure correct sandbox \ prod separation.
- CI\CD, documentation, deployment scripts, etc
- Tidy up - remove unused code

~ Thursday

- Stress test: e.g. delete a chaplet and see if it recovers.
- Hopefully finalize & finish moving chaplets from flyio to kubernetes

↑ Daily Notes - 2025 June

Jun 24., Tue



Chaps

Levy Melnikov · Mon, Jun 23 · APIs · Added an API to delete a thread message · Added an API to rename a thread · F...

↑ Jun 24., Tue

Chaps

 Levy Melnikov

 2025. Jun 23., Mon

APIs

- Added an API to delete a thread message
- Added an API to rename a thread
- Fixed an issue with chaplet not being able to report it's usage

Infra v2

Had a catchup with Balint B. around new infra and also started to look into the code on my own. We established a following goal \ working approach.

Current `v1` chaplet infra lives on `api.chaps.app` and corresponds to Fly.io machines. We're now building `v2` chaplet infra that lives on `apiv2.chaps.app` (not there yet) and corresponds to new k8s based cluster on Digital Ocean. My goal would be to ensure that:

- All APIs on `apiv2.chaps.app` work the same way as in `api.chaps.app`
- The behavior on the new cluster is robust: no chaplet duplication, if something comes to a chaplet it's restarted, env variables and secrets are passed, correct image is used, request proxying is fast and correct and so on.

👉 After that we will just need to:

- Switch `api` to `apiv2` on client and backend
- Delete chaplet infra from Flyio

 2025. Jun 24., Tue

Infra v2

Focus on infra v2. After tomorrow I will be able to tell what are the hard parts and how does the timeline look like.

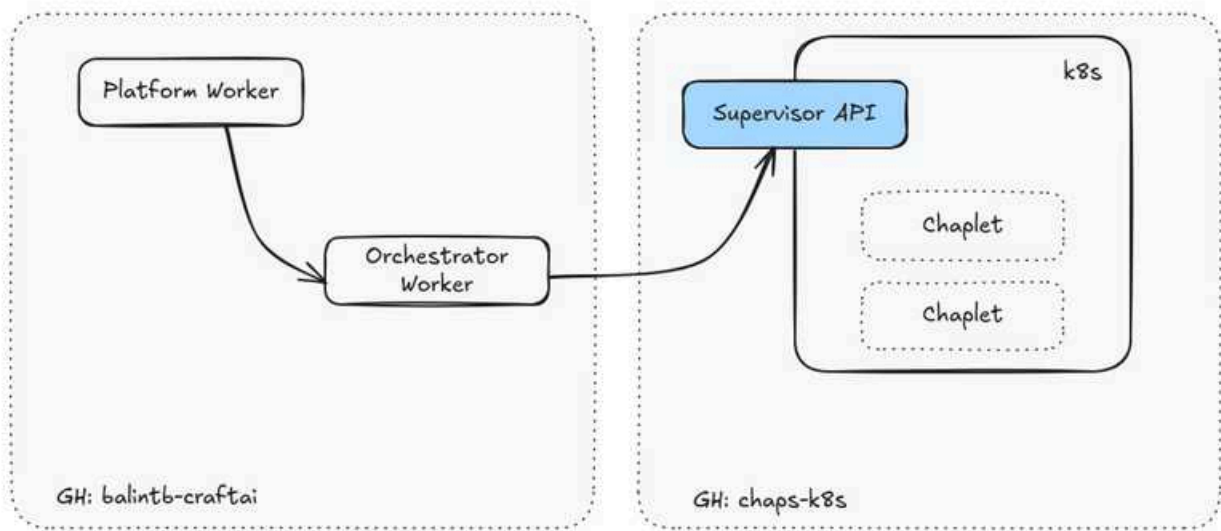
Some steps for tomorrow:

- Setup the `apiv2` domain
- Add logging to the new `apiv2` supervisor
- Check on how env variables \ secrets are passed
- Go over APIs

Meetings and planning today.

Levy and I went over the architecture for Chaps and agreed on work split + rough timings.

Currently, the new infra elements are split between two repos:



These roughly correspond left → right to higher → lower level. The plan is to go bottom → up, keeping as much backwards compatibility as possible.

Note: all of this exists as a PoC, it needs to be wired into what we have now in a transparent way.

- Chaplet state overlay R2 handling in new infra - **balint**
- Save all current state (/data dir) from current chaplets as state archive files
- Remove volume dependency from current chaplet
 - Starts only with state overlays
- Move current supervisor to K8s: - **levy**
 - Moving current supervisor routes into PoC supervisor
 - → enables **current** layers to work with Chaplets in Kubernetes
 - Move routing from pooled logic to custom chaplet ID (= user IDs) routes
- Start chaplet on user's first interaction - **levy** + **balint**
 - Start with state sync → no more volumes
 - **No stop** for now

We want to get to a point by mid-week where Chaplets run fully on new DigitalOcean infra. This part is quite ambiguous, it has the most risk of affecting the overall delivery plan.

Initially, we will cut scope by **not stopping** chaplets. This reduces overall complexity on the control plane. Once we are confident ephemeral chaplets can run, we'll modify the logic to deliberately and automatically stop. We're anticipating "organic" stops as a HW failure can happen at any time or the cluster can be scaled up → down for maintenance.

After the lowest-level infra is in place, we'll move up towards platform, starting with Auth. Changing the way connections are handled and synced will be the last step. I want to keep Chaps as operational as possible during the transition so we can onboard beta users.

Housekeeping

- Found why the Docker builds were extremely slow on Github → fixed. We were building `arm64` images alongside `amd64`, which was needed for when I was experimenting with the VMs on different providers. Apart from cutting build times to about 1/3, there was also an issue with `arm64` images and the `zstd` lib used for state compression. This has been fixed.
- Access control to new infra - updated perms for Levy

Tomorrow

Update delivery plan with new infra rollout and Levy back on board, + agent SDK as agreed this morning.

Finish the State handling in Kubernetes, move on to orchestration → starting chaplets on a users first interaction.

Agent SDK plans

A concise summary here, more details may follow in a separate doc

The Chaps Agent SDK should address the following topics:

- expose a configuration API surface that covers both common agent development primitives (instructions, tools, memory, conversation history) and also chap application specific primitives (settings, UI widgets etc)
- a hopefully capable and pleasant developer experience to build and test chaps or chap components

Why "own SDK"? For several reasons:

Note: did a deep research over the weekend about agent SDKs, to have a fresh industry overview regarding their responsibilities, feature set, where they put emphasis on etc

1. we want to provide a higher level SDK than what agent SDKs typically cover, tackle the "application layer"
2. having good agents will be a core value prop in Chaps, where we want control and flexibility to do our own things (vs any limitations existing agent SDKs have), and no lock-in to a specific platform
3. also agent SDKs out there don't know about chap specific elements, like chap settings, chap UI elements and widgets

Vision

composable chap agents: composable agents, where behaviors and features are defined as an encapsulated set of instructions, tools, memory/state, and application elements. These agent components then can be combined to define complete chaps, and can also be turned on/off based on settings, feature switches, tested individually, even on-demand assigned to Chaps as circumstances or workflows dictate (eg one Chap lending a capability to an other)

Similar how React hooks did an amazing jobs to encapsulate state, render output and effects to define reusable components and behaviors, we can do something similar with agent behaviors. An agent component encapsulates

- `state` (memory, settings)
- agent `instruction`, `context` and `tool` output
- potentially chap `lifecycle actions`
- and other `chap application specific items`, like widget api endpoints, chat UI components etc.

Inside each such component can be realized with varying complexity, eg it can be a simple instruction template, but it can also implement a complex state machine / workflow inside if it wants. Higher level agent component can use lower level components as building blocks etc.

Example: `UserPreferences` agent component

`Already works, see later` consists of the following primitives (assuming current file storage)

- `instruction` rule attaching contents of `_user_learnings.md` to main agent's system prompt
- `instruction` rule explaining `_user_learnings.md` file, and hints when to add things to it
- including the `tool` that allows updating user learnings file, in our current setup this the command line execute tool

Format: as ground truth stick to a declarative description, YAML, as that is

- the most flexible can be passed around and injected dynamically, without security downsides of dynamic code execution
- and also platform agnostic (Python or TS or on-client Swift execution equally supported)

Tooling to author the config: YAML based in first generation, eg a [monaco-yaml](#) playground webapp (with validation, autocomplete). Later can provide eg a Typescript playground (which serializes to YAML).

The playground will also be able to connect to a chaplet, inspect and update its configs on the fly (simple API endpoints, we mostly have it already), maybe even have a basic chat interface later.

Scope & roadmap until beta

can rough-plan in a few days, but I expect this to be highly dynamic, depending on chap learnings and current greatest priorities

Latest proceedings

merged Agent's understanding current date and conversation time flow fixed

How to use: This does not necessarily need newly created chaps, mostly works with older chaps as well (header will be added for them, but they have no extra instruction clarifying this header)

Introduced a [FrontMatter](#) (a simple metadata header) for all messages going into LLM, giving it a proper understanding of the conversation time flow and also clarifies source and intent of messages (eg user messages generated internally, or different assistant messages).



feat(frontmatter): introduce a FrontMatter giving the LLM a proper understanding of date and s...

#60 · Opened by Schipy · Merged by Schipy

Merged

This completely replaced the current date injection, which had big issues as the conversation history itself, eg if talking about dates, sent it off track regarding what is the current date. Also even if injected current date was super clarified to trump history, the agent would still not have sense whether messages in convo are from minutes or weeks ago. Having a date meta attached to each message is the true solution to this problem.

This metadata is also handy to indicate source and message intent

- internal user messages (eg automatic lifecycle prompts, or context info injected as user message)
- different role assistant messages (eg progress vs final response)

- later different users in a group chat

And thirdly, having a FrontMatter header is robust to delimit messages and keeping their date & intent, should they be merged internally by provider, like how [Anthropic does with consecutive message of the same role](#).

```
1 ---
2 date: 3 days ago
3 ---
4 What are my open high prio issues?
```

```
1 ---
2 date: 3 days ago
3 intent: response
4 ---
5 17 You currently have no open issues...
```

last user message has actual anchor date for the whole flow (at prominent end of input position, not in the messy middle of context)

```
1 ---
2 date: 2025-06-23T08:30:46
3 ---
4 do I have tasks today?
```

▼ More examples:

agent progress message

```
1 ---
2 date: 3 days ago
3 intent: progress
4 ---
5 Listing issues assigned to you to filter for open and high priority
```

example of user messages that are created internally, eg prompt of communicator agent:

```
1 ---
2 date: 2025-06-23T08:46:49
3 author: system
4 ---
5 {"history": [{"role": "user", "content": "can you create a..."}
```

merged fixed todo_chap (it had an issue in its config after a refactor)

How to use: This needs newly created `todo_chap`

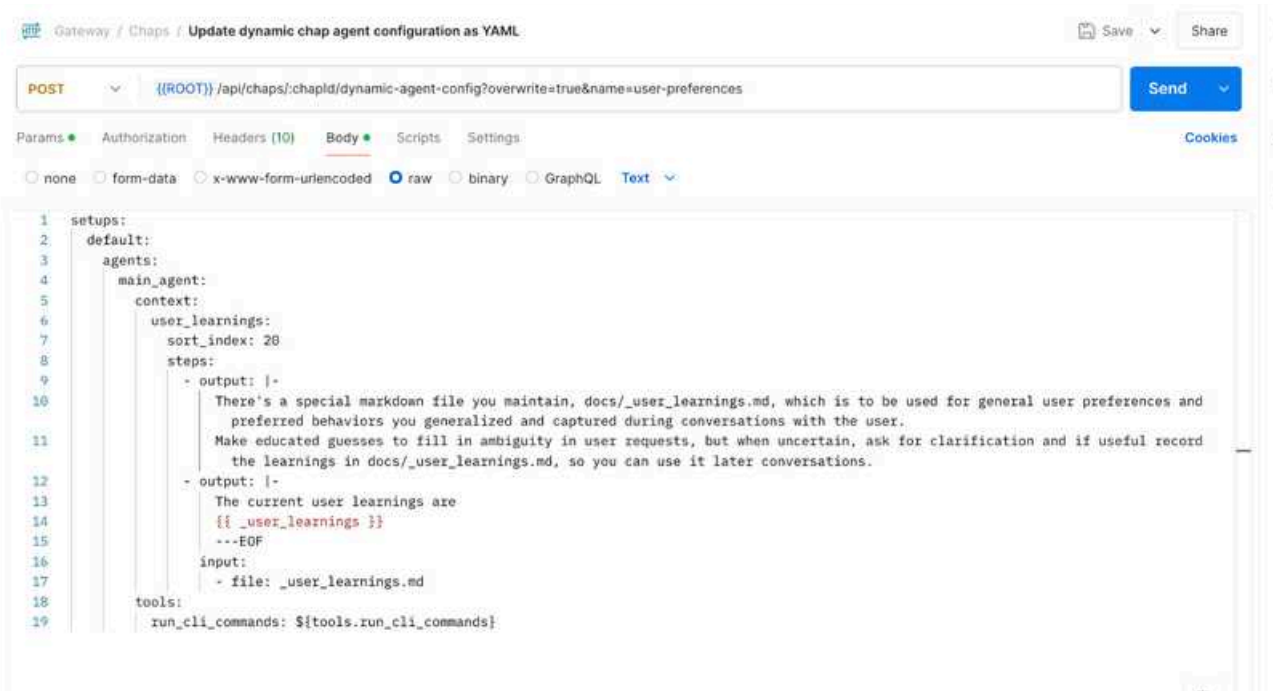
in-progress support ~current feature set with modular agent config

have 2 goals:

- be able to inject a user preferences agent component
- be able to inject using Anthropic model with native web search tool config component

Already have the following working:

posting a self contained `user-preferences` config that is merged into rest of the chap to enrich it with the capability to save and use learnings



then had conversation:

User: be default let's always search in Chaps.Agent project, can you remember that?

Chap: Retrieving the project details for 'Chaps.Agent' to store its ID for default searches

Chap: Recording user preference to default all searches to the Chaps.Agent project

Chap: ✅ I've set your preference so that all future searches will, by default, be focused on the Chaps.Agent project. You won't need to specify the project each time—I'll automatically use this context for your search and issue-related requests. If you ever want to change this default, just let me know!

saved to `_user_learnings.md`

```

1 - Default project for search: Chaps.Agent (id: 682561ee-de2e-4250-922e-1468c1e818fe)
2
  
```



Chaps

From Balint Orosz : · Tool Use - Web · As we can now test the agents from the app, I'm starting to see all sorts of us...

↑ Jun 23., Mon

Chaps

From  Balint Orosz :

1. Tool Use - Web

As we can now test the agents from the app, I'm starting to see all sorts of use cases which would be great to address. I think one of the most basic one is **general web search capabilities**. Most modern models have this built-in and I would love to be able to enable this - **either per request or per chap**.

Maybe we need to have it always on by default

cc  Peter Sipos let's discuss. Just adding the :online version didn't work so well, but newer models (Claude 4, gemini 2.5 etc) have their own tools if you tell them they will use, and I think this is going to be default for all LLMs moving forward

Sipi: Researched this a bit, there are 4 ways to get **web search**:

Context: we're using OpenRouter, which is the de-facto standard `/completions` api based.

`Responses` API (specific to OpenAI) cannot be used via OpenRouter.

Options:

Web search implemented by LLM provider

1. on current OpenRouter stack, use a model, that has built-in web search, for OpenAI models, this is basically only `gpt-4o-search-preview` (`models`), eg 4.1 does not have a web search capable variant
2. OpenAI also has model independent web search tool, which can be enabled on responses API for any model as I understand. For this we would need to move away from not just OpenRouter, but also hack Orchestra (as it uses the more widely adopted completions api, no support for responses api) (or +1 for Mastra I guess, as that [has support](#), though still the whole thing is OpenAI specific, so will not be available with other providers)

Web search implemented by OpenRouter

3. this is the `:online` slug, if added to the model, then OpenRouter [adds the web search](#) tool description, captures web search tool calls and executes them itself (actual search powered 3rd party [Exa](#)) - this is a convenience service for clients

We "implement" the web search tool

4. We add a web search tool ourselves, and have to provide an implementation (using a 3rd party web search provider)

Also, there's actually lots of stuff here that models are starting to provide like code execution tool, text editor tool etc.


Documentation
Code execution tool

Code execution tool - Anthropic

 <https://docs.anthropic.com/en/docs/agents-and-tools/tool-use/code-execution-tool>

I'm wondering if we should add all of these tools to the agent SDK? **Essentially it would mean we add tooling and a persistent environment on top of the MCPs (along with nice tweaks)?** OpenAI also has these through the `responses` endpoint. (still figuring out what that means)

cc  Balint Bartfai  Peter Sipos

2. Speed

Another thing is **speed** - this thing is quite slow 😞. A simple Hi is 9 sec, see :

 Sign in | Better Stack
Ship higher-quality software faster. Be the hero of your engineering teams.

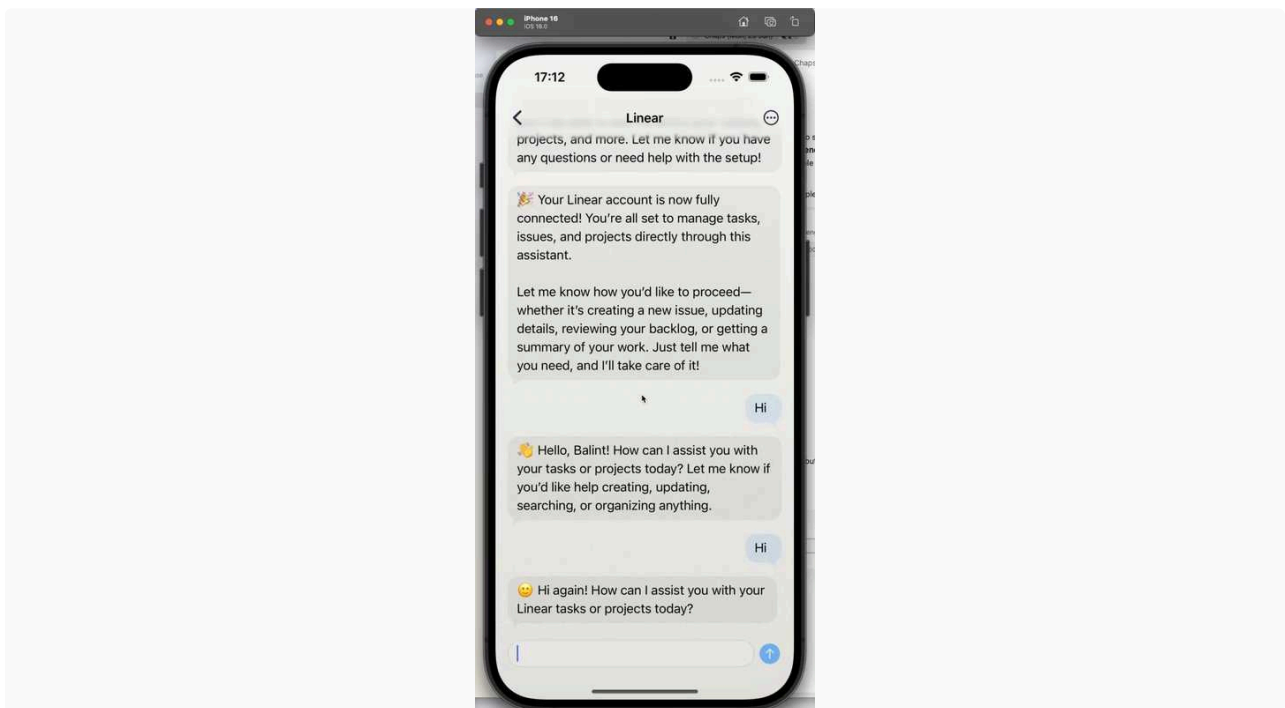
<https://telemetry.betterstack.com/team/377861/tail?s=1337041&a=1750427401878000.11717794&pt=overview&q=context.traceld%3ABL...>

It's like :

- 0.5 sec → From start to Setting the "Thinking Status"
- 2sec → Random delay until executor is started?
- 0.2sec → Parsing etc, tries to connect to MCP. Why?
- 1 sec → Connected to MCP
- 0.1sec → Load Tools
- 0.15sec → Get User Tool
- 0.5sec → LLM Response (Tool Call Loop)
- 1sec → LLM Response (Actual response)
- 0.1sec → Do stuff before Comm Agent
- 1sec → Communicator Agent Response
- 1.5 sec → Time until message is actually sent

So essentially for this request, LLMs were around 2.5sec, but it took us 9 seconds, so cca 4x overhead. This seems very slow to me?

If you want to test this is how you get the traceld (it copies a better stack link you can paste in browser) :



API Requests from  Balint Orosz : ( Balint Bartfai  Levy Melnikov these would improve experimentation a *lot!* So **not blocker**, but big enabler)

- Delete Message From Thread → *So I can delete messages and try different models/configs on the same history*
- Rename Thread → *So I can create multiple threads on the UI and test them parallel*
- System Prompt in BetterStack trace ( Peter Sipos on it) → *So I can get the whole flow and send it directly to OpenAI/Claude etc api*

Heads down today to proto everything for execution starting Monday.



lukilabs / chaps-k8s
Chaplet infra K8s cluster



lukilabs

TypeScript

Stars 0

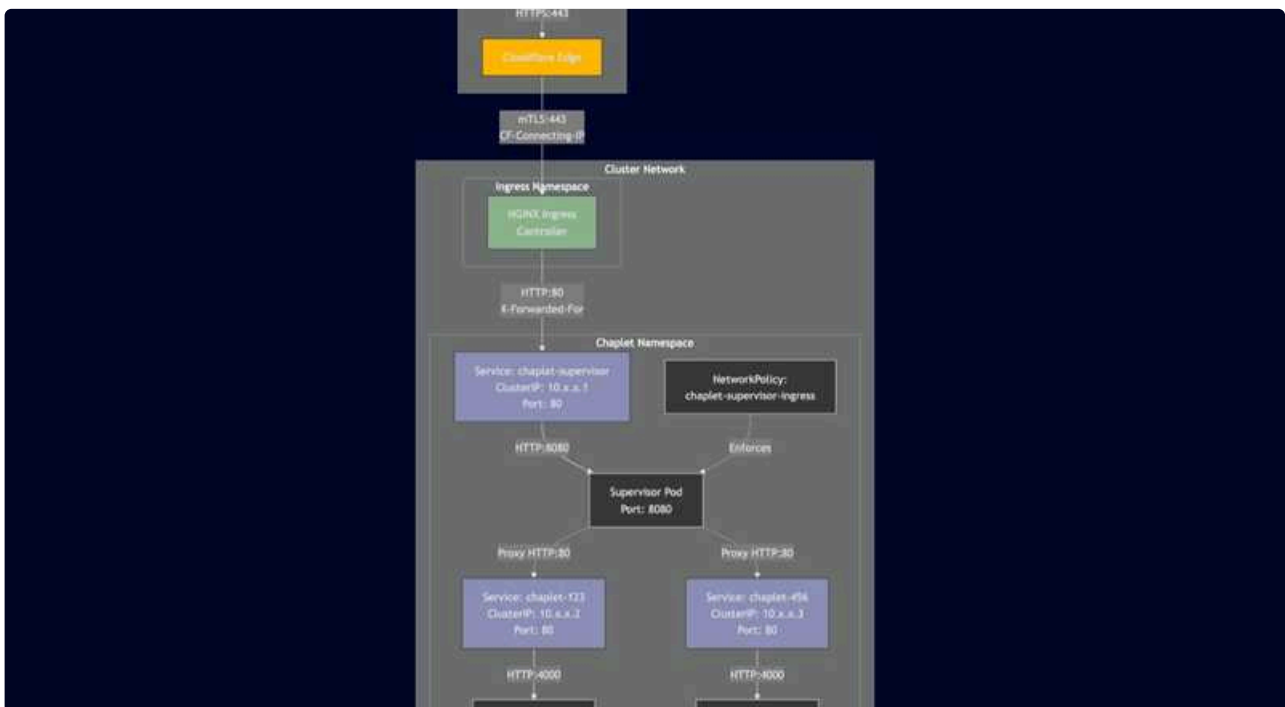
Housekeeping - setting up our DigitalOcean container registry, shuffling infra around to be in the same `nyc3` region.

I finally got the current chaplet to build and run on the K8s cluster.

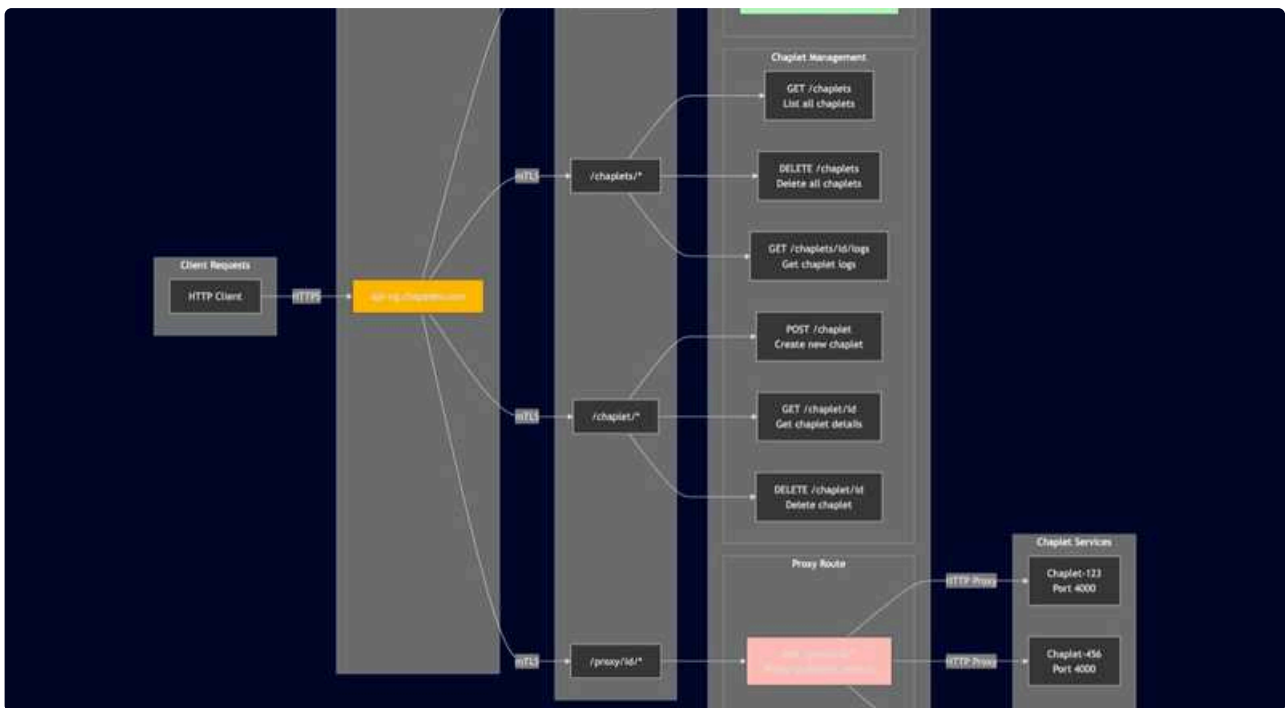
Set up debug and logging HTTP endpoints for chaplets - apart from dev, we can use this for alerting in the future.

I set up HTTPS two ways:

- TLS termination on the ingress controller. This involves managing certs with letsencrypt, and allowing traffic via HTTPS directly to the load balancer. Downside here is this needs a wildcard domain to work (and LE to issue certs)
- mTLS authenticated origin pulls via Cloudflare. This means Cloudflare authenticates itself when doing an origin fetch to the cluster. TLS termination is done on the CF edge, all traffic to the cluster is HTTP with mTLS certs.



We will go with the second option. Cloudflare will terminate the SSL connection on the edge and identify itself to the cluster. All requests are proxied through Cloudflare. This includes additional benefits such as DDoS protection, being able to run a worker on the route (eg. path rewrites). Downside here is that we're unable to send requests via IP to the cluster, all traffic must go through a domain in Cloudflare - but it's toggleable on/off for dev.



Added chaplet lifecycle event tracking to supervisor, we can now send webhooks when something changes in the cluster.



I feel we're on solid nextgen foundations now. Monday will be a full day going over the new architecture with Levy and planning / starting execution.

What's still unknown and a risk: characteristics under general load. Characteristics under chat thread (==user VM) spikes. I have demonstrated the chaplet cluster to scale, but have not done so under real spikey traffic. This will have to be tested.

Key topics for next week:

- infra execution - chaplet to auth layer
- Mastra PoC planning

if Chaps fails we could get into the business of docker container hosting 😊 we've built out a generic infra for running containers with our own orchestration

↑ Daily Notes - 2025 June

Jun 20., Fri



Chaps Team

Balint Bartfai · Chaplet orchestration infra · Worked on the low-level chaplet orchestration today. I weighed the pros...

↑ Jun 20., Fri

Chaps Team



Balint Bartfai

Chaplet orchestration infra

Worked on the low-level chaplet orchestration today. I weighed the pros-cons of each approach to orchestrate and run chaplet VMs.

Solution	Pros	Cons	Cost
Stay on Fly	No changes.	Costs too much for dedi CPU. Capacity issues. App instance cap. For VM pool, see below.	1x 10USD / user fix + running machines (x 36USD) + hot pool.
Own VM orchestration	Fast hot VM starts from pool.	Complex orchestration. Pool size needs to be managed manually. Cannot scale fast enough for spikes. VM start 50-60sec. Resource waste due to idle hot machines.	Fast: VM / all users. Optimal but slow: running VM per thread user.

Own Docker server running chaplet containers	Fast, full control over containers running. Slightly easier orchestration.	Full responsibility for server. Vertical scaling needed, will get costly. SPOF, if host goes down all chaplets die and lose state.	Fix cost of Docker server.
App platform-style docker container hosting	Easy setup and running of containers. Start can be fast. Easier orchestration (API calls).	App instance cap → custom pooling needed for instances. Only GCE and AWS allow addressing single instances of containers. Both have high overhead and complexity of setup / maint.	1x VM per user running + hot pool.
Kubernetes with pods as chaplets	Resource usage more optimal. Host nodes scale automatically up/down. Easy addressing of running chaplets.	Complex to set up.	N number of hosts running. Depending on size, 1 host = 8-32 active threads.

Orchestration PoC

So today, I basically built our own Fly.io on DigitalOcean with Kubernetes. At least a working PoC.



lukilabs / chaps-k8s

Chaplet infra K8s cluster

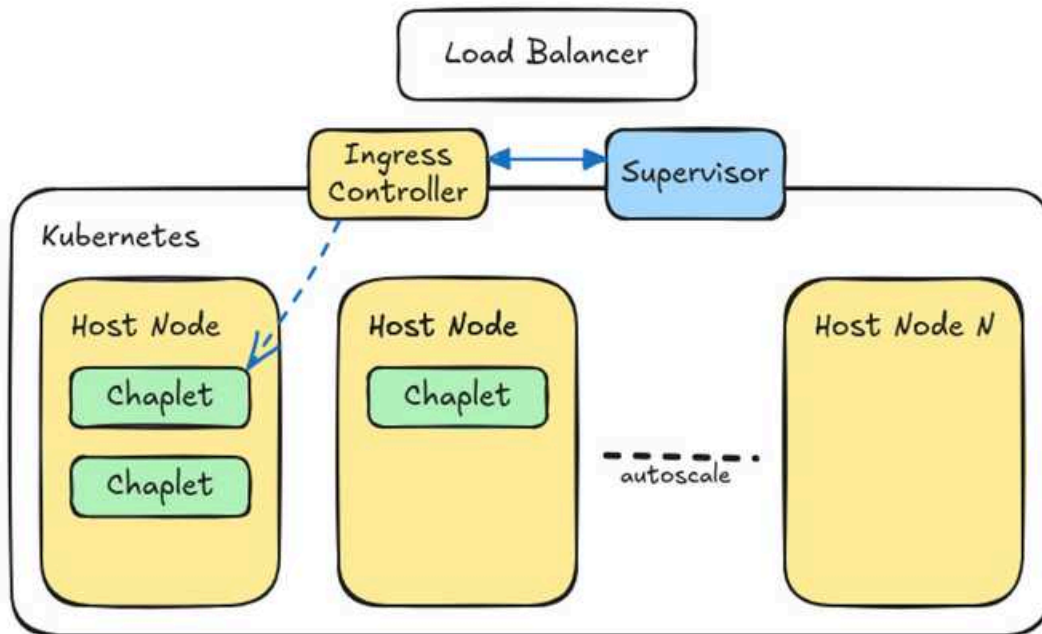


lukilabs

TypeScript

Stars 0

It still needs refinement, but an overview:



A Kubernetes cluster consists of host *nodes*, which are capable of starting *pods* - docker containers. A host node can run around 4-12 active chaplet VMs. Once the control plane detects N nodes have reached an aggregate of 80% capacity, it spawns a new host node. Other direction, once usage drops below 60%, a host node is stopped. When a request comes in to start a chaplet, Kubernetes is responsible for finding a healthy node with capacity to start the container on. The supervisor sits between running chaplets and the API caller, orchestrating requesting starting and stopping of services in the cluster.

A *host* start is around 60sec, we need to set limits accordingly. VM start is in the 2-3sec range. Host starts are only needed when we hit capacity%, and will scale down automatically.

The solution is built on DigitalOcean's hosted Kubernetes service, with a load balancer in front (todo: terminating SSL). When a chaplet is started, an entry is added to the ingress controller inside the cluster to route requests on certain paths to the chaplet itself, and a pod (=chaplet) is started with whatever params need to be passed.

New endpoints - called by the platform services - are [in Postman](#).

Chaplet routes

Chaplets are proxied through the LB / ingress controller, they are available auth'd from the outside, on routes like `/proxy/{chapletID}/...`

The chaplet sees this route as if it were their `/`

Currently, this proxying is done in the supervisor server itself - I ran into some issues with properly proxying via ingress / LB and headers. I'll solve this.

Fun fact: This was the first time I [had to google something](#) for the model to be able to solve an issue.

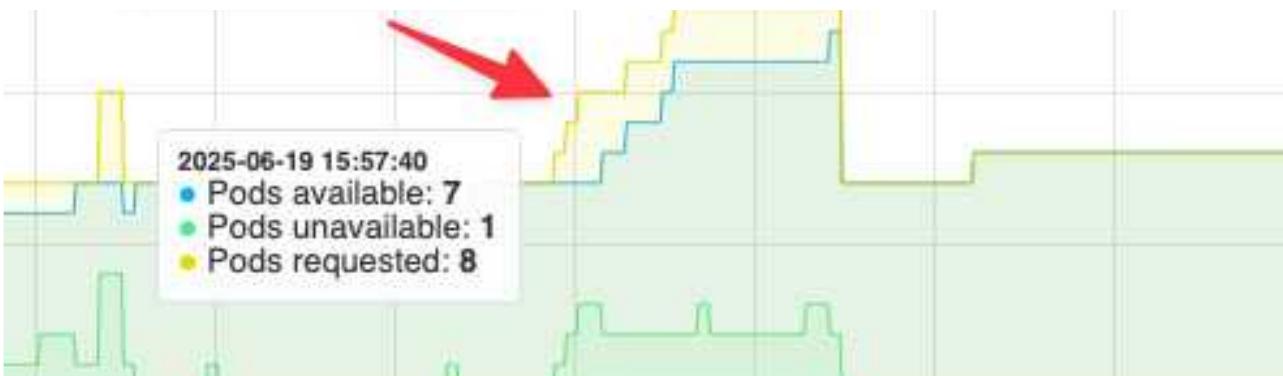
Current limitations of the PoC:

- I'm running a dummy chaplet. Will need to get the actual chaplet compiling and running on DO. I ran into problems with the build & push from GH today with some strange `docker buildx` error
- Supervisor needs the usual Auth, chaplet management endpoints. We can use the same routes as the current supervisor

Pod deployments on the cluster:



Autoscale with limits set lower for testing



Costwise, the complete 3 node cluster (= 3x 8-ish chaplets) costs 72USD / month. This can scale up if needed, and we can reduce the number of standby nodes to cut on this cost. Compared to the other solutions, it's much more cost effective and wastes less resources once we reach 10 users.

Tomorrow

Levy will be coming back Monday, so I'll be spending the day tomorrow finishing up this orchestration. We'll have a proper planning Monday once he's here.

What remains is basically running the chaplet VM itself here on Kubernetes, hoisting over supervisor routes, and enabling chaplet state overlays via block storage. Then we have full Fly.io capability *without* volumes, which are not needed anymore.

If I have time I'll wire this up with the Cloudflare PoC → this means a full Chaps Frontier Infra PoC.

ooo 📅 2025. Jun 20., Fri (Friday)

Next week: more agent logic: communication rhythm, user learnings (aka preferences) capability etc

Introduced progress messages on chat



emit tool call progress messages from execution as chat messages

#56 · Opened by Schipy · Merged by Schipy

Merged

How to distinguish progress vs final messages:

- I introduced a `messageIntent` in `metadata`: "response" | "progress" | "filler"
- plus added sending the source `agentId` in metadata (it's also good to know, especially when this message history is used later in conversation history in chap)

example:

```
1  {
2    ...
3    "senderType": "chap",
4    "content": "Listing issues assigned to you in the Craft Web team to filter
for open medium priority",
5    "type": "text",
6    "metadata": {
7      "agentId": "main_agent",
8      "traceId": "bSMY1WDezgqmC1t4pJ59m",
9      "messageIntent": "progress"
10   },
11   ...
12 },
```

New flexible agent context (system prompt) definitions system

- migrated old `auto_context_files`, `system_prompt_addition`, and hardcoded `current date` injection as well to new config



feat(agents): flexible agent context definition system

#58 · Opened by Schipy · Merged by Schipy

Merged

Fixed Orchestra internal messages to show up in tracing



chore(tracing): fix orchestra's internal logs to appear in tracing

#59 · Opened by Schipy · Merged by Schipy

Merged



Chaps Team

Peter Sipos · merged add tracing on BetterStack for prompt requests in chaplet · chore(logging): Tracing prompt re...

↑ Jun 19., Thu

Chaps Team

 Peter Sipos

merged add tracing on BetterStack for prompt requests in chaplet



chore(logging): Tracing prompt requests

#54 · Opened by Schipy · Merged by Schipy

Merged

had to fix up several things before it started work e2e. I tested both locally and on sandbox.

- added `traceId` to context not just when receiving a realtime chat message, but also when an internal prompt run is triggered from lifecycle events (so we have tracing eg for welcome message)
- had to fix a `traceld` propagation issue where end client message was forwarded to orchestra (had a new `traceld` assigned by the time it arrived to execution)
- had a mismatch where in Python process I logged `trace_id` and not `traceId` (as per Python convention), but then names did not line up on BetterStack of course
- then all orchestra service logs showed up as ERROR logs, because as I started to use standard Python logging instead of console output, the default is it logs to `err` (as `std_out` is traditionally for actual output of process). As our log pusher forwards `std_out` as INFO logs and `std_err` as ERROR logs (btw, all other severities are reduced to these now from all processes), this meant all python logs showed up as ERROR. I fixed by adding my own log handlers
- Caveat: actual Orchestra (meaning the lib) internal logs still show up as `error`. I did have one go at it to replace its handlers to mine, but did not work right away and really didn't want to spend more time on this project
- Caveat 2: I realized this also means Orchestra internal logs don't get the context information so won't have the `traceld` in them, and won't show up anyway (if just filtering for `traceld`), so this has to be revisited for one more fix round, as for actual tool calls we so far relied on internal logs. So either need to control the handlers in the Orchestra logger as well, or we will log the interim messages on our side as well (though they will be then duplicated in total)

Then finally returned to Chaps logic, where I started working on agent context enrichment with essential Chap domain specific info.

I split the topic to two main capabilities/steps.

1. being able to enrich context based on MCP tools, local files, Chap settings etc
2. tool call caching in general, which can optimize this context enrichment, but may also be used for caching actual tool calls issued by LLM.

Worked on the first one, where the goal is to be able to use any data source that the Chap has (chap's local files, agent's tools, chap settings etc) and use templating power to inject derived and formatted information as context to the agent before calls. Managed to get to some results:

Linear agent.yaml snippet

```

1  main_agent:
2    # ...
3    context:
4      - output: |-
5        The user's id is {{ access_token.split(':')[0] }}
6      input:
7        - file: .oauth/linear/.access-token
8      var_output:
9        - name: user_id
10         value: "{{ access_token.split(':')[0] }}"
11      - output: |-
12        The user's Linear name is {{ get_user.result.name }}, email is
13        {{ get_user.result.email }}.
14      input:
15        - tool_call:
16          name: get_user
17          props:
18            query: "{{ var_ctx.user_id }}"

```

resulting snippet added to main agent's system prompt:

```

1  The user's id is 3c3af189-7ca7-455d-8e53-0b51678a04c4
2  The user's Linear name is Peter Sipos, email is schipy@luki.io.

```

What we see here:

- user's id is needed for some linear tool calls as filter value. There's no direct tool call to get it, and LLMs had bad track record of figuring out how to get it (eg from a ticket of the user), especially if it would be just a building block in a larger query plan. However the user id is present in Linear's OAuth access token, so we already have it. So we inject it to system prompt, and also save it as a value that we want to use in the next step.
- In next step, we fire an MCP tool call to Linear to fetch the user profile based on the user_id, to inject the user name and email as well into context . This may not be totally necessary in the end, but it seems quite useful to have the user's name as well, and wanted to test the new context system.

 2025. Jun 19., Thu

continue `agent context` work

- add `list_calendars` tool call to gcal chap as context (covers timezones to use when doing stuff)
- migrate old `auto_context_files` to this new, more general syntax
- expose chap settings (instance config) to these context definitions
- test, test, test
- maybe even add current date-time injection also to the new general syntax (which is a hard coded enrichment now), for this I would also need context step inheritance from base agent.yaml (which is almost straightforward except I have to look into array merge behavior)

Cloudflare PoC for Platform



Cloudflare Chaps PoC

#2 · Opened by balintb

Open

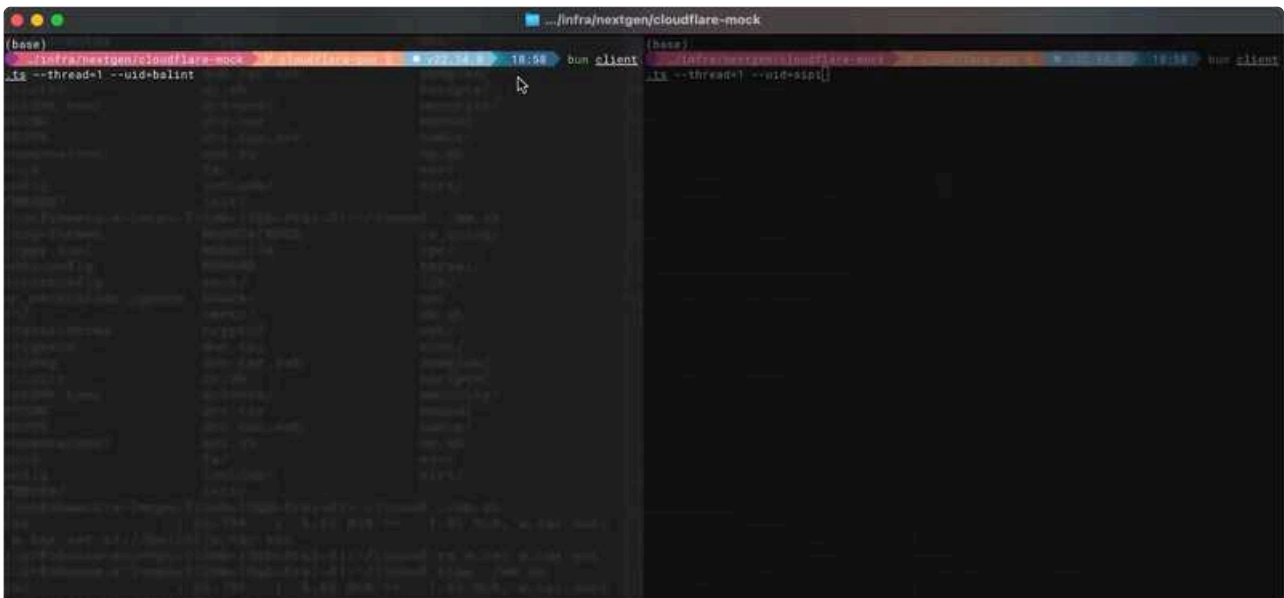
I built a PoC for the messaging and chaplet lifecycle flow detailed in my daily note from yesterday. Code in my AI tests repo. Unit and integration test suite.

The prototype uses Cloudflare's Durable Objects to sync state for a thread and a chaplet.

Components:

- **Platform worker:** like the current platform API and messaging socket server
This is deployed to <https://chaps-nextgen-poc-chat-production.accounts9469.workers.dev>
- **Dummy orchestrator:** like our current supervisor
Deployed to <https://chaps-nextgen-poc-orchestrator-production.accounts9469.workers.dev>

This PoC demonstrates how Cloudflare's DOs can be used instead of custom chat message broadcasting to create a chat room which clients can connect to via websockets. Messages are strictly ordered and the thread state is strongly consistent across edge as well.



Multiple clients connecting to same thread

Building on this PoC, I also created another version where all communications are done via Socket.IO here: <https://github.com/lukilabs/balintb-craftai/tree/cloudflare-poc-socketio/infra/nextgen/cloudflare-mock>

State overlay management

Checked benchmarks and did some calculations for different archivers using a single-processor.

- **zip** 30MB/s compress, 80MB/s decompress
deflate per file (bad perf if there are lots of files) - comp. ratio 0.65
- **tar** IO-bound only, around 200MB/s
no compression - ratio 1.0
- **tar.gz** 50MB/s compress, 150MB/s decompress
older gzip algo, but better for streaming to/from disk - comp. ratio 0.6
- **zstd** 180MB/s compress, 500MB/s decompress
modern format optimised for streaming - comp. ratio 0.55

This will need to be measured more precisely, but the bottleneck will be network to-from object storage, therefore some compression is needed. But compression is CPU-bound, so we'll have to find the sweet spot between compression level and size.

Facebook's **zstd** seems to be the clear winner - it provides almost the speed of **tar** compression, 4x faster decompression than **gzip**, with a 45% decrease in size compared to **tar**. I've prototyped the state management services on the chaplet using **zstd**, see below.

An example using the [linux kernel](#) documentation

10.596 files (docs), 78.6MiB source (58MiB tar data)

```
1  $ time tar -cf doc.tar -C Documentation/ . && time zstd doc.tar -o doc.tar.zst
2
3  real    0m0.474s
4  user    0m0.051s
5  sys     0m0.174s
6  doc.tar          : 22.59% ( 58.6 MiB => 13.2 MiB, doc.tar.zst)
7
8  real    0m0.570s
9  user    0m0.495s
10 sys     0m0.050s
11 $ find Documentation/ -type f | wc -l
12 10596
```

Archiving took ~1sec, size is 22,59% of original.

For copying to/from object storage, **s5cmd** instead of aws cli was the winner in terms of speed. It is widely used to sync multiple files to/from S3. If archiving proves to be too costly time-wise, we could explore syncing all files as individual objects - but we're probably going to be network-bound 99% of the time for normal usage, so uploading 1 file makes more sense than syncing lots.

I tested a full E2E archive, compress, upload with specs:

- Same source - 10k+ files, ~78Mb source, ~13Mb compressed
- 1vCPU
- 512Mb mem
- Source: VM in FRA1
- Dest: R2 in US-EAST

→ End to end 4 seconds.

. . .

- `drivers/` : 36k files, 1,1GB source, 128Mb compressed
- Same VM
- Same regions

→ End to end 28.372 sec. ~10x size, roughly 8,5x time

. . .

A more realistic example for a user

- 197 files, 5,9Mb source
- Same VM, regions

→ End to end 1.8sec

. . .

If the VM and object storage are in the same region, which I haven't tested, this should go down further.

I didn't go into testing download time.

My assumption based on this, is if we have hot standbys for chaplets, state syncs will be the bottleneck in terms of startup time. These tests are uploads, which is expected to be slower than download.

State overlay Chaplet PoC

To test this in a real Chaplet, I built a PoC for saving and restoring state from an object store as opposed to mounting a volume:

 **Chaplet state persistence to object store**
#55 · Opened by balintb

Open

Currently it includes logic to download and restore state on chaplet start, and save state if there's a **graceful shutdown**. It includes API endpoints for kicking this off up & down which return timing information for compress-uncompress and push-pull. I'm not sure yet what strategy would be best to ensure data is saved if a chaplet goes down without the orchestrator telling it to. Probably periodic sync - but this needs to be explored.

Scripts are ready as well, next step is to actually use this instead of the mount, which I'm going to work on tomorrow.

Tomorrow

Todo for infra planning & design

- pooling logic - the control plane to actually start a users chaplet
- move current chaplets to "stateless" → pulling state from object store on demand

After these are solved, I can put together the implementation plan. Pending Levy's availability, I'll go over the current plans with him and put together the delivery.

↑ Daily Notes - 2025 June

Jun 18., Wed



Chaps team

Balint Bartfai · Today I was planning, minor prototyping and testing out infra providers for the new infra. · First, as a...

↑ Jun 18., Wed

Chaps team



Balint Bartfai

Today I was planning, minor prototyping and testing out infra providers for the new infra.

First, as a viable alternative to Flyio, I trialed DigitalOcean.

DigitalOcean

App Platform (Docker hosting)

I managed to get the chaplet running on App Platform. I didn't run a Chap on it though. The chaplet started, connected, and was running on the most minimal resource runner (512Mb RAM, 1 vCPU).

▼ Chaplet on DigitalOcean

```
[lukilabs-chaplet] [2025-06-17 09:28:48] Loaded build-time environment variables
[lukilabs-chaplet] [2025-06-17 09:28:48] Loading runtime environment variables
[lukilabs-chaplet] [2025-06-17 09:28:53] 09:28:53 4ilogpusher | Logtail is not enabled, reason LOGTAIL_ENABLED is not set to true
[lukilabs-chaplet] [2025-06-17 09:28:54] 09:28:54 3logpusher | {"message":"Environment variables loaded","NODE_ENV":"productio
[lukilabs-chaplet] [2025-06-17 09:28:54] 09:28:54 3logpusher | {"message":"Connecting to gateway","url":"http://localhost:4000
[lukilabs-chaplet] [2025-06-17 09:28:54] 09:28:54 3logpusher | {"message":"Starting agentclient service...","context":{},"dt":
[lukilabs-chaplet] [2025-06-17 09:28:54] 09:28:54 3logpusher | {"message":"Connection error","error":"xhr poll error","context
[lukilabs-chaplet] [2025-06-17 09:28:55] 09:28:55 0gateway | {"message":"App listening","host":"0.0.0.0","port":4000,"contex
[lukilabs-chaplet] [2025-06-17 09:28:55] 09:28:55 0gateway | {"message":"AgentClient Service Connected","context":{},"dt":"20
[lukilabs-chaplet] [2025-06-17 09:28:55] 09:28:55 0gateway | {"message":"FSWatcher Service Connected","context":{},"dt":"20
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 3logpusher | {"message":"Connected to gateway control channel","context":{}
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 3logpusher | {"message":"Init existing chaps","context":{},"dt":"2025-06-17
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 2iwatcher | {"message":"Connected to server","context":{},"dt":"2025-06-17
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 3logpusher | {"message":"Init existing chaps","created":0,"skipped":0,"cont
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 2iwatcher | {"message":"Watcher ready. Watching for changes in /data/chaps.
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 0gateway | {"message":"Internal-client Connected","clientId":"rYGHrV9oVfSh
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 3logpusher | {"message":"Connected to gateway as internal client","context":
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 3logpusher | {"message":"Internal client connected successfully","context":{
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 0gateway | {"message":"Orchestra Service Connected","context":{},"dt":"20
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 1orchestra | {"message":"Connected to server","context":{},"dt":"2025-06-17
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 1orchestra | {"message":"[CRON] Initializing CronService scheduler","context
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 1orchestra | {"message":"CronService initialized","context":{},"dt":"2025-06
[lukilabs-chaplet] [2025-06-17 09:28:56] 09:28:56 1orchestra | {"message":"[CRON] Started 0 cron jobs","context":{},"dt":"2025
```

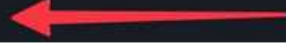
DigitalOcean runs with different configs than Fly, but I didn't have to write a custom config, only pull some levers and turn some knobs, and the container deployed from Github CR.

- ▼ App platform containers do not allow for volumes to be mounted → we would only be able to use the setup / teardown method for persistence.

Storage Limits

- The host instances running App Platform containers do not provide persistent data storage. Data in the host instance's local filesystem is permanently lost after deployments and other container replacements. The local filesystem is additionally limited to 2GiB, and if it is filled to capacity, the container is detected as unhealthy and replaced.

Apps should only use the host instance's local filesystem for small amounts of temporary storage. For persistent storage, you can use [Spaces Object Storage](#) or [DigitalOcean Managed Databases](#).

- File uploads to apps timeout after 100 seconds.
- App Platform does not support volumes. 

Droplets (VM hosting)

- custom image possible, but we would need to change it to run with `cloud-init` (haven't explored complexity)
- we would also need to build for amd64 and arm64 as well. A [github actions workflow](#) was running for **1,5hrs** but managed to build for both platforms
- ~~shared CPU. Can't find information on throttling, but since it is a shared resource there's a good chance it's valid~~ - this is not true. Larger providers can sustain 100% for days based on docs and forum feedback. Fly has less capacity, quota system
- **we are still billed for stopped droplets** - so we can't have a dedicated VM per user, which is stopped, we would need pools

Overall: DO Droplets seem like a viable alternative to Fly if we can

- solve pool size, possibly by automatically starting and destroying droplets
 - this is still "wasteful", as in we have idling resources, but not as bad as having a VM per user
- setup / tear down persistence

Hetzner Cloud, Vultr, Scaleway

Did not do in-depth testing with actual VMs, However, these providers all charge for stopped VMs, which will be cost-prohibitive if we want to start/stop VMs per user.

Scaleway is EU-only.

Persistence strategy

I took a closer look at our options for persistence. The two viable options we have are to either mount via block volumes, or setup / tear down execution environments (the chaplets FS). As detailed in my daily note yesterday: volumes are problematic - tied to regions, not mountable everywhere. Network volumes don't guarantee the same file ops as a locally mounted volume (locks, perf., stat(), etc).

For setup / teardown, we can either go with a syncing strategy - `rsync` or similar - or an archived *overlay*. By overlay, I'm referring to a snapshotted `state.tgz` stored in object storage.

I'm in favour of the setup / teardown strategy with an overlay. By storing state in object storage, but using the local FS for file ops, we get

- replicated, fast state storage. Setup is fast and easy (transfers from S3/R2 are *fast in-region*)
- if the user is not storing gigabytes or 10k+ files, archive / unarchive is fast and can be done on a ramdisk
- restore, backup, etc. becomes simple without syncing individual files
- we keep FS ops

Having this persistence strategy in mind, the VM layer also becomes vendor-agnostic. If chroot-ed properly, in the future we can also move on from the one VM per user concept.

Added complexity comes from locking the state. We can't allow concurrent execution on different runner VMs, only different chaps on the same chaplet.

Cloudflare conceptual design

I've modified the previous design from an agent executor worker to include VMs as the execution layer.

Note: this does include vendor (Cloudflare) lock-in! However, the edge logic would be the same on any edge computing provider.

The design allows us to scale horizontally, and in the future, to different regions for chaplet VMs. Chaplet VMs are provider-agnostic. All 'expensive' operations are designed to run on the edge: user sockets and all user-facing APIs are handled by DOs and Workers which can scale. Socket connections from Chaplets are bound by the number of active chaps - this could be multiplexed in the future.

Components

I've tried to keep the "warm-path" latency minimal, and the "cold-path" under a couple of seconds.

Pooling logic is excluded. This still needs to be planned properly. I'm thinking we should have hot standbys running, or use an app hosting platform (eg. DO).

Previous components unchanged: Platform DB, Auth DB.

- **Worker (endpoint):** Similar to Platform, the app connects here and can call RESTful endpoints. The worker is in charge of auth, fetch() -ing resources, and coordinating DOs (see below).
- **Thread [Durable Object](#):** runs on Workers runtime. The Thread DO is where a client connects to receive messages. The connection to this DO is handled by the platform worker. DOs order messages, they are in charge of fan-out to multiple clients listening on the same thread.
 - strongly consistent in-order delivery, broadcast
- **Chaplet Durable Object:** locks chaplet state (overlay), provides chaplet VM state, provides thread list for chaplet.
 - SSO for chaplet life
 - Control-plane lock for chaplet
- **Life-cycle workflow:** [CF workflow](#) managing chaplet routing, setup / teardown, timeouts.
- **Orchestrator API:** this manages the actual chaplet VM start stop, pooling.
- **Overlay object storage:** contains users state archives (R2 if on Cloudflare)
 - Decouples state from compute. We pay for state per user, compute per pool node

All WS comms can stay on Socket.io.

Message send (user → chap)

- client POST to thread with message
- req hits worker, gets forwarded to the thread DO
- thread DO writes to DB (guaranteed order), broadcasts to all listeners
- if no chap sockets present, thread DO wakes chaplet DO → this allocates, starts chaplet VM. If chaplet is awake, this socket is connected (from agentclient)

Chap receives message

- broadcast from thread DO hits chap's websocket
- chap decides whether to respond by calling the platform API - POST thread messages

Chap sends message

- POST thread messages
- hits worker, worker sends to thread DO, thread DO writes history, broadcasts

User downloads widget

- GET /chaplet/.../widget
- worker asks chaplet DO for status of chaplet
- chaplet alive:
 - DO responds with VM address
 - worker streams / fetch() -es, proxying request to chaplet possibly on internal network
- chaplet cold:
 - DO triggers wakeup, returns HTTP 202
 - worker retries and sends when ready

Chaplet startup

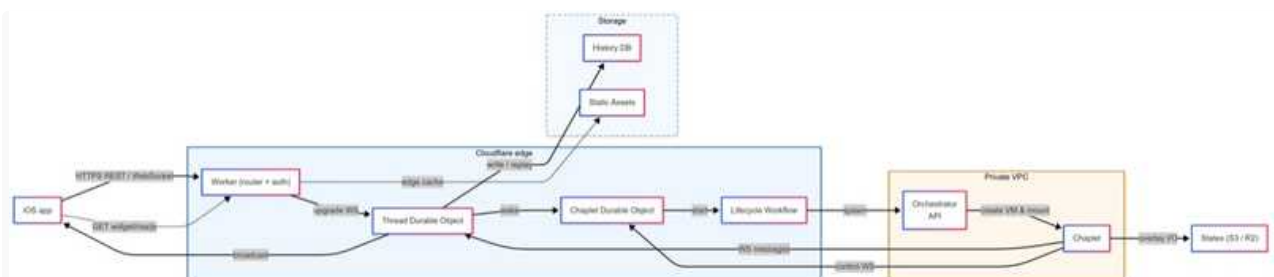
- chaplet DO receives wakeup call from thread DO or worker
- chaplet DO starts workflow instance - sends workflow params on which threads active
- workflow calls orchestrator to spawn / allocate VM from pool - passing params
- orchestrator starts VM with params on where to find state and thread connections
- chaplet internal:
 - state download, unarchive, FS setup
 - chaps open websockets to each thread DO
- chaplet sends ready to chaplet DO
- workflow marks chaplet status as alive

Chaplet idle stop

- thread DO has last user timing for thread
- after timeout, sends timeout to chaplet DO
- chaplet DO tells workflow to end
 - orchestrator called to kill chaplet
 - chaplet state is saved and pushed to store
- chaplet DO sets status off

Any later message restarts wake process.

Diagram



Incremental build & rollout

At a high level, this would be the plan to roll out the nextgen infra.

1. State management: We can move to archived states without disturbing current flow.
2. Supervisor pooling: modify current pooling logic to start dedicated CPU VM on fly without volumes, only state overlay
3. Modify supervisor to use other provider, allowing shared CPUs. At this point our Fly.io reliance is only for more lightweight request-based services
4. Move platform REST to the edge, keep websockets on current platform (proxied)
5. Add lifecycle workflow to start/stop as needed, waking on messages (keeping it alive). This is done by sleeping the workflow for N days after each wake (by platform REST worker), and once the workflow wakes from a sleep due to timeout, kick off the chaplet teardown process

6. Redo websocket handling with DOs

Next steps

- Will need to prototype on Cloudflare if we agree on this architecture
- I still need to hash out the VM control (+ low-level infra) layer, including provider selection

Merged a set of fixes for Google Calendar Chap



GCal Chap fixes

#53 · Opened by Schipy · Merged by Schipy

Merged

- add rsvp support to `update-events` tool (`responseStatus` prop)
- add `calendar.readonly` scope as well to access available calendars and be able to use `get-freebusy` tool
- prompt chap to know the timezone before acting (a quick fix, will be solved with other means in the future)
- **introduce a context enriching `_tool_learnings.md` file, for additional API learnings and instructions on top tool descriptions provided by MCP server (for now a static set provided by us, but later chap itself may extend it over time as it uses the MCP).**
 - as a first use case, help the gcal chap use the `search-events` tool with `_tool_learnings` (as the `query` prop's syntax and exact behavior is not described, not even by the [official google calendar API docs](#)).

Tool Learnings

Expanding on this a bit more as I think this is an important concept

I've seen now many examples when the MCP api is not completely described, besides the above mentioned gcal list `query` field, the same is true for Linear's `list_issues` tool `query` parameter ("An optional search query"), also seen one of the apis miss to indicate a field is mandatory etc.

While MCP APIs will surely get better over time, plus we can also fixup and contribute to some MCPs specifically, it seems clear to me to have Chaps scale to a regiment of MCPs, we need a simple way to amend these issues by future Chap developers, and even more desirable, have the LLM automatically explore and fix these gaps for itself. This can take several forms:

- runtime learning, where the LLM is instructed to take notes of api errors it recovered from, or undocumented behaviors it experiences, so a chap instance will actually get smarter over time. We will also collect these learnings from the wild and later ship new Chap versions already with the learnings (that earlier instances gathered)
- it may also work in some cases to have the LLM "scan" the api, and make deliberate tests, eg this is what I actually did with the gcal search query, where I instructed the chap to explore how the property works and summarize it for me, it got it not perfect, but fairly right (as I also tested it manually with Postman) and this will only get better.



LLM self-learning the usage of `query` property

I prompted the chap agent itself to learn the search-events query parameter · User: ◀ I would like you to explo...

- verification of the learnings hints: After some testing with search learnings in place I concluded in this form it seems GPT 4.1 often still does not take into consideration (maybe needle in haystack problem in lot of instructions), but tested with `o3` and that one already leverage it better and could search events more successfully, so I think pattern will become more and more efficient, as models get better

Also worked on having e2e tracing for prompt processing across all services, plus in general fixup the Orchestra logs to appear in BetterStack (json formatted and context enriched)



chore(logging): Tracing prompt requests

#54 · Opened by Schipy

Merged

- on incoming realtime message, `traceld` is accepted from `metadata.traceld` or **generating a new**
- added to async local storage
- sent in prompt websocket message to orchestra service (and back)
- passed over to Python process
- **Python process gets same logging helper as TS code (formatting logs to json, enriching with context, severity filtering)**
- `traceld` is passed back and also resumed in async local storage when prompt messages are received in agent client
- **`traceld` is emitted on chat message by chap as `metadata.traceld`**

📅 2025. Jun 18., Wed

test e2e the tracing PR above, and bring it home.

As next topic, based on recent testing, I would like to experiment with a similarly important concept as the tool learnings, which is something we could call **remote data caching or tool call caching capability** in the Chaps.

I didn't have now time write it up, but what I see is that the missing context what I complained about on Friday's testing, both for GCal and Linear, could be solved by actually mapping some of the remote data into the Chap as cached data and use it to enrich context (eg in gcal user's calendars with their default timezones, in Linear the user id, and projects statuses with their ids etc). These are pieces of data that are very useful for building later successful queries on top, and it is both

- very inefficient to always refetch them when processing a user message
- and also very error prone as LLMs often instead of fetching the prerequisite data for certain tool calls, just try with inferior queries and then fail to fulfill the intent of the user

↑ Jun 18., Wed · Chaps team

LLM self-learning the usage of query property

62

I prompted the chap agent itself to learn the search-events `query` parameter

User: < I would like you to explore and figure out how the search-event tool's query parameter works. Eg case sensitivity, is it a prefix search, does it support multiple keywords, keywords with AND/OR etc?
have a meeting next week called "Monthly live presentation", use that as a test subject. Devise and execute test cases until you have a confirmed model on what the API supports

> [t] search-events

Testing if exact case and full phrase match returns the event

```
{"calendarId":"primary","query":"Monthly live presentation","timeMin":"2025-06-23T00:00:00+02:00","timeMax":"2025-06-30T23:59:59+02:00"}
✓ "Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart: 2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction), schipy@craft.do (accepted)\nReminders: Using default\n{"type":"tool_result","tool":"search-events","result":"Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart: 2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction), schipy@craft.do (accepted)\nReminders: Using default\n","agent_id":"main_agent","timestamp":"2025-06-17T09:04:44.068081"}
```

> [t] search-events

Testing if lower-case full phrase match returns the event (case sensitivity test)

```
{"calendarId":"primary","query":"monthly live presentation","timeMin":"2025-06-23T00:00:00+02:00","timeMax":"2025-06-30T23:59:59+02:00"}
✓ "Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart: 2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction), schipy@craft.do (accepted)\nReminders: Using default\n{"type":"tool_result","tool":"search-events","result":"Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart: 2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction), schipy@craft.do (accepted)\nReminders: Using default\n","agent_id":"main_agent","timestamp":"2025-06-17T09:04:44.265477"}
```

> [t] search-events

Testing if a single word (partial) returns the event (prefix/substring search)

```
{"calendarId":"primary","query":"Monthly","timeMin":"2025-06-23T00:00:00+02:00","timeMax":"2025-06-30T23:59:59+02:00"}
```

✓ "Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart: 2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction),

schipy@craft.do (accepted)\nReminders: Using default\n{"type":"tool_result","tool":"search-events","result":"Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart:

2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction), schipy@craft.do (accepted)\nReminders: Using

default\n{"agent_id":"main_agent","timestamp":"2025-06-17T09:04:44.454450"}

➤ [t] search-events

Testing if a different single word from the title returns the event (substring search)

```
{"calendarId":"primary","query":"presentation","timeMin":"2025-06-23T00:00:00+02:00","timeMax":"2025-06-30T23:59:59+02:00"}
```

✓ "Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart: 2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction),

schipy@craft.do (accepted)\nReminders: Using default\n{"type":"tool_result","tool":"search-events","result":"Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart:

2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction), schipy@craft.do (accepted)\nReminders: Using

default\n{"agent_id":"main_agent","timestamp":"2025-06-17T09:04:44.623124"}

➤ [t] search-events

Testing if partial phrase match (not full event title) returns the event

```
{"calendarId":"primary","query":"Monthly presentation","timeMin":"2025-06-23T00:00:00+02:00","timeMax":"2025-06-30T23:59:59+02:00"}
```

✓ "Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart: 2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction),

schipy@craft.do (accepted)\nReminders: Using default\n{"type":"tool_result","tool":"search-events","result":"Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart:

2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction), schipy@craft.do (accepted)\nReminders: Using

default\n{"agent_id":"main_agent","timestamp":"2025-06-17T09:04:44.801665"}

➤ [t] search-events

Testing if logical AND is supported in query

```
{"calendarId":"primary","query":"monthly AND presentation","timeMin":"2025-06-23T00:00:00+02:00","timeMax":"2025-06-30T23:59:59+02:00"}
```


✓ "" {"type":"tool_result","tool":"search-events","result":"","agent_id":"main_agent","timestamp":"2025-06-17T09:04:44.967132"}

> [t] search-events

Testing if logical OR is supported in query

```
{"calendarId":"primary","query":"monthly OR presentation","timeMin":"2025-06-23T00:00:00+02:00","timeMax":"2025-06-30T23:59:59+02:00"}
```

✓ "" {"type":"tool_result","tool":"search-events","result":"","agent_id":"main_agent","timestamp":"2025-06-17T09:04:45.089809"}

> [t] search-events

Testing if two adjacent words (partial phrase) returns the event

```
{"calendarId":"primary","query":"Monthly live","timeMin":"2025-06-23T00:00:00+02:00","timeMax":"2025-06-30T23:59:59+02:00"}
```

✓ "Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart: 2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction),

schipy@craft.do (accepted)\nReminders: Using default\n"" {"type":"tool_result","tool":"search-events","result":"Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart:

2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction), schipy@craft.do (accepted)\nReminders: Using default\n","agent_id":"main_agent","timestamp":"2025-06-17T09:04:45.258322"}

> [t] search-events

Testing word order sensitivity in the query

```
{"calendarId":"primary","query":"live Monthly","timeMin":"2025-06-23T00:00:00+02:00","timeMax":"2025-06-30T23:59:59+02:00"}
```

✓ "Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart: 2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction),

schipy@craft.do (accepted)\nReminders: Using default\n"" {"type":"tool_result","tool":"search-events","result":"Monthly live presentation (4vvvtb26pi29lgtg329aqimkg1)\nStart:

2025-06-24T11:00:00+02:00\nEnd: 2025-06-24T12:15:00+02:00\nAttendees: orsolya.frecska@craft.do (accepted), all@craft.do (needsAction), schipy@craft.do (accepted)\nReminders: Using

default\n","agent_id":"main_agent","timestamp":"2025-06-17T09:04:45.422424"}

```
{"type":"end_tool_use","content":"Tool usage complete","agent_id":"main_agent","timestamp":"2025-06-17T09:04:46.185723"}
```

text: 🔍 How the Event Search Query Works

Here's a confirmed summary of how the search-event tool's query parameter works, based on tests with your "Monthly live presentation" event:

- **Case insensitive:** You can use any combination of upper or lower case; results will be the same.
- **Space-separated keywords:** All words you enter must appear somewhere in the event (title, description, etc.), but they can be in any order and do not have to be adjacent.
- **Partial and substring matching:** You can search by any single word or partial phrase, and as long as all the words appear, the event will match.
- **No support for AND/OR:** Boolean operators like `AND` and `OR` are not supported; they will not change or broaden your results.
- **Word order and adjacency:** The order of keywords does not affect the results, nor do they need to appear together in the event title.

Best practice: Enter all relevant keywords separated by spaces. Avoid using Boolean operators.

If you want to explore more—such as searching within event descriptions or attendee lists—I can help you design new test cases.

I added one more important test case, and further compressed this with o3, and this is how I got

search-events.query: Case-insensitive search matches exact words (no prefix matching); every space-separated term must appear (implicit AND); no Boolean operators; term order irrelevant.

plus I added manually one more tip:

"Watch out for user abbreviations/mistypes, carefully consider, if query is user given, and use only reliable keywords or try multiple variations of user given keywords if necessary."

↑ Daily Notes - 2025 June

Jun 17., Tue



Chaps

Balint Bartfai · Interview · Onsite for TPM candidate with Sipi. · Infra rethink · I took a couple of steps back to review...

↑ Jun 17., Tue

Chaps



Balint Bartfai

Interview

Onsite for TPM candidate with Sipi.

Infra rethink

I took a couple of steps back to review our options for a more resilient infra layer given the constraints.

Persistent volumes with filesystem ops are a constraint we must build into the execution layer of the chaplet. Therefore, executing the LLMs for Agents becomes tied to a VM, which also means we are in charge of the complete orchestration of executor VMs. This adds some complexity.

Patterns we could use

- Multi-tenant, single (=unified) persistent execution node (pool)

By having one type of node for execution, we can scale it out horizontally. Complexity comes from isolating and *potentially* moving user data around. Mounting volumes for user data could be a possibility - which again requires orchestration of where a volume is mounted, which executor node handles the request, etc. This solution allows immediate executor code updates.

- Single-tenant, persistent, distinct execution nodes

Pretty much the same architecture as we have now with the Chaplet, one per user. This is pretty wasteful, as we'd have idling VMs. Some intelligent orchestration could be built, where these nodes are snapshotted and then put to sleep (~ stopping VMs on Fly.io). A drawback is the redeployments needed when underlying execution code changes.

- Single-tenant, on-demand execution nodes

VMs capable of running chaps, which are built and teared-down "on demand", with grace periods after the last op to them. Conceptually this is the least wasteful, only running compute resources when they are truly used. We could even orchestrate this per-chap, which would allow "hot" chaps to stay online, while a user's other chaps idle with their state stored. However - this would almost certainly require shuffling user data around if we keep the concept of one user = one "volume", or would require volumes to be mounted on multiple nodes.

Persistent storage

This is by-far the hardest to migrate later if we're tied to block storage (volumes).

With on-demand execution, there will have to be a workflow orchestrating the setup and tear-down of agent state (= user data). I've been looking at different alternatives for us to not have to do a lot of heavy lifting, such as

- network mounts
 - instead of mounting a volume directly to the VM, we use NTFS/etc to read user data. I'll need to explore this more.
- S3-compatible filesystems
 - this is a big no-no, random access is slow, editing a file requires a full reupload *unless* partial streaming is implemented in the FS driver. The general consensus seems to be don't do this.

The simplest solution to user data seems to be persisting volumes and attaching them on-demand. All the providers I've looked at so far (DigitalOcean, AWS, Render.com, Azure) tie volumes to one region, meaning the execution node will have to be in that same region - we're stuck in that region unless we migrate the volume and bring up a new executor node.

Cloud-agnostic storage options

There's another potential solution, which would require storing user data as a snapshot in, eg. S3/R2, and copying on VM start. The benefit we get is our execution becomes region-agnostic, however the burden of keeping this data "safe", synced to an external object storage, is on us.

Using block storage (=volumes) does not allow the agent to be executed anywhere on the edge, primarily due to edge functions not being allowed to read/write the FS. To operate on the edge, all state would need to be stored in a centralized storage somewhere, which would need some rewriting of the agent execution logic.

Rsync could be used for setup/teardown with the current storage box we have. This way no volumes would need to be mounted at all, pulling data from file storage onto the VMs ephemeral storage. Risk of data loss is higher - rsync can be run periodically on the VM, but due to performance we can't run it continuously, only at intervals.

Another option is using the git remote, committing every file change and pushing to origin. This would also allow a full history. Some calculations I did a while back showed that the repo, if containing binary files, grew very fast.

Execution

Another challenge we face with our current Chaplet architecture is how execution is tied to the VM itself, and can't be run as a function. Orchestra is in Python, services on the chaplet run on Typescript.

A potential solution here would be to detach agent (Orchestra) execution from the runner itself, which is not hosted on the chaplet. This way we'll run into the same issues around persistence setup and tear-down.

Widgets are an interesting problem to solve. Having them hosted on the chaplet allows widgets to operate with the same state as the chap. However, hosting them means the chaplet's HTTP server must be aware of different routes, which adds some complexity to a "shared" (multi-tenant) execution node. We could use the vhost concept (Apache/nginx) and route by host (DNS subdomain), allowing widget routes to only be called on a subdomain pointing to the chap itself. This removes the need for per-chap HTTP listeners.

Agent SDK and connections to Platform

The Agent SDK maintains a persistent websocket connection to Platform to listen to incoming messages. Replacing this with webhooks could work, or pulling jobs from a queue can also be viable. The question becomes where do we listen for these events?

One solution I've been looking at is using [queues](#) on Cloudflare which would kick off [Workflows](#). This still needs exploration and prototyping, but a potential flow would be

- Listen on job queue for inference jobs. This does not have to be a long-running process, just a trigger defined

- Once a job comes in, **start** or **wake** a workflow which takes care of setting up the execution environment as its first step if *started*. If *waked*, there is already an execution env setup
- then running the agent execution - on a VM somewhere
- then sleeping until either a new job item arrives **OR** a certain number of time has elapsed
- on timeout (which means no inference jobs for a while) the workflow takes care of tearing down the execution VM

The executor VM would use Platform APIs to send messages if needed, which could then be stored in a threads Durable Object. This DO is what the end clients (app) connect to and receive messages from.

In this scenario, orchestration becomes starting and stopping a VM, which is handled by the workflow. Routing requests to that VM is handled with a worker, either reading routing from edge KV or using a subdomain to reach the chap (auth'd).

Host infra

Deciding which infra provider we choose depends a lot on the execution layer and how much we want to solve ourselves. Options are

- managed PaaS (App platform, ECS, etc) ← I'm in favour of this approach for AI execution
- running our own VMs (or VM, if we go with shared execution)
 - potentially Kubernetes for orchestration
 - this gets complex fast, even with K3S on top of Firecracker. We'd essentially be building our own Fly.io
- edge functions. Though we can't run everything there, we can run most of the APIs, including VM orchestration, on the edge

Explored *some* potential options for hosting the services. This is in no way complete.

- DigitalOcean
App platform can run Docker containers. Autoscaling.
- Render
Web Services can run Docker containers. Different service types available.
- AWS ECS
- Azure AKS
- Google GKS
- Cloudflare

Provider	Execution (containers)	Edge	Persistence opts
DigitalOcean	App platform PaaS. Shared and dedicated CPU	Functions	Block volumes 0.10 USD/Gb/month. Object storage
Render	Web Service containers on VMs (shared? dedi?)	None	Block storage 100Gb 25USD. Object storage

AWS ECS	Fargate	Lambdas	EBS, EFS, S3
Azure AKS	Managed K8s	Azure Functions	Azure Disk, object storage
Google GKE	Managed K8s	Cloud Functions	GCE disks, object storage
Cloudflare	Workers. Containers coming soon.	By design	R2 object storage only, D1, KV DBs

More to look at: <https://www.scaleway.com/en/serverless-containers/>, Fly.io with dedicated CPU for pooling executors, Hetzner.

TL; DR, my current thinking, will continue exploring from here

Least amount of orchestration, with horizontal scaling.

Queue every user job (message); a stateless Worker consumer either

- *(calls Worker / Workflow for light tasks not needing FS persistence, where an LLM is called, or)*
- runs Workflow: spins up micro-VM from an up-to-date snapshot *if not already there*, streams the chap's object-store working set to VM FS, runs the agent - this way **FS operations are fine** - stores results, and sleeps/exits

Spinning up a VM would either be done using current pooling logic, or just creating a new VM, depending on capabilities of hosting.

Chat history and small config live in a per-chap Durable Object (and DB!), which is streamed to clients on connect. Widget HTTP traffic is routed by hostname through an edge Worker. No volumes, no idle VMs, best of both worlds.

 **Peter Sipos**

morning interview then Chaps sync discussions on infra and chap logic

In the afternoon worked on dynamic Chap agent configuration (for testing & fast learning, and later who knows...)



feat(config): dynamic agent configuration

(also included a pack of cleanups and renames to make the agent config a bit more stable and uniform across Chaps)

created a more permanent writeup in Craft KB about concepts and usage



Chap configuration

Glossary · A Chap has the following type of configurations · Agent config: description of the LL...

Last Edited 6/17/2025

Heads up:

- just finished the writeup and did not have time for an extra test round on prod, will do tomorrow morning
- needs new chaplet and newly created chaps (redeployed chaplet for myself and BalintO)

In action:

Gateway / Chaps / Update dynamic chap agent configuration

POST{{ROOT}}/api/chaps/chapid/dynamic-agent-config

Send

ParamsAuthorizationHeaders (10)BodyScriptsSettingsCookies

Query Params

Key	Value	Description
Key	Value	Description

Path Variables

Key	Value	Description
chapid	chap_2ac5d340-5b6a-4e80-bb6b-b0b21443b57a	Description

BodyCookiesHeaders (5)Test Results

200 OK · 7 ms · 521 B

Save Response

JSONPreviewVisualize

```
1 {
2   "success": true,
3   "message": "Dynamic agent configuration saved successfully",
4   "updatedConfig": {
5     "prompts": {
6       "main_agent_system_prompt_addition": "You are a test agent, always include the word bazinga in your response",
7       "communicator_agent_system_prompt_addition": "You are a test agent, always include the word bazinga in your response"
8     }
9   }
10 }
```

Chat with todo_chap (chap_2ac5d340-5b6a-4e80-bb6b-b0b21443b57a)ischipy@craft.do | id: 2v4j6n...

```
[{"type": "text", "text": "A new document has been added to record your preferences. You can now keep track of your preferences in one place. (bazinga)", "role": "assistant"}, {"type": "text", "text": "If you want to update, review, or add preferences, let me know how you'd like to proceed.", "role": "assistant"}, {"type": "text", "text": "A new document has been added to record your preferences. You can now keep track of your preferences in one place. (bazinga)", "role": "assistant"}, {"type": "text", "text": "If you want to update, review, or add preferences, let me know how you'd like to proceed.", "role": "assistant"}]
```

Type /f to browse files, /i to send an image, /q to quit, /u to manage your account, /s to switch chap, /m to manage your email

2025. Jun 17, Tue

I have a range of Chap logic fixes on plate for the current chaps based on [last Friday's testing](#).

↑ Daily Notes - 2025 June

Jun 16., Mon



Chaps

Levy Melnikov · Fri, Jun 13 · Chap Registry · I finalized the chap registry CLI tool, deployed it and published first cha...

↑ Jun 16., Mon

Chaps

 Levy Melnikov

 2025. Jun 13., Fri

Chap Registry

I finalized the chap registry CLI tool, deployed it and published first chap types. Clients can get a list of all known chap types via `GET {{API_ROOT}}/api/v2/chapTypes` (Also in Postman). I've also prepared a small writeup, but if you want to use \ extend something and it's not clear, please ping me:

 CRAFT

Chap Registry

Centralized Login

To hit the ground running I started exploring the project docs from services team:

 CRAFT

Centralized Login

 2025. Jun 16., Mon

I will join services to implement Centralized Login: by default I won't attend standups. But please don't hesitate to reach out to me if there's anything I can help with.

Chaps Frontier Infra



Chaps Frontier Infra

Approaching the alpha/beta phase of Chaps, it has become apparent that runtime characteristics o...

Last Edited 6/16/2025

Took a large step back to review the whole infra stack. Now that the clients can be tested E2E, next week will be planning and prototyping our nextgen ("Frontier") infra.

I put together where our current infra is lacking, and drafted a conceptual architecture. It's planned with Cloudflare, but the core concepts could be used elsewhere. Everything runs on the edge.

Updated [delivery plan](#) for next week and the week after. Levy will be available from around Thursday to work on Chaps.

Chap install

Finished and awaiting deploy Monday so nothing breaks for potential weekend integration work. The service and tooling is fully covered with automated tests, but E2E might be different - as we have seen.

Mastra prototype



lukilabs / chaps-mastra-proto

Proof of concept for Chaps on Mastra

lukilabs TypeScript Stars 0

Put together a proof-of-concept to test if Mastra can be used with dynamic configs like Orchestra.

It supports JSON configs, Mastra workflows, MCP calls and multi-agent setups with specified endpoints. So far I'm liking Mastra. Will need to test observability and evals as well.

For new code in Chaps, and this prototype, **we are building with a full suite of integration tests.**

Candidate reviews

Reviewed candidates for the AI Quality Engineer role. 1 will be progressing to the interview stage with me.

Next Week

Next week will be thorough planning and prototyping Chaps frontier infra from the Chaplet layer up.



Chaps

Levy Melnikov · Thu, Jun 12 · Chap Registry · I've finalized the backend. We now have a "Chap Store" that can list all...

↑ Jun 13., Fri

Chaps

 Levy Melnikov

 2025. Jun 12., Thu

Chap Registry

I've finalized the backend. We now have a "Chap Store" that can list all known chap types and their version. It also allows management of those chap types: creating new versions and uploading bundles to them.

I also created a [script](#) for pushing chap types to the store from our chaplet repo. I will later integrate it with CI\CD.

I took some shortcuts with auth. There's an API Key that will allow any action(reading the bundle, updating \ deleting \ creating) for any chap type. For now we can just put this key as a secret in all of the components and figure out fine grained controls later.

Fly.io Limits

I took a closer look at issues with Balint's chaplet. Turned out that fly.io is aggressively throttling the CPU on the chaplet. The solution is:

- Now on prod we are using full(non throttled) CPU instances and on sandbox shared(throttled) CPU instances. Full cpu instance is ~\$35 \ month.
- I've already redeployed Balint's machine and you can use redeploy API to get the non shared CPU on prod.

This is a temporary solution so that we can test chaps with current infra. We now have time to explore a more robust infra for chaplets \ chap execution.

 2025. Jun 13., Fri

Chap Registry

There's a bunch of missing pieces

- Add documentation(as I will be mostly in the beginning of the next week)
- Push all of our chap types to the store on prod
- CI\CD for automatically updating the chap types
- Provide a minimal PoC on how to "download" latest chap type in chaplet on chap creation

 Peter Sipos

admin: ooo on this Friday officially, but will do a few hours. Will try to check on Slack from time to time, but note that I cannot guarantee quick response times

it was a bit fragmented/loaded day with interview, washup, PSR, and chaplet machine overloading investigations, so only small progress in Chaps logic

- added new-thread-welcome and respond-to-completed-setup logic to all chaps
- changed the gcal chap to edit calendar rights (fixed scope, later will be something the end user can discuss with the Chap itself)
- under the hood: renamed the agent config.yaml files to `agent.yaml` (backward compa way), which is a more fitting name
- smaller prompt improvements, eg injecting the current chap's main function to the generic meta/support agent, so it is more in picture what chap it is providing support for.

Next:

- adding a GMail Chap (actually, already did much of it, but could not test it as Google Console was down at the time), will revisit tomorrow morning
- test current Chaps and gather insights on their behavior

 Balint Bartfai

4h meetings + prep

Interview

Interviewed a candidate onsite with Sipi for the AI TPM role. This was the first time we used LLM-specific tasks in an onsite.

Chap sharing

We agreed with Balint that chaps can be “shared” initially by customising config (Agent SDK) and pushing that to the chap store. There will be no sharing of an individual chap instance across users, except when these users are in a thread with the chap owner.

Chaplet CPU throttling

Balint's chaplet was throttled due to high CPU, depleting the allocated CPU quota to the firecracker VM. This happened when a chap was installed, leading to a jump from 5% consistent CPU usage to a spike of 23%. The spike was interesting, user CPU usage only jumped to 9% (gateway on the chaplet), sys CPU was low, iowait was up to 12%. This indicates the CPU was idling, waiting for an io op to complete - either network or disk.

From my investigation, in one case the network jumped for ~20sec from 3-6b/s to 2,6Mb/s. This seems to indicate the chap was installed (package downloaded and built). A jump in network io was enough to raise iowait to the point where our cpu quota was exceeded.

The implication of this type of throttling is that **we can't rely on fly.io for spikey traffic in production**. Being throttled paralyses the chaplet into a state where it does not recover at all. These characteristics of chaps were not known until the whole system was run E2E. The only upside: this degradation was limited to a single user due to resource isolation with VMs.

Short-term bandaid was to increase CPU count to 2. This only pushed the problem further, as any messages sent - which ran orchestra - consumed CPU as well, and our quota was slowly running out.

We settled on a temporary solution where we will give the chaplets a dedicated CPU, for 3x the cost, for the duration of the alpha/beta. This is cost-prohibitive in the long run. The duration of the closed beta will give us time to rethink the infrastructure below the API layer.

Housekeeping

- Started on chap update on chaplet startup. This will be finished tomorrow.
- Moved our Neon DB to a prod instance, with dedicated resources on a paid plan (19USD / month)

Ahead

For this week we will be 100% focusing on releasing the MVP. Next week our priorities on the infra layer will shift towards replanning the layers under (and potentially in) our APIs.

↑ Daily Notes - 2025 June

Jun 12., Thu



Chaps Team

Peter Sipos · merged 📌 · feat(chap-lifecycle): add run prompt as possible action, new thread and user step compl...

↑ Jun 12., Thu

Chaps Team

merged 📌



feat(chap-lifecycle): add run prompt as possible action, new thread and user step complete ho...

#42 · Opened by Schipy · Merged by Schipy

Merged

Refined lifecycle execution

- moved `LifecycleExecutor` from `shared` to `gateway` (simpler dependency injection, as it needs now some stuff that is available in gateway)
- extracted a `LifecycleExecutionContext` to encompass what it needs to execute actions, for example access to sending message agent client. By default it is available on `global` within gateway process, so don't have to worry about it much.
- and expose a new endpoint `/api/chaps/{chapId}/run-lifecycle`, where all internal processes can trigger lifecycle events (eg orchestra the pre-agent-execution hook), and also opens up the possibility to define lifecycle events that are triggered by the chap client (native app)!

New lifecycle actions: `run.prompt` and `run.prompt.chat`

- `run.prompt` is a simple prompt invocation via `InternalClient`, which is for internal background invocations that do not need to be emitted in chat, not used right now yet, but can imagine a few use cases, eg trigger processing on some content in the background, and was super cheap to add
- the more interesting `run.prompt.chat`, which is for executing a prompt "on a thread". This is achieved by hooking into agent client via cross-service socket messaging: we relay a message over to agent client, which assigns a thread to the invocation (currently supported strategies are specifying an exact `threadId`, or `last_used`), and then sends the prompt via the normal chat message execution flow, meaning its results/realtime status etc are forwarded to the thread. If we want we can later control the emit behavior via some flags, eg may not want to send realtime status, or not use the thread history in some use cases (all this would be a config in the `run.prompt.chat` action and consequently in the agent client execution)

New `new_thread` lifecycle hook

we handle now `thread:created` coming from realtime messaging, and fire a `run-lifecycle` with `new_thread`

one tricky part here is how to match up the `threadId` - I introduced a concept where lifecycle hooks can have `args`, eg a `new_thread` would have a ``threadId``, which can then be interpolated into the `run.prompt.chat` action as thread target.

```

1 new_thread:
2   - type: run.prompt.chat
3   prompt: "You have been added to a new thread. Welcome the user and offer if
4     there's any setup to do before the application can be used."
5   threadTarget:
6     type: threadId
7     id: "${args.threadId}"

```

New `user_step_change` lifecycle hook

We also want to trigger the chap when user completes a setup step. Here the tricky part is we have a method to dynamically calculate the completed state of a step, but not a saved state whether they're complete, which is very robust, and avoids state duplication etc. I did not want to change this yet (though if we must at some point, that could also be ok), so ended up with a `withUserStepChangeDetection` higher order helper function, which can execute a piece of code with user step state detection and `user_step_change` execution wrapped around it (checks before and after and triggers lifecycle if changed)

```

1 await lifecycleExecutor.withUserStepChangeDetection(
2   chapId,
3   ['config.required'], // Monitor config.required user steps
4   async () => {
5     configHelper.saveConfig(chapId, body, true); // merge with existing config
6   }
7   // Uses global context automatically
8 );

```

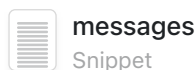
I actually did not yet see these in action in prod only in local testing, as I still got older chaplets from older pooled machines. Will retest tomorrow as there are now endpoints to help forcing latest chaplet deployment.

Other things

moved chaplet to port 4000 to avoid port conflicts

configured google auth client for prod and sandbox

UPDATE: gcal auth and test request went through on prod 🎉



Note: for anyone to try, their email has to be registered on Google console, I added now the current Chaps team. We have a limit of 100 test users before we have to review the OAuth client app with Google. (after review, it is open to any email)

📅 2025. Jun 12., Thu

have some meetings: a tech interview in the morning for Chaps Tech PM role, plus washup, plus PSR

I will have time for some smaller things, couple of candidates (not all will fit):

- general testing around and improving prompts of the existing Chaps to prep for opening up to staff
- specifically some cleanup in comm format? eg we still deliver markdown
- look into a web search Chap
- some cleanup in magic link UI to prepare it for design facelift by BalintO
- work on read/write scope for gcal Chap - it has now fixed read calendar events scope (this means multiple scope options support for OAuth and magic setup link in general)
- look into deeplink support eg in Linear

For next week (can do max half day on this Friday)

- more natural chap output (parallel comms)
- actually, I don't think we're far from re-enabling schedule support, to eg allow users to request a daily update from gcal or linear, so it is also something I could look into (currently it does mean adding local file system tool to these chaps, so will make them a bit more complicated overall, but now they're super simple, so there's buffer)

Chap Registry

Planning during standup. Agreed on the initial delivery mechanism: we'll keep chap templates in the chaplet codebase, delivery will be the same copy over as now. However, this way we'd potentially overwrite files which we wouldn't want to (for example, config which is set by the chap). I've implemented these for the chap 'package manager':

(see <https://github.com/lukilabs/chaplet/pull/43>)

- Chaps have a `package.yaml` in the root chap dir
 - This contains a version, and
 - a whitelist of files that can be **overwritten**
- A chap install/update library and cli (TS) with full test coverage, based on the `package.yaml`

Based on the `package.yaml`, a chap can be installed/updated - currently from the templates directory, soon from a downloaded package. We agreed that when a chaplet is redeployed (=updated), chaps will automatically be updated by scanning for their installed version (in the chap's `package.yaml`) and comparing to the template packages. I'll work on this update logic tomorrow.

Delivery planning

Discussing this tomorrow, <https://s-staff.craft.me/MY5ZwNSSZXJA3Q>

Interview(s)

- Interviewed a candidate for the AI Agent Quality Engineer position.
- Prepared with Sipi for tomorrow's onsite for the AI TPM technical interview. We agreed what problems we'll ask the candidate
 - Since this is a TPM position, we'll not go hardcore coding. Amongst other usual topics (previous projects DOE), we'll be asking the candidate to design a simple agentic system from a high level. We'll start drilling down into the details to gauge how the candidate understands LLMs, workflows, agents, etc.

Tomorrow

- Onsite interview
- Chap registry
 - Finish chap update mechanism by writing chaplet startup scripts to automatically update chaps which have newer version templates
 - Work with Levy on the API part
- → milestones refinement based on feedback

 Levy Melnikov

 2025. Jun 11., Wed

Ad-Hoc Support

We had a bunch of smaller issues in different components, but together we managed to ensure that **the latest feature(oauth forwarding) works on prod**. Amazing progress, I love this energy!

Envelope for Platform

I've introduced a new API envelope for platform API. It's exactly the same as the one for auth. This time I versioned it to avoid down time:

- `api/v1` - old envelope
- `api/v2` - new envelope

Postman is updated to reflect new API version and envelope: cc  Balint Orosz

Chap Registry

I finished basic CRUDs for chap registry. It can:

- Create, get, delete chap types
- Create, list, get latest chap type versions
- Upload bundles to chap type version

 2025. Jun 12., Thu

Chap Registry

Questions:

- How do we want to identify chap types?
 - By unique ids? e.g. `chap_type_<some-random-id>`
 - By short names? e.g. `linear` | `todo_chap` | `gcal` and so on. For this we will need to prevent creating chap types with the same name
- How do we want to version chaps?

Currently "latest" version is the one that was created most recently. Do we want to have strict versioning(semver for example) or do we want to be flexible e.g. allow `beta-01` \ `ver-1234` and so on?
- For the integration part I will need input on metadata e.g. descriptions, names, thumbnails, store assets and so on.

I'll continue with integrating chap registry into chaplets:

- CI\CD scripts to upload chap type versions on changes in templates
- Logic on chaplet to pull \ update new chap type versions

Envelope for Platform

Transition all components to use the new API versions with the new envelopes.

↑ Jun 12., Thu · Chaps Team

messages

```
1  {
2    "messages": [
3      {
4        "id": "msg_9ff182ba-568c-46bb-840a-513f3f4400af",
5        "threadId": "thread_2466ae47-9d5d-4d19-8aa6-7d61c86d4ec6",
6        "senderId": "cu1_c1af6f00-ec3e-476e-91cc-23714203726b",
7        "senderType": "user",
8        "content": "Hi",
9        "type": "text",
10       "metadata": null,
11       "createdAt": "2025-06-11T15:34:22.431Z",
12       "updatedAt": "2025-06-11T15:34:22.431Z",
13       "seq": 0
14     },
15     {
16       "id": "msg_56846fea-a41b-4b3e-a00f-f2ae1725f5e1",
17       "threadId": "thread_2466ae47-9d5d-4d19-8aa6-7d61c86d4ec6",
18       "senderId": "chap_27ec009b-95d6-467c-af7e-f2deff7b8c80",
19       "senderType": "chap",
20       "content": "Hello! 🙌 How can I assist you today?\n\nIf you'd like to
get started, I noticed that there's a step to connect your Google account. You can
do this by following this link: \n[Connect your Google account](https://
login.chaps.app/connect?s=5LyP9zPVi4ViyMy6)\n\nLet me know if you need help with
setup, have questions about this Chap, or want to know more about what I can do!",
21       "type": "text",
22       "metadata": {},
23       "createdAt": "2025-06-11T15:34:33.026Z",
24       "updatedAt": "2025-06-11T15:34:33.026Z",
25       "seq": 1
26     },
27     {
28       "id": "msg_215f7de1-3da7-4578-973e-7c467bab09df",
29       "threadId": "thread_2466ae47-9d5d-4d19-8aa6-7d61c86d4ec6",
30       "senderId": "cu1_c1af6f00-ec3e-476e-91cc-23714203726b",
31       "senderType": "user",
32       "content": "What are my meetings tomorrow?",
33       "type": "text",
34       "metadata": null,
35       "createdAt": "2025-06-11T15:54:01.219Z",
36       "updatedAt": "2025-06-11T15:54:01.219Z",
37       "seq": 2
38     },
39   ],
40 }
```

```

41         "id": "msg_390209f9-7298-4454-a6e2-ca4a2977f16b",
42         "threadId": "thread_2466ae47-9d5d-4d19-8aa6-7d61c86d4ec6",
43         "senderId": "chap_27ec009b-95d6-467c-af7e-f2deff7b8c80",
44         "senderType": "chap",
45         "content": "📅 Your Meetings for Tomorrow\nHere are your meetings
scheduled for tomorrow (June 12, 2025):\n\n1. **Technical interview - Stefan
Rotarus - Technical Product Manager - AI** \n - **Time:** 08:00-08:45 \n -
**Location:** Onsite \n - **Details:** [Interview briefing and feedback]
(https://app.ashbyhq.com/interview-briefings/491a3651-0978-4799-98ca-4d625f31b82f)
\n\n2. **Technical interview - Balint Bartfai, Peter Sipos** \n - **Time:**
09:00-10:30 \n - **Location:** Onsite \n - **Attendees:** schipy@craft.do,
balint.bartfai@craft.do\n\n3. **Web Standup** \n - **Time:** 10:00-10:30 \n -
**Attendees:** Multiple (accepted, needs action)\n\n4. **Chaps standup** \n -
**Time:** 11:00-11:30 \n - **Attendees:** balint@craft.do, levy@craft.do,
schipy@craft.do, balint.bartfai@craft.do\n\n5. **Washup - Stefan** \n -
**Time:** 14:00-14:15 \n - **Attendees:** balint@craft.do, viktor@craft.do,
schipy@craft.do, balint.bartfai@craft.do, zita@craft.do\n\n6. **Sipi PSR** \n -
**Time:** 15:15-16:15 \n - **Location:** Krausz-4-Azure dream (4) \n -
**Attendees:** viktor@craft.do, schipy@craft.do\n\n7. **🍷** \n - **Time:**
16:00-18:00\n\n8. **Out of office** (Starts at midnight) \n - **Time:** 00:00
(June 13)-00:00 (June 14)\n\nLet me know if you need more details, want to set
reminders, or make changes to any of these meetings.\nIf you want to I can also
help you with reminders, rescheduling, or sharing more details about any meeting.",
46         "type": "text",
47         "metadata": {
48             "chapJsonStr": "{\n\"icon\": \"calendar-days\",
\n\"icon_emoji_fallback\": \"📅\", \"title\": \"Your Meetings for Tomorrow\",
\n\"body\": {\n\"type\": \"text\", \"text\": \"Here are your meetings scheduled for
tomorrow (June 12, 2025):\n\n1. **Technical interview - Stefan Rotarus -
Technical Product Manager - AI** \n\n - **Time:** 08:00-08:45 \n\n -
**Location:** Onsite \n\n - **Details:** [Interview briefing and feedback]
(https://app.ashbyhq.com/interview-briefings/491a3651-0978-4799-98ca-4d625f31b82f)\n
\n\n2. **Technical interview - Balint Bartfai, Peter Sipos** \n\n - **Time:**
09:00-10:30 \n\n - **Location:** Onsite \n\n - **Attendees:** schipy@craft.do,
balint.bartfai@craft.do\n\n\n3. **Web Standup** \n\n - **Time:** 10:00-10:30 \n
\n\n - **Attendees:** Multiple (accepted, needs action)\n\n\n4. **Chaps standup**
\n\n - **Time:** 11:00-11:30 \n\n - **Attendees:** balint@craft.do,
levy@craft.do, schipy@craft.do, balint.bartfai@craft.do\n\n\n5. **Washup - Stefan**
\n\n - **Time:** 14:00-14:15 \n\n - **Attendees:** balint@craft.do,
viktor@craft.do, schipy@craft.do, balint.bartfai@craft.do, zita@craft.do\n\n\n6.
**Sipi PSR** \n\n - **Time:** 15:15-16:15 \n\n - **Location:** Krausz-4-Azure
dream (4) \n\n - **Attendees:** viktor@craft.do, schipy@craft.do\n\n\n7. **🍷**
\n\n - **Time:** 16:00-18:00\n\n\n8. **Out of office** (Starts at midnight) \n\n
- **Time:** 00:00 (June 13)-00:00 (June 14)\n\n\nLet me know if you need more
details, want to set reminders, or make changes to any of these meetings.\"},
\n\"followup\": {\n\"note\": \"If you want to I can also help you with reminders,
rescheduling, or sharing more details about any meeting.\", \"buttons\":
[{\n\"label\": \"Set a reminder\", \"action\": {\n\"type\": \"prompt\", \"prompt\":
\"Remind me about my 8:00 AM interview tomorrow\"}}, {\n\"label\": \"Share meeting
details\", \"action\": {\n\"type\": \"prompt\", \"prompt\": \"Show more info about
the Web Standup meeting\"}}, {\n\"label\": \"Reschedule a meeting\", \"action\":
{\n\"type\": \"prompt\", \"prompt\": \"Help me reschedule the Chaps standup\"}}],

```

```

49  \"body_md\": \"Here are your meetings scheduled for tomorrow (June 12, 2025):\\n\\n
\\n1. **Technical interview - Stefan Rotarus - Technical Product Manager - AI** \\n
- **Time:** 08:00\\-08:45 \\n - **Location:** Onsite \\n - **Details:**
[Interview briefing and feedback](https://app.ashbyhq.com/interview-briefings/
491a3651-0978-4799-98ca-4d625f31b82f)\\n\\n2. **Technical interview - Balint
Bartfai, Peter Sipos** \\n - **Time:** 09:00\\-10:30 \\n - **Location:**
Onsite \\n - **Attendees:** schipy@craft.do, balint.bartfai@craft.do\\n\\n3.
**Web Standup** \\n - **Time:** 10:00\\-10:30 \\n - **Attendees:** Multiple
(accepted, needs action)\\n\\n4. **Chaps standup** \\n - **Time:** 11:00\\-11:30
\\n - **Attendees:** balint@craft.do, levy@craft.do, schipy@craft.do,
balint.bartfai@craft.do\\n\\n5. **Washup - Stefan** \\n - **Time:** 14:00\\-14:15
\\n - **Attendees:** balint@craft.do, viktor@craft.do, schipy@craft.do,
balint.bartfai@craft.do, zita@craft.do\\n\\n6. **Sipi PSR** \\n - **Time:**
15:15\\-16:15 \\n - **Location:** Krausz-4-Azure dream (4) \\n -
**Attendees:** viktor@craft.do, schipy@craft.do\\n\\n7. **\\0\\0** \\n - **Time:**
16:00\\-18:00\\n\\n8. **Out of office** (Starts at midnight) \\n - **Time:**
00:00 (June 13)\\-00:00 (June 14)\\n\\nLet me know if you need more details, want to
set reminders, or make changes to any of these meetings.\"}
50      },
51      \"createdAt\": \"2025-06-11T15:54:23.648Z\",
52      \"updatedAt\": \"2025-06-11T15:54:23.648Z\",
53      \"seq\": 3
54    }
55  ],
56  \"hasMore\": false,
57  \"nextCursor\": null
58 }

```

↑ Daily Notes - 2025 June

Jun 11., Wed



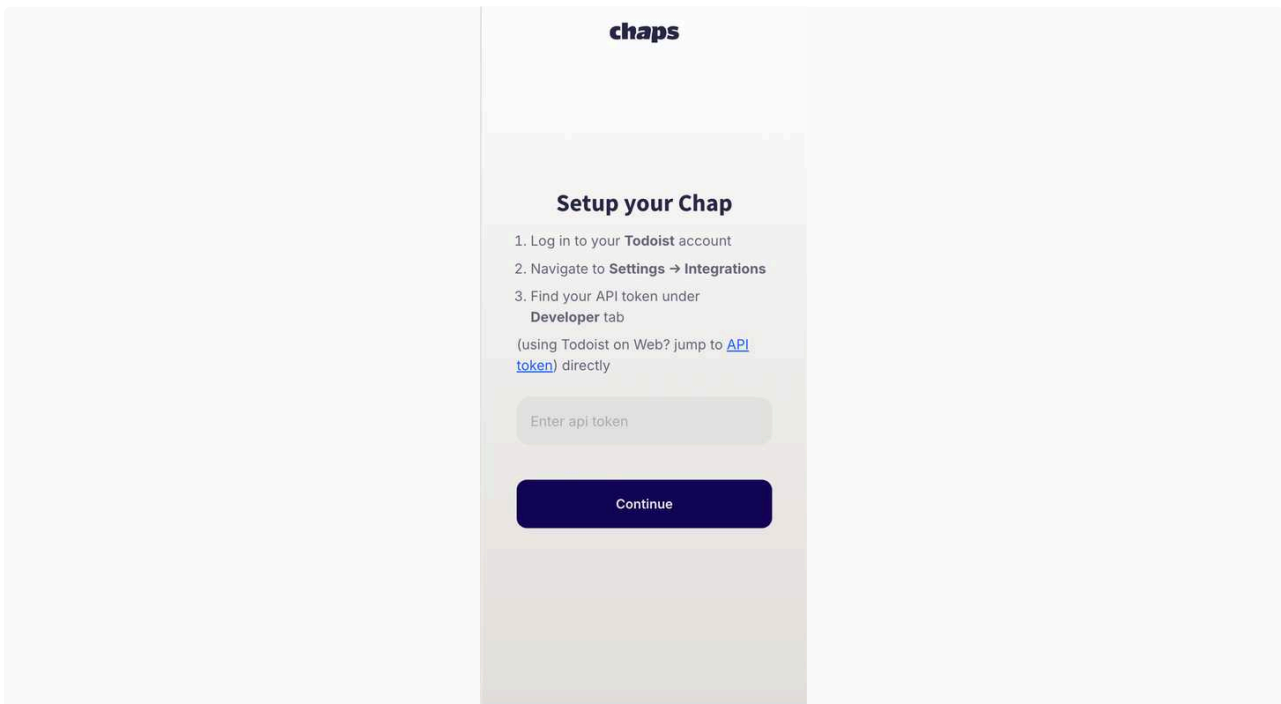
Chaps Team

Peter Sipos · merged a batch with magic link setup for config requiring chaps, like api key · Image · chaplet · feat(m...

↑ Jun 11., Wed

Chaps Team

merged a batch with magic link setup for config requiring chaps, like api key



chaplet



feat(magic-config): magic setup link flow for chap config steps

chap-auth



feat(magic-config): new ConfigPage and magic link flow for config requiring chaps

What's left: before I send it for some design polish, I want to make some tweaks to make that simpler (to apply design):

- unify the two pages we have, `/connect` and `/config`, so there's just one to style (also they need same flexibility anyway I think)
- plus maybe make a test page, which loads it with some test data to avoid the need for running the whole flow to get to a link which would render it

But even before that, I thought it's more important to progress with lifecycle improvements which allow the chap to start a conversation on a new thread ("Hi, I'm..."), or react to completed setups ("It seems like we're good to go...")

So **currently working on Chap lifecycle capabilities, ETA tomorrow afternoon**

- introducing agent execution as a possible action that can be configured (and emitting on chat threads, but triggered internally)

- "new thread" as lifecycle hook
- "completed user-step" as lifecycle hook

UPDATE: sneak peak (chap sends welcome message on thread as a response to the thread being created with the chap)

```

1  {
2      "messages": [
3          {
4              "id": "msg_78d567f3-70b6-4d13-8196-63d08b240ff3",
5              "threadId": "thread_9593224e-d70a-4327-89bd-431abc9c84f5",
6              "senderId": "chap_22aa2161-6742-46cf-b495-c5ada7595412",
7              "senderType": "chap",
8              "content": "Welcome! 🙌\n\nI'm here to help you get started with the
          application. Before you can use all of its features, you'll need to complete a
          quick setup step: connecting your Linear account. \n\nTo begin, please follow this
          link to authorize the connection:\n[Connect Linear Account](http://localhost:3000/
          connect?s=MJxs2ecps2c63VF4)\n\nIf you have any questions or need help during the
9         setup, just let me know!",
10             "type": "text",
11             "metadata": {},
12             "createdAt": "2025-06-10T20:55:38.476Z",
13             "updatedAt": "2025-06-10T20:55:38.476Z",
14             "seq": 0
15         }
16     ],
17     "hasMore": false,
18     "nextCursor": null
19 }

```


 Levy Melnikov

 2025. Jun 10., Tue

APIs

I've implemented proper envelopes for auth service. I've left some docs on the design & implementation [here](#)

I've corrected supervisor and platform to use the new format. ⚠️ I didn't fix OAuth connect page, so it's broken for now → will fix that tomorrow.

Chap Registry

I started working on chap registry. Nothing tangible yet, but I mapped the schema based on the [doc](#) and started working on the APIs.

 2025. Jun 11., Wed

Platform Envelop

I'll apply the same schema to platform APIs.

Chap Registry

Continue with the registry. Plan to deliver something tangible by Wednesday EOD.

Chap registry

We had an in-depth discussion with Levy/Sipi about how the Chaps registry (ChapStore) should work. There are two tasks we need to solve, in this order

- "Dynamic" registry
- Chap delivery

The registry is currently a single static JSON definition. We need to move it to API endpoint(s). For that, I've defined the minimum set of attributes needed for the client [here](#). Please comment if there needs to be more, or the client needs more assets, etc.

I did the same for our [internal representation](#).

Chap delivery is a different problem that we should tackle right after the registry endpoints are set up and wired up. Currently, a chap is created by copying over the chap's files (metadata) from the templates folder on the chaplet itself. This doesn't allow chaps to be updated without a chaplet being redeployed - which takes time - and then the config copied over again without touching the persisted state.

The to-be state is the chaplet being able to download a package from the registry and install the chap. This doesn't tie the chap config code to the chaplet codebase (allows separate repos), and allows updating individual chaps, or all chaps from one template.

For this to happen, package metadata needs to be delivered in the package download itself, which defines what can be overwritten and what post-install, post-update jobs need to run. Installation would be as simple as only overwriting files which match a certain pattern (the chap config), and copying every new file. Downloads will be protected with tokens issued to the chaplet itself.

Three components need to be built: storage & auth'd download (from R2), download initiation and token issuing to the supervisor (routed to the chaplet), and the chaplet job to extract and copy. We agreed Levy will start to tackle this after the auth envelope work. See [Package metadata](#).

Config

We talked with Balint about how the client should receive configs and be able to switch them for testing (ie. prod, sandbox, local). <https://s-staff.craft.me/257PZSyfoCmned>

Configs uploaded. Sandbox lives on `config.chapsdev.com/app.json` to mimic prod which is at `config.chaps.app/app.json`. Local config is only available on `config.chapsdev.com/app.local.json` - but we could use a `localhost` address for this, so dev can be faster by changing the values locally and having the client read them from `localhost`.

Technical to-solve

I've started writing up the areas we'll need to solve for productionization of Chaps. **These are not in priority order, they are bullet points by layer.** We'll also need to make product decisions regarding some of these. It's here: <https://s-staff.craft.me/TEcZrcMcJeWH3c>

Interview

Conducted the first interview for the [AI Agent Quality Engineer position!](#) 🎉

Tomorrow

- More in-depth planning and prioritisation for the delivery milestones - <https://s-staff.craft.me/TEcZrcMcJeWH3c>
- Another interview
- Chaps registry implementation planning/support with Levy

↑ Daily Notes - 2025 June

Jun 10., Tue



Chaps

High Level Goals : · This week I would like to have a few MCPs in a state where we can interact with them and test th...

↑ Jun 10., Tue

Chaps

High Level Goals :

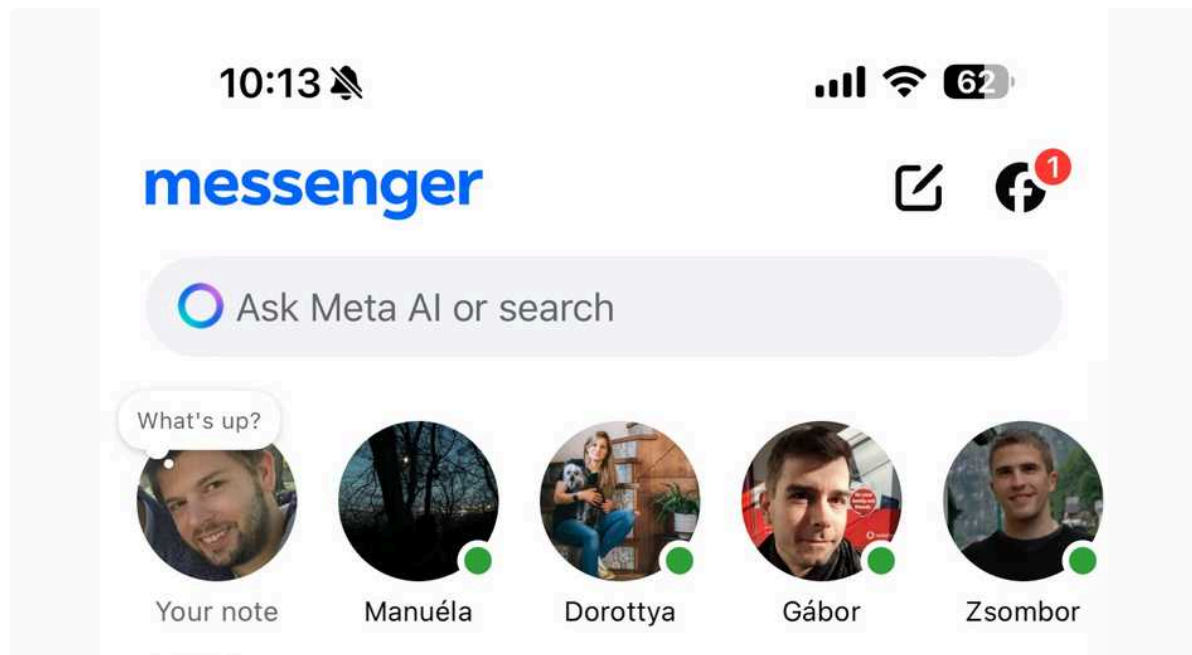
- This week I would like to have a few MCPs in a state where we can interact with them and test them. (where test them = people from Craft can build the app, install chaps and use them) I think Google Calendar, GMail and Linear would be great to have but open to suggestions. I would also create a simple "Web Search", and "Chaps Support" Chap.
- This will require :
 - The "Storefront" - having a registry/descriptor for Chaps/MCPs we can install
 - The Auth flow - Sipi is almost done with this
 - The Client - I am making good progress here.

Once we can do this, the following week we can divide into the following tracks :

- Making Chaps Feel More Natural (Comms part, Sipi 2nd point + App UX etc)
- Making Chaps Smarter (**Agent SDK** - how to add intelligence to these MCPs)
- Infrastructure improvements/enhancements
- Adding More Chaps

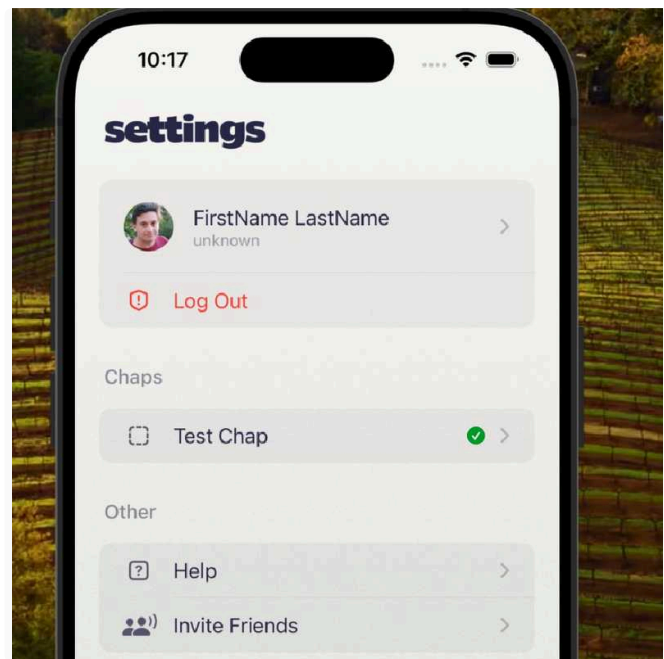
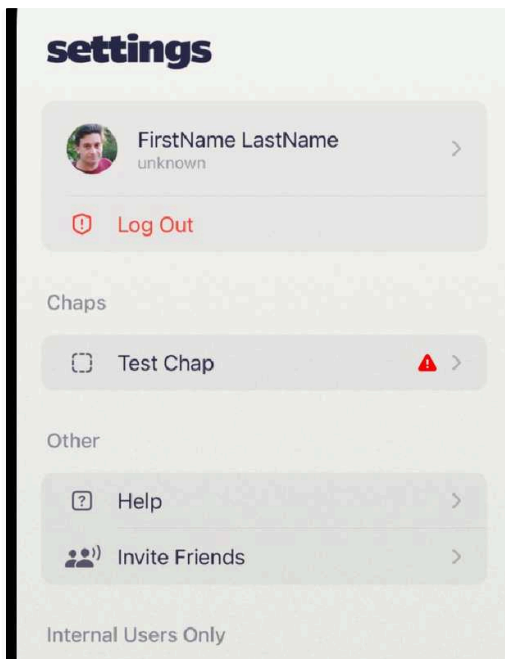
API Questions/Comments from  **Balint Orosz** :

- Can we **also add participants** to GET THREADS? Right now only the get Thread has it, but I would need this info in the list view. Also we will need :
 - a lastMessagePosted date (unless this is equal to updatedAt)
 - introduce paging (it might sound premature but very easy to add now, hard to add later)
 - (there is a chance that get Thread is obsolete, as the Thread Details is essentially the "Get Thread Messages".)
- **Introduce paging** to GET THREAD MESSAGES. Ideally client can configure page size, with a server side controlled max value.
- **Introduce generic envelope/error handling/messaging on API endpoints.** This can be very basic but I don't know how it works / cannot prepare with the client for error messages. A simple envelope for every request (this can be different per service, i.e. platform/auth etc, but ideally consistent inside services)
- On Platform introduce a high level "connection" status API
 - ▼ The goal is to have it similar to this



This should be **just the fact that is someone/a chap CONNECTED to the socket server or not.**

- Ideally we get back their latest known "health" status in LIST CHAPS. I don't know the format of this, but would want to something like the image below. Maybe it's a simple enum (Error, Info, Ok), and then we can do a detailed request later. **Let's discuss unsure about where this should go - maybe the connection status should be the green/alert?**
 - ▼ Example



 Levy Melnikov

 2025. Jun 6., Fri

API Improvements

I merged & deployed old PRs with API improvements.

- Creating a thread now validates input → you can only invite yourself and your chaps
- Added delete thread API. It tombstones threads and they don't appear in other APIs

Logs

- Added separation between sandbox & prod logs
- Cleaned up all log statements except in chaplets

Redeployment

Redeployment was not working. I fixed it: the problem was that on restart metadata is erased so `user_id` was not set for the chaplet. The solution is an endpoint on supervisor that can resolve `user_id` by `app_id` and `machine_id`. That endpoint is called from the `run.sh` script if metadata is empty.

Also: on redeployment full config is passed as otherwise env vars are not set for the machine.



Update: had a few extra hours

API

- Get threads is now paginated and includes participants
- Get threads and Get thread now includes `isConnected` property that indicates whether participant is connected to the socket or not.
- Fixed a bug in OAuth forwarding logic → I've now tested it on prod and sandbox and it works!
- ⚠ Scaled down the `api-gateway` down to 1 instance. It's not ready yet to be scaled → it can happen that send message API gets routed to a different instance that doesn't have thread participants connected to. We have first to solve it.

CI\CD

Investigated some issues in CI\CD

- ✅ Fixed: Github Actions used `'` instead of ``` which prevented the deployment
- ⚠ There's still an issue in `chaps-api` → I believe that the fly io token (`FLY_API_TOKEN`) is set for sandbox org instead of prod. I will need admin access to the repo to fix that.

Logs

Completed the logs setup. All services including chaplet are mostly using the new solution. Prod \ sandbox separation is working. Trace is propagated from auth to chaplet. I think it's the right place to stop → all services have the plumbing, if something is not covered let's handle in an ad-hoc basis.

 2025. Jun 10., Tue

APIs

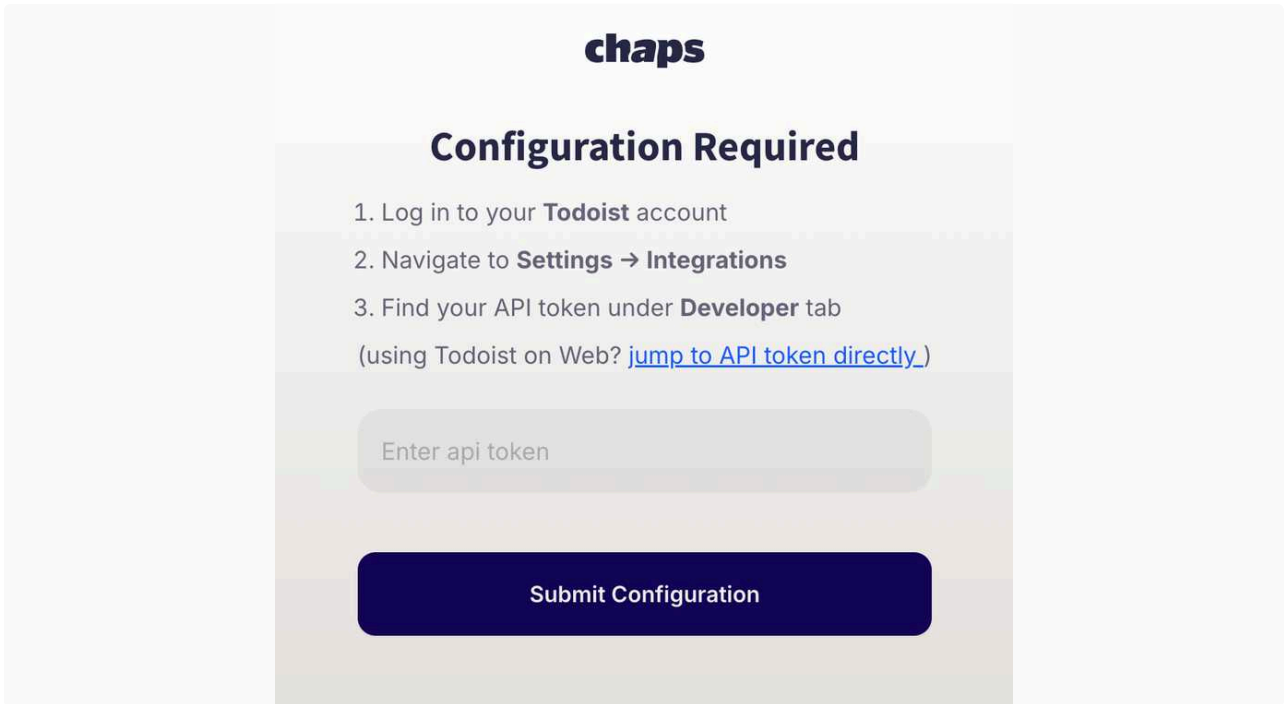
Address issues that Balint raised above.

[Last Friday's update](#)

This week I have two topics in mind:

1. chap setup flow improvements

for example over the weekend progressed and have the magic link flow working for api key based chaps as well, needs some polish to bring it home



also implementing the agent response to completed configuration

plus maybe support send magic link in email? (so user can perform config/connect on desktop)

2. chat-like user facing communication of chaps

start by experimenting with the parallel comm agent execution flow [described here](#) (it's the approach I'm most confident in at the moment, considering what we want to achieve). Initial version should be doable in hours.

↑ Daily Notes - 2025 June

Jun 6., Fri



Chaps

Highlights · CX Input. Tasks + AI · I'm sharing a non-actionable user feedback on Craft tasks as it might be relevant t...

Highlights

CX Input. Tasks + AI

I'm sharing a non-actionable user feedback on Craft tasks as it might be relevant to chaps:

I am not using them. I wish I was, I literally need to be using them, but I'm just not. I'm trying to look for some kind of service that uses AI to prioritize and check up on me regarding my staks but idk where to look and I'm not sure why the big AI companies haven't build this yet. I'm trying to use linear and claude but it's just truly terrible of an experience.

So there you go, build that, change for it, simple as.

I hope you find this helpful and if you do not wish to see notes like these in the future, let

 **Peter Sajevis** know where you stand. :)

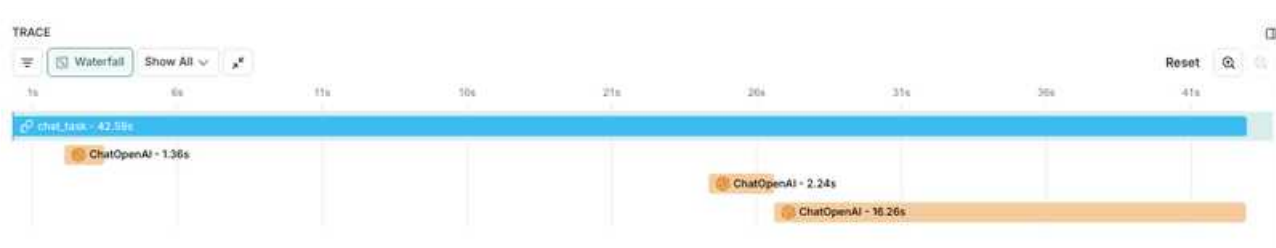
 **Peter Sipos** → next week

I think we should experiment with introducing a **parallel communicator execution chain** in Chaps.

What it solves: answering to the user in the way the Chaps wants, when the Chap wants, independent of the constraints of the actual LLM execution chain that answers the user message.

Specifically, we would like to have a quick initial response (on it, right away Sir, let me look into this), and also be able to provide interim messages of progress. Now any user facing message from LLM can come from two sources:

- reasoning output of reasoning models ([reasoning summaries](#)): this is a **no-go**, as reasoning models are heavyweight and slow for Chaps use cases. Also, even if they were fast enough, the reasoning output is a *trained feature of these models*, their frequency and content may not be fully controlled, likely only affected by Chaps prompts, I don't think we'd be really able to adjust their style and rhythm. Also, we don't have it while the LLM is waiting for a long running tool call, see below.
- output that we get when a tool (function) is called. We can highly influence the content of the summaries here by prompting (verbosity, style), but the issue is that **we don't control timing, and have no output to send in many scenarios**
 - We have no output until first tool call (or if there's no tool call, then until actual user response is emitted in second LLM call)
 - if LLM is running for a long time, we have no output (I've seen 15sec for heavy content with our current rel. fast GPT 4.1)
 - if an MCP call takes a long time, we have no output ({list_my_issues, limit: 20} on Craft Linear project takes 20 secs now)



example of execution where we have no chance to communicate for a 20 sec and a 15 sec period

The only solution I see to this is to run independent, parallel LLM requests, that "observe" the main track proceedings and generate user facing messages based on them. An added advantage is separation of concerns, where the format/verbosity of the output (short text SMS output vs rich chat experience) or the personal style / tone of voice of the output (helpful agent, military, butler style) can be independently managed.

These parallel communication LLM calls would get a combined history

- of actual proceedings of the main execution (tool calls)
- but can also get out-of-LLM context, eg "list_my_issues() tool is running for 10 seconds", or "agent is thinking for 5 seconds"
- the context size would be kept small for swift responses, it does not need the Chap's main tool defs in its prompt, the actual data returned by MCP calls would be omitted, it may not even get thread message history before current request (or only a highly compressed version of it), etc.

one downside could be cost, but as we would limit context size and use models that are on the faster, cheaper side, this may be actually much smaller than cost of the main execution.



ooo today (Friday)

ideas for next week in highlights sections

merged make agents aware and handle chap setup issues



feat(app-agent): agents be aware and handle chap setup issues

#34 · Opened by Schipy · Merged by Schipy

Merged

reached a point when the Chap answered with the setup link that we were working on this week 🎉



Recap - and the new proceedings today with **bold**:

introduced something called `chap_state`, a collector of pre-collected chap state info that is now also exposed to agent configurations, and can be included in prompts. It has now initially two keys:

- `chap_state.user_steps.all` all setup steps defined in chap's lifecycle
 - `chap_state.user_steps.incomplete` inly incomplete ones
- these user-steps (configs, onboarding etc), also include their completion state, and unnecessary details and secrets are redacted, so they're not going out to 3rd parties in plain text in prompts or into chat messages

also added a new agent tool called `get_step_url`, for now to communicator, which can return a setup url for a user-step. Under the hood it calls the gateway's `/authorization-url` endpoint to get the url (which is basically a start authorization flow endpoint)

tried using these elements in the communicator agent to make it return the setup link in the conversation, but it has so strict, different purposed instruction set about response format etc, that it did not work out. Therefore I created a new base-template-defined agent called `chap_app_agent` which is the intended responder to application/chap level configuration, settings, troubleshooting, user profile questions etc (something that anyway felt we should do).

Added a simple rule now to execution: if there are any incomplete critical user-steps, we delegate immediately to chap_app_agent instead of conductor (and skip communicator). Later we will refine a lot I think which responds to user (need a system of delegation, conducting, handoff), but for now this is enough, and actually, quite desirable, as the chap_app_agent's only concern for now is to help the user set up.

another piece is that the domain agent was not aware of communication issues, and even with not working MCP servers, it returns happy responses. **I tried injecting the failed MCP tools as error tools, which worked nice, except that tools are not provided to the agent by Orchestra in the final response phase (after the tool call phase)**, so when formulating that actual response, the domain agent again lost context that there was a MCP tool issue. **Ended up injecting MCP tool init failure into the system prompt which is always available.**

Note: since for incomplete oauth steps we delegate to chap_app_agent, this does not really come into play for initially missing setup, rather only if there are later issues coming up.

And finally, separated `user-steps` from `lifecycle actions` in lifecycle, because user steps good to have an id (to refer to eg in tool calls) and will have their own lifecycle when (eg triggering the agents on a successful setup (oauth complete))

while testing I also realized I actually fixed unconfigured gcal MCP Server issue in the meantime when I removed the auth server parts from it, which caused it to process.exit earlier, which kept the init hanging. This does not solve the issue in general (eg if an other stdio server exits), but at least for now we won't run into this with gcal chap.

Another thing I have (or for someone) for next week is to extend the magic config link concept for api key based Chaps. I think it fits the existing infra quite ok, but needs a new config page on auth-fe, and some lifecycle config extensions to represent the config user-steps (plus maybe a schema to describe the instance config params of a chap themselves)

 Levy Melnikov

 I'm OOO 08:30 - 10:00 (Need to go to the bank)

 2025. Jun 5., Thu

Logging

I finished adding logging to chaplet. The final solution is to tail log files in a separate process\service and send them to betterstack. I want to test this out on sandbox \ prod, but I'm not sure how chaplet redeploying works after the pooling rework.

✅ The current state is that each component has logging library setup and logs are getting into betterstack.

I still need to work a bit more on:

- Ensure that all console.logs are replaced with new solution
- Distinguish sandbox \ prod
- For critical flows like chap(let) creation ensure that traceId is working

Craft MCP

```
Chat with Untitled Chap (b21fd1ed-c4bd-4137-9487-2b4c4121828c)

In 1 sentence what each team was doing today?

> [!] get-daily-note
Retrieving today's daily note to summarize each team's activities
{"day": "2025.06.05"}
> {"#2025.06.05\n#Services\n#Overview... (truncated)" {"type": "tool_result", "tool": "get-daily-note", "result": "#2025.06.05\n#Services\n#Overview... (truncated)", "agent_id": "conductor_agent", "timestamp": "2025-06-05T17:18:07.219347"}

{"type": "end_tool_use", "content": "Tool usage complete", "agent_id": "conductor_agent", "timestamp": "2025-06-05T17:18:08.991504"}

>
text: 📄 Today's Team Activities - One Sentence Each
**Services Team:** Focused on daily monitoring, started a back-fill job for search snapshots, explored Cloudflare, worked on centralized login and web clipper flows, and began decommissioning old document snapshot tables.

**Web Team:** Retested Home & Settings bugfixes, reviewed and tested the Web Clipper, collaborated on marketing website direction, and fixed UI issues and analytics logging.

**CX Team:** Handled user login issues, followed up on overdue $50 payments, created/reviewed support materials and video content, and managed support tickets.

**Growth Team:** Coordinated regional marketing emails, negotiated influencer partnerships, explored new social platforms, created and translated community content, and analyzed user engagement.

**App Team:** Advanced tag feature implementation, continued whiteboard improvements, and fixed editor and search-related bugs.

**Chaps Team:** Improved OAuth forwarding/logging, separated sandbox/production infrastructure, worked on redeployments, and enhanced agent setup awareness for onboarding.

**Design Team:** Refined tag filtering animations, updated marketing site branding and layouts, documented search research findings, continued recruitment, and collaborated on visual asset creation.

**Atlas Team:** Processed accountant packages, handled salary and invoice requests, and advanced recruitment efforts.
If you want a deeper summary or details for any team, just let me know!

> |
Type /f to browse files, /si to send an image, /q to quit, /u to manage your account, /s to switch chap, /m to manage your email
```

I started exploring MCPs & building chaps by building one for Craft.

- Tools: list daily notes, get daily note content
- Limitations:
 - Authorization & Space IDs are hardcoded
 - For getting blocks a crude script is used, new markdown export would be 10x better

Some early thoughts:

Having multiple spaces brings some complexity -> when I ask about a daily note which space do I mean? We can limit chap to 1 space or add preference option to set default space, just something to think about.

Although I used a very crude markdown generation script(the one we have in new markdown export would be 10x better) the results are kind of good already.

Advice for future chap builders: first ensure that MCP works without chaps via [MCP Inspector](#) then start integrating it into Chaps.

The most challenging part is integration & testing. For example the schema is parsed on chap creation, mcp is started on prompt. If external chap creator won't have access to logs of these processes, then it would be hard for them to debug. Once we'd get to actually allowing people to add their chaps \ mcps we'll need some sort of platform feature that would allow fast iterations with debugging chaps \ mcps.

 2025. Jun 6., Fri

MCPs

I'd like to continue exploring MCPs. I now want to understand settings \ OAuth flows more. For that I'll try to create a chap with some settings \ OAuth.

We'll also have a catchip with Schipy potentially there will be some work around lifecycle \ user steps.

 **Balint Bartfai**

Sorry guys I'm down with some virus type of thing, started antibiotics. Out tomorrow as well.

↑ Daily Notes - 2025 June

Jun 5., Thu



Chaps

Levy Melnikov : On Friday I will need to go the bank at 9AM. I will be back at ~10AM. · Levy Melnikov · Wed, Jun 4 · ...

↑ Jun 5., Thu

Chaps



Levy Melnikov : On Friday I will need to go the bank at 9AM. I will be back at ~10AM.

 Levy Melnikov

 2025. Jun 4., Wed

Chap OAuth Forwarding

I finished OAuth request forwarding. Now chaplet can generate a page link that authorizes it with MCP, page doesn't require the client to be logged into Chaps. This page link can be sent over email \ IM. Currently it just contains "authorize" button, but we can extend it.

I only tested it locally, but I'm sure that if something doesn't work out on prod, it will be a simple fix.

Logs

I've sent invites to betterstack, it's shaping up:

- auth - 
- orchestrator - 
- api - 
- chaplet -  (it's unstructured logs from across all chaplets, still working on it)

Chaplet logs are a bit tricky. It's critical to have user_id in the logs (e.g. user X reports a bug, how to find their logs?). The problem is that default log exporter doesn't have user_id in the logs. user_id is in the metadata of the instance, but it isn't surfaced in the logs. We'll probably need some custom forwarder that can extract it from metadata or env.

 2025. Jun 5., Thu

Next steps

I will probably have capacity to take on new tasks.

Logs

Continue with logs:

- auth logs are incorrectly timestamped, they look as if they were emitted with ~5 seconds delay
- traceId passing still doesn't work
- prod & sandbox logs are mixed together

Chap OAuth Forwarding

Once the prod \ sandbox env is ready → test it out

merged "enterprise" OAuth configuration support with Google Calendar MCP Chap

compared to last days update it needed some more plumbing/cleanup

- inject client_id and client_secret as env vars (later this might be part of some kind of chap secret management, as it is not really sustainable to maintain this for N providers)
- added support for some interpolation in chap lifecycle.yaml as well, so this can be expressed nicely

```
1  # Actions to execute after chap installation
2  user_steps:
3    - type: oauth.authorize
4      id: google-connection-setup
5      provider_name: google
6      client_config:
7        issuer: "https://accounts.google.com"
8        client_id: ${env.CHAP_GOOGLE_OAUTH_CLIENT_ID}
9        client_secret: ${env.CHAP_GOOGLE_OAUTH_CLIENT_SECRET}
10     # ...
```

- also the oauth redirect_uri now comes from env (needs to be as it depends on environment)
- stripped down the src-included Google MCP Server, to not pollute the repo too much, dropped the tests and everything related to its own auth server

on branch Then started to work on the agent's awareness of their own connection issues and setups steps and being able to communicate the setup url

I have many moving pieces here, there is a lot of progress, but not ready e2e

1. **I introduced something called `chap_state`, which is also exposed to agent config**, and can be included in prompts. It has now initially two keys:
 - `chap_state.user_steps.all` all setup steps defined in chap's lifecycle
 - `chap_state.user_steps.incomplete` only incomplete ones

these user-steps (configs, onboarding etc), also include their completion state, and unnecessary details and secrets are redacted, so they're not going out to 3rd parties in plain text in prompts or into chat messages

It's up to us how to incorporate them into prompts, for now I added `all` to the communicator prompt

2. **I also added a new agent tool called `get_step_url`**, for now to communicator, which can return a setup url for a user-step. Under the hood it calls the gateway's `/authorization-url` endpoint to get the url (which is basically a start authorization flow endpoint)

I could not yet completely set up all services to get a valid auth frontend url, so for now it will return the direct authorizationurl, which is good for local testing. Tomorrow I want to sit with Levy to iron out the setup, so I can test it with the magic (standalone) auth frontend url.

These two steps are in theory allow the communicator to detect missing configuration, create a setup url for it and include it in response. However the rest of the comm prompt for now works a bit against it I think, as it strictly defines what kind of response it should emit, and this "config help" is not really part of that system. This is something for tomorrow.

btw, I have a feeling that eventually it should not be the comms agent who answers in this case, but this is a later topic. In any case, even the comms & domain agents have to be aware of the issue on their own abstraction level.

3. **a third piece is that the domain agent is not aware of communication issues**, and even with not working MCP servers, it returns happy responses, which I think also doe snot help the comm agent to recognize issues and trigger setup help.

This has two parts:

- for one, eg **badly set up stdio MCP Servers** like the Google Calendar **hang at the moment**. I did try impementing a connection timeout in the Orchestra fork mcp_adapter, but unfortunately seems like Python's async exit CancelScope context manager framework is super convoluted, or maybe some deeper 3rd party is screwing us, but even after 1-2 hours LLM session I dd not have a working solution, so shelved it for now. Ridiculous really, as in JS/TS this would be a no brainer. Will revisit with a fresh mindset later. The end result is actually an other exception, not timeout, so at least hang can be solved.
- the other thing is that even if we have an error, what happened is that the **tool was skipped**, meaning the domain agent had no clue there was something wrong in the first place.

To fix the second, I now added a feature that **failed MCP server initializations are actually added as a `get_error_report` type tool** (eg `get_linear_mcp_client_error`), making the agent aware that there should have been something, ant it can query the error details if needed. **This did improve awareness and I started having meaningful, warning type of responses instead of happy responses for queries that could not be answered because of missing tools.**

4. I also started preparing for triggering the agents on a successful setup (oauth complete) → splitting the "user-steps" from "lifecycle actions".

📅 2025. Jun 5., Thu

test magic link flow e2e

reach a point where the communicator agent successfully sends the magic link in chat if it is not yet set up (or in worst case, if you ask for it)

then proceed with triggering the agents on a successful setup (oauth complete) so it sends a welcome message

Otherwise there's still a bunch of plumbing:

- apply magic link and user-step concept to apiKey type of MCP chaps (including a config screen with help? completion trigger etc)
- oauth revoke handling
- revisit hanging MCP server issue

But, these could be postponed I think for more interesting **agent communication topic** (how responses are "streamed" etc) - **I plan to think about this for next week**

note: this Friday I'm officially ooo, though may put in a few hours, eg prep for next week

Low level infra today with a bit of wrestling with Fly.io.

Slack deployment notifications have been set up in `#chaps-deploy`

Sandbox env

Implemented all splitting of sandbox and production. Lots of services were not really prepared for full configurability, lots of stuff baked in. I had to change all of the moving parts (auth, platform, supervisor, chaplet) to ensure each one is prepared to run in isolation. Simplified the infrastructure a bit as well, pulled together different Redis instances to use namespaces instead → we get the benefits of splitting namespaces without the added complexity of managing different instances. Lots of plumbing.

Vercel pre-prod branches are auth'd, this required changes to the way Auth is called from sandbox.

There's still some issues with Chaplets built from the non-main branch. Sandbox works in isolation, but Chaplets built on GH actions do not get pushed to the sandbox base image, from which sandbox chaplets are created. Prod chaplets are pushed, enabling sandbox to work.

Chaplets needed a tricky way to grab their user IDs after started. The main issue is that if we update a machine's config - which would be the canonical way to set env vars - the machine is rebuilt. Which takes years. Instead, we set the machine instance's metadata (only milliseconds) with the chaps user ID, but that means the machine has to retrieve the metadata on start. For some reason API calls to the machine API from the chaplet calling it were failing. I found an undocumented API endpoint which involved calling the API from a UNIX socket at `/ .fly/api` with the API. This injects the auth headers and returns with the correct metadata.

Apart from Chaplets not being deployed from any branch, sandbox is fully ready.

Chaplet(s) redeploy

Implemented chaplet redeploy. This wasn't as straightforward as with one-app-one-machine. Endpoint set up to redeploy a users chaplet with their API key - which should not be a common use-case, also because we don't ever see their session token. Admin call also implemented for us to be able to redeploy arbitrary users chaplets.

The tricky part was not one chaplet: Implemented pool redeploy as well. The mechanism I chose was:

- for all machines in the pool, in parallel:
- acquire lease on pool machine. This is a lock.
- start the update process on the machine - this means a full rebuild with the new image and can take a while
- wait until update is complete
- release lease

While there's a lease on the machine, no clients can change it. New user registrations just wait. I've increased timeouts to 90sec to accomodate for this.

This way there is a slight chance the machine pool is unavailable, ie. users don't get a machine immediately. But this guarantees machine update atomicity - they are updated before being assigned to new users, and is done as fast as possible. This way, we can run multiple supervisor instances that do the updates - horizontal scalability.

Also for scalability, I removed the in-process routing mapping. We'll go straight to Redis for routing info. The current implementation was very fast but required a single supervisor process.

I've set the supervisor to 2 machines, let me know if something is fishy - I'd rather have us find out now than when we need to scale

Technical

- To call sandbox auth, set the `x-vercel-protection-bypass` header to the value at <https://vercel.com/craftdocs/chaps-auth/settings/deployment-protection>
- Sandbox should be fully function at the domain `chapsdev.com`. Ensure you are using different auth tokens for these environments - your user ID is different
- Set up asset worker and domain as well
- Chaplet config vars (sent on machineConfig) are set with `CHAPLET_` env vars. Use this convention if you need to add anything there
- Please use the `#chaps-deploy` channel if you're deploying to any of the services apart from supervisor. Eventually I'll get round to automating this, but for now manual is fine. Supervisor and Platform has automatic deploy messages

Tomorrow

- Fix chaplet sandbox builds
- If nothing higher-prio then I'll kick off chap registry

This is just something I tried after work - Cloudflare implementation of Platform:

 **lukilabs / chaps-cloudflare**
github.com/lukilabs/chaps-cloudflare

 lukilabs TypeScript Stars 0

Thinking if we wanted to try to do what Platform does now with Cloudflare D1 & Durable Objects for sockets



Chaps Team

Peter Sipos · In the morning we had discussion about chaps app independent authorize link flows. · merged Fixed s...

↑ Jun 4., Wed

Chaps Team

In the morning we had discussion about chaps app independent authorize link flows.

merged Fixed so that a reading chap config with missing api key config or missing oauth secrets does not blow up (will just use placeholders). A step toward chap being able to handle and help with its own configuration.

on branch Made good progress in making OAuth flow & chap configuration work with a 3rd party Google Calendar MCP server. This is an entirely different setup, so having it working raises confidence significantly in our MCP integration flexibility:

Status: fixed/solved all config differences, and almost works, in the last few issues, eg I need to transform the expires_in received from OAuth to the expiry_date expected by google oauth libs.

Note: Even if we end up using a different MCP server solution for this (though this the most developed I found so far), I wanted to plumb this through to battle test our setup.



nspady / google-calendar-mcp

MCP integration for Google Calendar to manage events.



nspady

TypeScript

Stars 311

Differences (compared to previous Linear MCP) - basically everything:

1. Google OAuth uses classic static app registration that you have to manually do on Google Console. It uses a predefined client_id and has client_secret that needs to be injected, our config supports this
Left: client_secret injection is to be properly figured out
Also: different test vs prod configuration is to be figured out (will do something simple for now)
2. the OAuth variant it supports is oidc instead of oauth2 (uses different metadata descriptors etc), we support this
3. the MCP server is not an npx runnable package, but has to be built on the fly (or prebuilt and bundled in) the Chap
 - we now build this MCP server in the init.ts of the Chap (which will be migrating into lifecycle as "install steps" at some point as a follow-up for a comment from BalintB, but that's just a side-note)
 - and then to run it, I had to add a cwd support in our Orchestra fork (to the mcp_adapter), so we can run the MCP server subprocess from within the chap's own meta folder as working directory, not the common chaplet executor folder (where the parent Python process runs). The underlying mcp Python package does support cwd, so this is a legit generic improvement that could even be merged upstream.

→ Now that I think about it, we may want to do this even by default, so that no 3rd part code MCP servers run in the common process dir (even npx executable ones)
4. and the resulting token needs to be injected in a completely different way for the MCP server, not as a header param, but it needs to be put down to a config file - simulating as if the MCP server - which has its own naive, desktop-environment-assuming oauth solution built-in - has already performed OAuth and saved it for itself.

For this I added a new injection method to our MCP config called `file`, which can contain jinja2 templates to inject config files for the MCP server

I also had to inject not just the access token, but all oauth related data to the agent config.yaml (token_response and metadata_config which is ~= the client config)

```
1  tools:
2    - type: mcp-stdio
3      name: gcalServer
4      command: node
5      args: ["build/index.js"]
6      cwd: google-calendar-mcp
7      file:
8        - path: google-calendar-mcp/.gcp-saved-tokens.json
9          input: ${secrets.oauth.google.token_response}
10         template: |-
11             {
12               "access_token": "{{ input.access_token }}"
13               {% if input.scope %}, "scope": "{{ input.scope }}" {% endif %}
14               {% if input.token_type %}, "token_type": "{{ input.token_type }}" {%
15             endif %}
16               {% if input.token_issued_at %}, "token_issued_at":
17             {{ input.token_issued_at }} {% endif %}
18               {% if input.expires_in %}, "expiry_date": {{ input.expires_in *
19             1000 }} {% endif %}
20             }
21         - path: google-calendar-mcp/gcp-oauth.keys.json
22           input: ${secrets.oauth.google.metadata_config}
23           template: |-
24             {
25               "installed": {
26                 "client_id": "{{ input.client_id }}"
27                 {% if input.client_secret %}, "scope":
28               "{{ input.client_secret }}" {% endif %}
29                 {% if input.redirect_url %}, "redirect_uris":
30               ["{{ input.redirect_url }}" ] {% endif %}
31             }
32           }
```

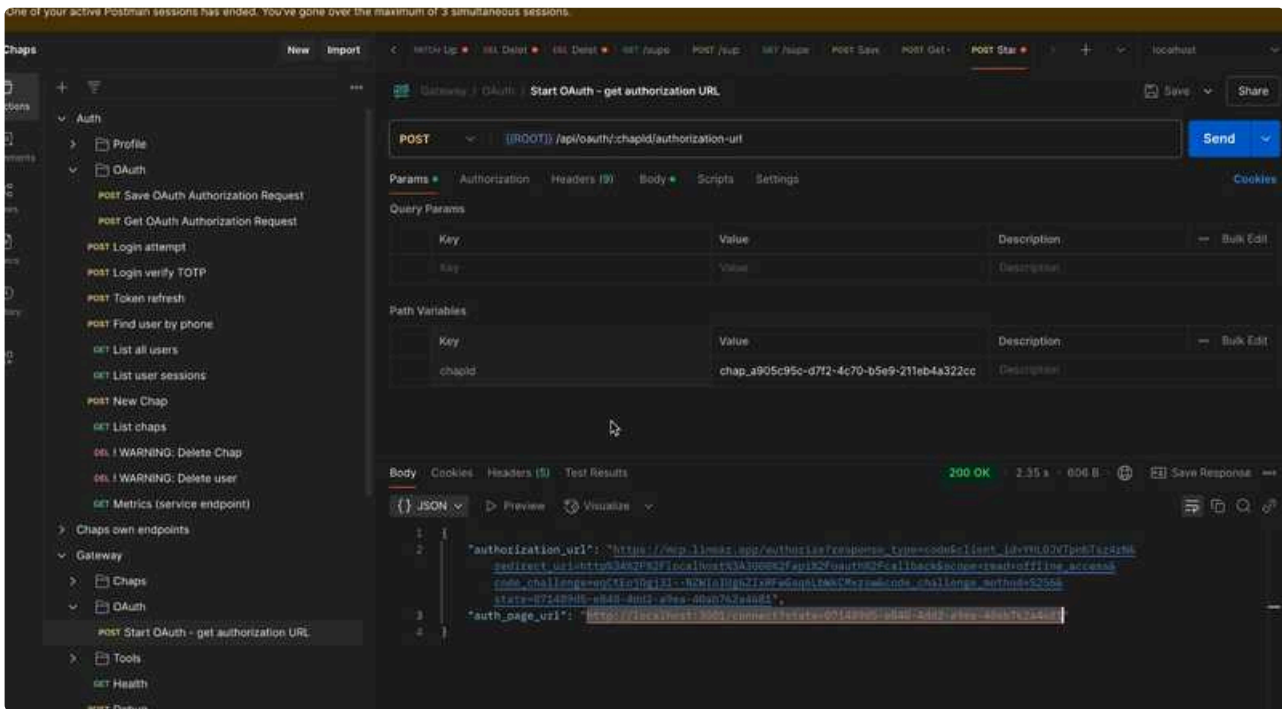
Left: I don't actually have the redirect_url saved in our metadata config, will need to fix this up

Ahead

still have many items we discussed around setup:

- new connect screen in auth-fe and integrating the new endpoints Levy created to support the standalone config link (which can work from desktop email, whatsapp etc)
- have the chap itself be able to send you the config link in chat (and in general make it aware of the incomplete setups and setup issues)
- trigger agent flow when setup is complete (so its sends a welcome message to user)
- revoke authorization support, and revoke automatically on chap deletion

OAuth



tldr; This is a first step to enable us to authorize chap with OAuth MCPs on any device browser

- Auth backend service has new APIs for getting \ setting OAuth Authorization Request details.
 - Set: called by chap to save OAuth Authorization Request details, authorized by token
 - Get: called by auth page → authorized by providing state
- New OAuth Page on `chaps.app/connect?state=<state>`
 - Gets OAuth Authorization Request details from Auth backend
 - Displays chap details, user details and shows a button with authorize url
- Chaplet now lists both actual Authorization and Auth OAuth Page url in `/authorization-url` endpoint

Not implemented yet → this will not work on production. We need to make `redirect_uri` to point to auth service so that it can attach an auth token so that supervisor can route the request to chaplet.

Logs

I've added new logging system to supervisor.

I also found a small issue: Bun doesn't support [async hooks](#) that are needed for logging. I will adjust the solution to use supported [AsyncLocalStorage](#).

📅 2025. Jun 3., Tue

OAuth

- Test out & deploy changes from yesterday
- Add redirection \ routing logic on auth→supervisor

Logs

Continue with logs

- ✅ auth - via vercel plugin
- ✅ supervisor - via SDK nodejs library
- 🛠 platform-api - via SDK nodejs library
- 🛠 chaplet - via loki container that we already have OR SDK nodejs library
 - executor - I'm not sure about this one. It's python, probably will require a bit more tinkering to pass traces and manage context, but should be doable

Reply from Fly.io about VM limits (from yesterday's email):

There is a machine limit on Fly orgs, and it's mainly there as a failsafe to prevent runaway creation of machines. We can raise it whenever you feel necessary! I've bumped up your `craft-docs-kft` Fly org's limit to 100 machines - hopefully this covers your needs, but feel free to let us know if you need more.

Had a lengthy architecture discussion today with Sipi, Levy on how we should approach OAuth. Levy has started work on the redirect and token capture flow needed for chaps to store tokens.

Pooling

Deployed pooled machines. Core functionality is working with the new pooling logic, it's much faster.

When a new chaplet is spun up, a background job backfills the available machines in the pool. How many standbys we want is configurable, **I now have it at 3**.

This was the supervisor deployed to EU region.

After moving machines to IAD, then BOS, I am consistently getting a Fly.io error

```
1 2025-06-03T17:34:27Z proxy[683dd42cd05468] iad [error][PR04] could not find a good
   candidate within 20 attempts at load balancing
```

This is on the supervisor app, not the chaplets. According to different forum posts this can mean Fly.io is at capacity in a region and cannot route requests when using fly-replay, which we use to send requests to chaplets.

I created - deleted - created - deleted chaplets until my new chaplet was placed in BOS, however, IAD is still trying to route the request, which is failing via Fly's proxy.

It turns out, when you proxy requests to machine instance IDs, you *must* specify the app ID as well. This, as with many other things on Fly.io, is not documented, and it took lots of trial and error.

I'm not too happy with Fly. They're good to start with for hobby projects, but anything more than a couple of VMs it seems like they have [scaling and resource issues](#) - which **we** have to handle and orchestrate - defeating the purpose of an infra that "just works".

Highly experimental, but I have this <https://github.com/lukilabs/chaplets-supervisor/tree/digitalocean>

Redeploy is not implemented yet - this is something I'll do tomorrow. I'm anticipating some added complexity, as a redeploy uses a new machine image - which means not only the running chaplet needs to be cycled, but the pool should also get the new images. If it's not critical to do this now I can skip and move on to other higher-pri tasks. Redeploy has never fully worked as it wasn't deemed critical - the only time we'd want to redeploy is if the chaplet infra (the VMs) change. Currently, we build chap templates into the VM - but these templates will be delivered via other mechanisms.

Sandbox - Prod

I started the split between sandbox and prod environments so we can deploy and test all the moving parts.

- registered `chapsdev.com` domain on cloudflare
- set up `api.` and `auth.` and build automation
 - Supervisor is now deployed automatically on push of any branch to sandbox
 - Auth is deployed on push to `main`, we'll manually promote to production. My opinion is auth is too critical to be deployed on every push to `main`

Work to do:

- Set up `platform.` with build automation similar to `api.`
- Set up automatic chaplet deploys with newest images
 - This might be a bit more complex: due to pooling, I'll be recreating pools when a new image is pushed

Tomorrow:

- Chaplet redeploy plan. I'll have a better idea of the complexity tomorrow morning, and prio based on that.
- Anything that crops up with the new pooling. There might be edge cases we'll only discover when using it (like the proxy header issue above).
- Finish up sandbox - prod split on dev domain
- I'm happy to jump on OAuth if help is needed there so Sipi can implement the flows
- I'm also happy to work on observability and metrics, continuing Levy's work there
- If there's nothing higher-priority, I'll be starting to scope out the Chap registry (see <https://staff.craft.do/editor/d/a74950fb-ace8-80e7-ec4d-383742d9cc03/eb71c60e-413d-4ffa-90db-a620134ada7b/b/b831dc9f-b7ac-4f6c-8137-dd63d88b0c8e/Levy-in-Chaps/Chap-registry?s=QTZyAd5NVDaAif62YMqzMFAuGU222bH7RMNfa2QamFeJ#EDA3CDB9-F7EE-4A2E-89F5-9C7154F12D1A>)

↑ Daily Notes - 2025 June

Jun 3., Tue



Chaps

Levy Melnikov · Mon, Jun 2 · APIs · Added validation to the thread creation API. User is allowed to create a thread on...

↑ Jun 3., Tue

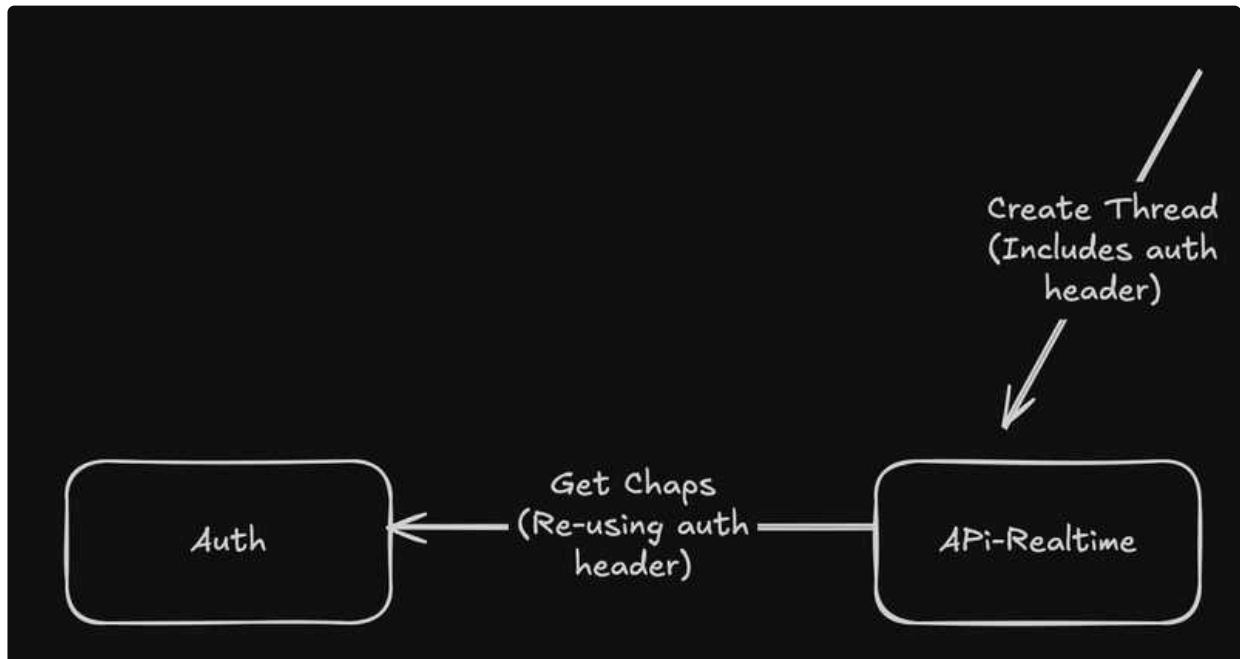
Chaps

 Levy Melnikov

 2025. Jun 2., Mon

APIs

- ▼ Added validation to the thread creation API. User is allowed to create a thread only with themselves and their chaps. To implement this, API-Gateway needs to know which Chaps does the user have. This info is available only on chaplet or Auth. I propose to make a sub-request to Auth. **To authorize the auth header is re-used in the sub-request to Auth.** [PR](#)



- Added an endpoint for deleting threads: [PR](#) . On deletion they are marked as deleted and filtered out of all other requests. For this a new field is added `isDeleted` .

Logging 2.0

I started working on improving logging.

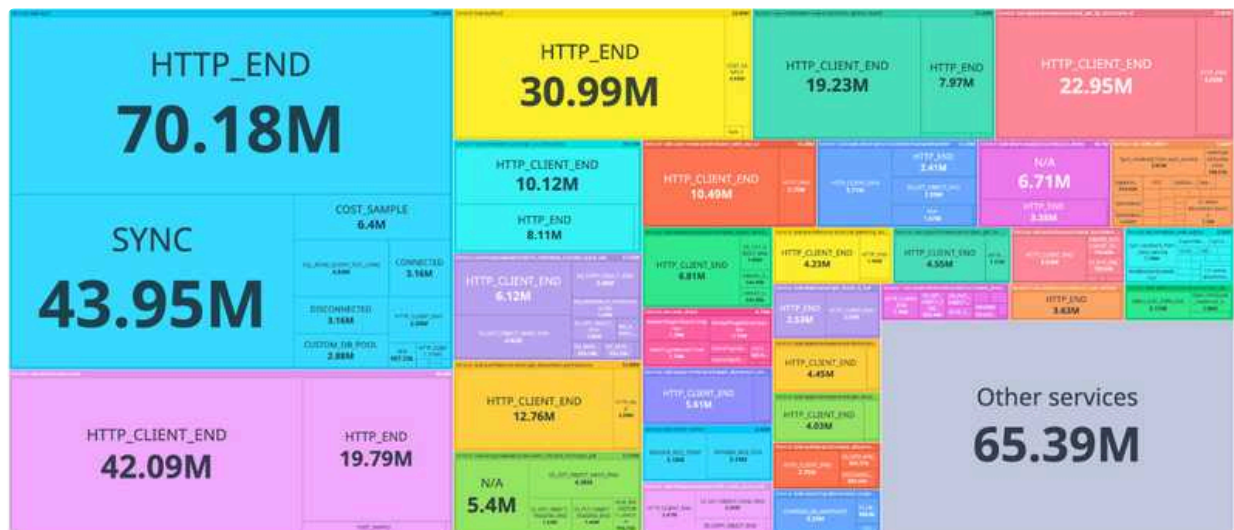
First I created a new account in datadog, account activation can take up to 3 weeks 🤔 → I started to look for alternatives with OpenAI Deep Research and google based on our requirements.

I've found <https://betterstack.com>

- ▼ Comparison: Datadog invoice

Committed Log Ingestion - Monthly Datadog Subscription, 2025-04-01 - 2025-04-30, 280 Committed Log Ingestion	280	.1	USD \$28.00
On-Demand Audit Trail - Monthly Datadog Subscription, 2025-04-01 - 2025-04-30, 1 On-Demand Audit Trail	1	17.007	USD \$17.01
On-Demand Custom Metrics - Monthly Datadog Subscription, 2025-04-01 - 2025-04-30, 6265 On-Demand Custom Metrics (bundle of 100)	6265	.05	USD \$313.25
On-Demand Indexed Log Events - 7 Days (Bundle of 1M) - Monthly Datadog Subscription, 2025-04-01 - 2025-04-30, 1690.297 On-Demand Log Events - 7 Days (Bundle of 1M)	1690.3	1.905	USD \$3,220.02
On-Demand Infrastructure Host (Pro) - Monthly Datadog Subscription, 2025-04-01 - 2025-04-30, 3 On-Demand Infra Host (Pro)	3	18	USD \$54.00
On-Demand Log Ingestion - Monthly Datadog Subscription, 2025-04-01 - 2025-04-30, 8657 On-Demand Log Ingestion	8657		USD \$865.70

▼ Datadog logs



1 week of indexed logs

▮ Ingest any logs, get structured & SQL-searchable data

- Super easy to set up
 - Vercel is a 1 click integration
 - Has SDK for nodejs
 - Has ingestion plugins for prometheus, opentelemetry
 - Has an easy to use, built-in transformer - can parse json into searchable fields from text fields
- Usage pricing, no payments for users
- Beautiful UI, exactly what we need in terms of functionality → logs searchable by arbitrary fields + metrics + dashboards
- Free tier with 3GB of logs, then \$0.45 per GB + \$0.025 per week of retention

I've already deployed a prototype for auth:

- 1 simple function to log
 - Maintains context (you can add values that will be persisted across all logs in 1 invocation)
 - Structured (guarantees that log will be searchable in betterstack)

 2025. Jun 3., Tue

Logging 2.0

Continue with logging. I want to have chap creation flow covered:

- All logs across all services end up in betterstack
- Trace IDs

Plan is:

-  auth - via vercel plugin
-  platform-api - via SDK nodejs library
-  supervisor - via SDK nodejs library
-  chaplet - via loki container that we already have OR SDK nodejs library
 - executor - I'm not sure about this one. It's python, probably will require a bit more tinkering to pass traces and manage context, but should be doable

merged add support for chaps authentication with OAuth towards services/providers



feat(chaps-oauth): add support for chaps authentication with OAuth towards services/providers

- OAuth flow implementation
- `/api/oauth/callback` gateway endpoint for OAuth
- exposing chap's access tokens to agent config
- new chap `lifecycle.yaml` to describe eg oauth steps
- Orchestra update to v0.34 to support HTTP MCP Server
- new `templates/chaps/base` chap template to extract common config.yaml elements
- new Linear chap



APIs for integration of user oauth into mobile client (Chaps app)

GET `/api/chaps/:chapId/user-steps`

`luki-labs.postman.co`

Returns the list of user facing setup/onboarding etc actions that the chap defines. For now there's only `oauth.authorize` type action here. Plan is there will also be a `config.required` type action as well for Chaps that need an api key, and later eg a Chap can add its own onboarding widget here.

each action has a state: `incomplete` | `completed` (can elaborate later to have pending or failed states as well, but did not want to complicate for now)

example response

```
1  {
2    "user_steps": [
3      {
4        "action": {
5          "type": "oauth.authorize",
6          "provider_name": "linear",
7          "client_config": {
8            "resource": "https://mcp.linear.app/sse"
9          }
10     },
11     "state": "incomplete"
12   }
13 ]
14 }
```

Performing `oauth.authorize` user step

Update: as of now we won't yet be able to perform the flow nicely in an embedded WebView as OAuth requests are mostly blocked from webviews. (eg [Google](#)) And in the ext browser the user is not logged into Chaps. Options:

- on top using of the below steps, the app needs to implement a deeplink entry point, which will serve as the `redirect_url`, and then the app can forward the token response to the Chaps backend with a proper chaps-authenticated request. The same challenge also means that as in ext browser user is not logged into chaps, I cannot easily combined the below steps to a single endpoint that could be launched in ext browser (as generate link endpoint needs to be an authenticated call).
- alternatively we could use a one time token (tunneled in the state handback of the OAuth callback), in which case can keep the experience within an in-app browser (which can be closed automatically), but have to discuss that with BalintB, as that involves the supervisor auth layer / auth BE as well.
- 3rd option: the oauth callback endpoint is not behind our own auth - I'll have to explore this a bit more, but could work (I think poke.cx also does this)

Can happen either automatically after chap is installed, or later when clicking on a menu item within chap etc.

1. POST `/api/oauth/:chapId/authorization-url`

luki-labs.postman.co

This is the endpoint to start an `oauth.authorize` user step from Chaps client

need to post the provider name

(it can also accept an `client_config`, eg echoed from the user-steps endpoint, eg I can imagine user has a read/write choice first, and then we need to initiate with a dynamic config here, which overrides scope parameter)

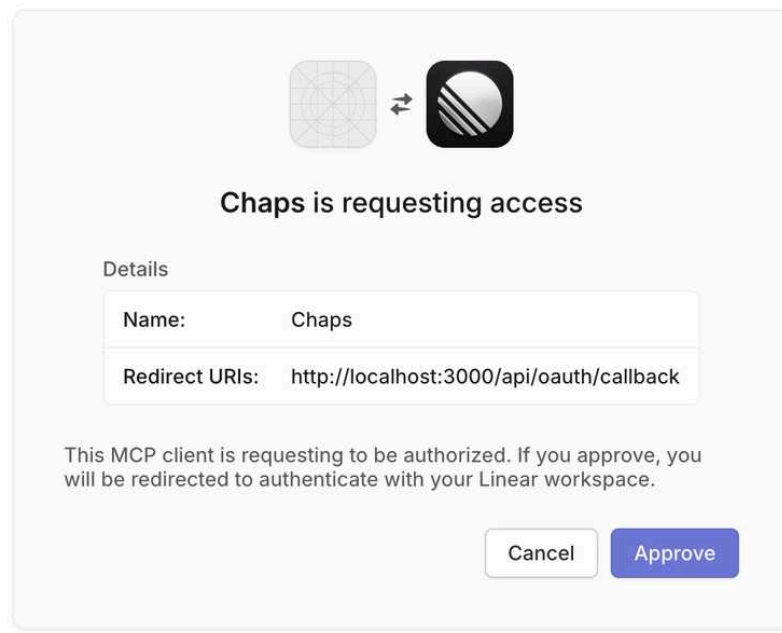
```
1 {
2   "provider_name": "linear"
3 }
```

and returns the authorization URL, eg

```
1 {
2   "authorization_url": "https://mcp.linear.app/authorize?
  response_type=code&client_id=1Y1514EwkfFjpGQe&redirect_uri=http%3A%2F%2Flocalhost
%3A3000%2Fapi%2Foauth%2Fcallback&scope=read+offline_access&code_challenge=tyKgHzj
xqkJKcCUxXzYsDBI4jn6ByE4i0_PWLwtA6ao&code_challenge_method=S256&state=e2bf0da8-2e
b4-4702-a506-0e523f1b4fbb"
3 }
```

2. Open the `authorization_url` in a full screen / large screen ~~webview~~ (with Chaps session token set in cookie) ext browser

the link will perform OAuth, as for user consent and redirect to a chaplet endpoint in the background. End result is a success screen (or failure screen).



Note:

- the success screen is not yet designed, just a "plain text" output.
- we may not even want to show it long, we can agree on a JS→Swift bridge method, which I call on load to signal it can be closed. (or maybe the webview can even be automatically observed from native side and if url changes to .../callback and loads, then can be closed by mobile client)

After this is done, `/api/chaps/:chapId/user-steps` will return this step as `completed`

note: Linear chap now blows up if you start a convo without OAuth configured, will fix and work on this tomorrow

📅 2025. Jun 3., Tue

will work around **triggering the OAuth setup from within chat conversation**, meaning:

- user starts thread in without OAuth setup
- asks something that needs API call
- the answer explains some config is missing and gives a link, which performs the config (a new small built-in chap endpoint in this case, which does what the app would do above in two steps)

Update: discuss app deeplink redirect in OAuth flow with Balint

Other follow-up items

- battle test our OAuth solution with **Google Calendar MCP**, which is not just an "other" OAuth MCP, but Google uses static app registration and uses `oidc` instead of `oauth2`, and the resulting token needs to be injected in a completely different way into the 3rd party MCP server, so very different. Yet, it is planned to fit into the framework - if this also works, I'll be very happy.

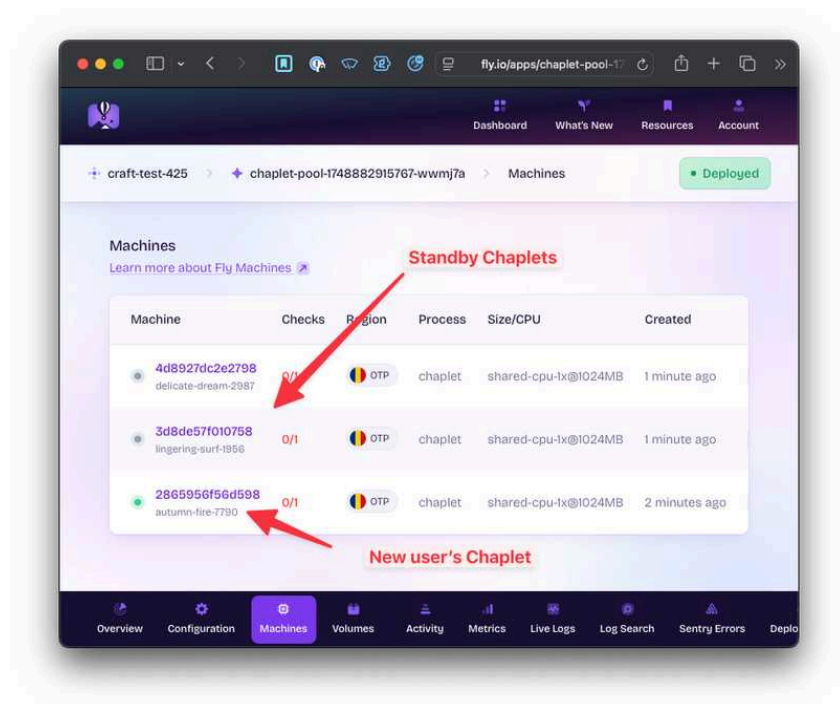
-

- extend the new user-steps approach to api key config requiring Chaps

Fly.io machine pooling

[Implemented app pools](#) on Fly.io today (see [doc](#)). Pooling allows us to

- use a common ("pool") app which has machines attached with different volumes per user
- new Chaplets are created from a pool of free machines, which is *much* faster than creating the VMs on-the-fly
- when a machine has been started for a user, the pool is replenished in the background by building machines from the Chaplet container image
- when an app is "full", we just create a new pool and new users get added there from that pool's standbys



Same pool with 2 machines in standby under 1 app:

```
1  {
2    "success": true,
3    "poolMachines": {
4      "chaplet-pool-1748882915767-wwmj7a": [
5        "3d8de57f010758",
6        "4d8927dc2e2798"
7      ]
8    },
9    "totalApps": 1
10 }
```

Originally, I wanted to create Volumes once a user has registered, and attach them to the pooled machine before starting it - meaning only VMs are in standby. This approach did not work due to a constraint in Fly.io: if a volume is created after the VM and we attempt to attach that volume to the new machine, there's a high chance the call fails. Volumes must be on the same host as the VM. If they are created after the VM, there's no guarantee they will be on the same host (even in the same region). The correct way on Fly.io to create VMs with volumes is to create the volume first, then create a machine with the volume ID in its config so it gets put on the same host.

The solution for pooling is we create volumes alongside the standby VMs. Those are mounted but the VMs not started, and on a new user registration they are issued a VM along with its mounted volume. Unfortunately volumes incur costs even though they are not mounted on a running machine, so this solution is slightly costlier. Stopped VMs do not cost us anything.

I created a [separate Fly.io org](#) to test machine creation. I've been testing this today, but will not start deploying to prod until the board demos have finished 📅 2025. Jun 3., Tue (or Wednesday). It's a very low-level change to Chaps infra, as such, there's some risk associated with it.

I've started writing integration tests for these components (which was very scarce before) so we don't break stuff in the future.

Reading through Fly.io's forums, there seems to be some cases where VMs cannot be created in regions due to capacity, and there seems to be (undocumented) limits to the number of VMs an account can create. I've reached out to Fly.io support to clarify as this would not be great when we start onboarding users for beta.

Due to this risk, I've spent a little time for abstracting away the cloud provider infra for us to be able to switch from Fly if it turns out there's no capacity or VM creation doesn't work well with ~100 users. DigitalOcean for example have a similar offering ([App Platform](#)). The foundations are there, but I won't spend more time on this unless we decide to bail <https://github.com/lukilabs/chaplets-supervisor/commit/bc3417d06c925664da06a02951d4ab290bcd790e>.

Tomorrow

- Code and local testing is done for pools. Tomorrow I'll be finishing up pooling by deploying to test with real users. **Expect some downtime while I reroute your users.** I'm expecting some issues to crop up, will be on those.
- P2 is doing the proper split of sandbox - production environments so we can continue with these changes without running the risk of breaking prod. Currently we develop locally and a deployment is prod.

↑ Daily Notes - 2025 June

Jun 2., Mon



Chap Team

Balint Orosz → Levy Melnikov · Feedback on Profile Endpoint : We need to make these more user-friendly / structur...

↑ Jun 2., Mon

Chap Team



Balint Orosz



Levy Melnikov

Feedback on Profile Endpoint : We need to make these more user-friendly / structured. Here's a list of things to do :

- **Update Profile Photo** → This should be a single call where I pass over the image. The service itself should handle requesting the signature URL's, updating the user's profile etc. If any transformation (i.e. resize) / validation is needed later that can be done here
- **Update User First and Last Name** → This should be a single call.
- **Update Username** → This should be a single call as no 2 users can have the same username, so it needs validation

(I am not blocked as I can mock these on the client side)

Model Usage Tracking

I've finalized the PRs. On chaplet side we wrap the openai library and make calls to Auth with usage data. On Auth we aggregate usage and store it in Redis. For now all usage is just summed up per chap per model. In future we can extend this to save each usage separately into a ledger type table and build billing on top of that.

I chose wrapping openai with custom code as this solution proved to gives us the most control, simplicity and independence from 3rd party libraries. Diagrams for this solution and an alternative one: <https://s-staff.craft.me/ViqBnf6fIBN8jG>

Model Usage in Grafana & Logs



I managed to get llm model usage into grafana. At the same time I realized that I don't like grafana. It might be that I'm just used to datadog, but grafana seems like a very cumbersome tool.

My idea was to emit a log with a json with all of the necessary info on model usage and plot it in datadog. Unfortunately logs are ingested as text strings, so json is not available. → I wrote a ~100 line log forwarder service that just tails log files on chaplet and sends them to grafana, when it encounters a json it "unpacks" it and sends top level fields as labels.

I think our logs are lacking. I'd like to look into it more, but some thoughts are:

- Current Grafana
 - Logs from vercel and realtime-platform are not present
 - Vercel support is only available in enterprise plan
 - Chart builder is very unintuitive
 - All logs are currently just text without fields \ labels →
- What we should have
 - All logs in one place

- Easily logging indexable fields \ labels: `log({chap_id: 123,...})` → being able to find this log by using a filter in the dashboard

Update:

-  Made APIs more user-friendly following suggestions from Balint from above

 2025. Jun 2., Mon

Fixes

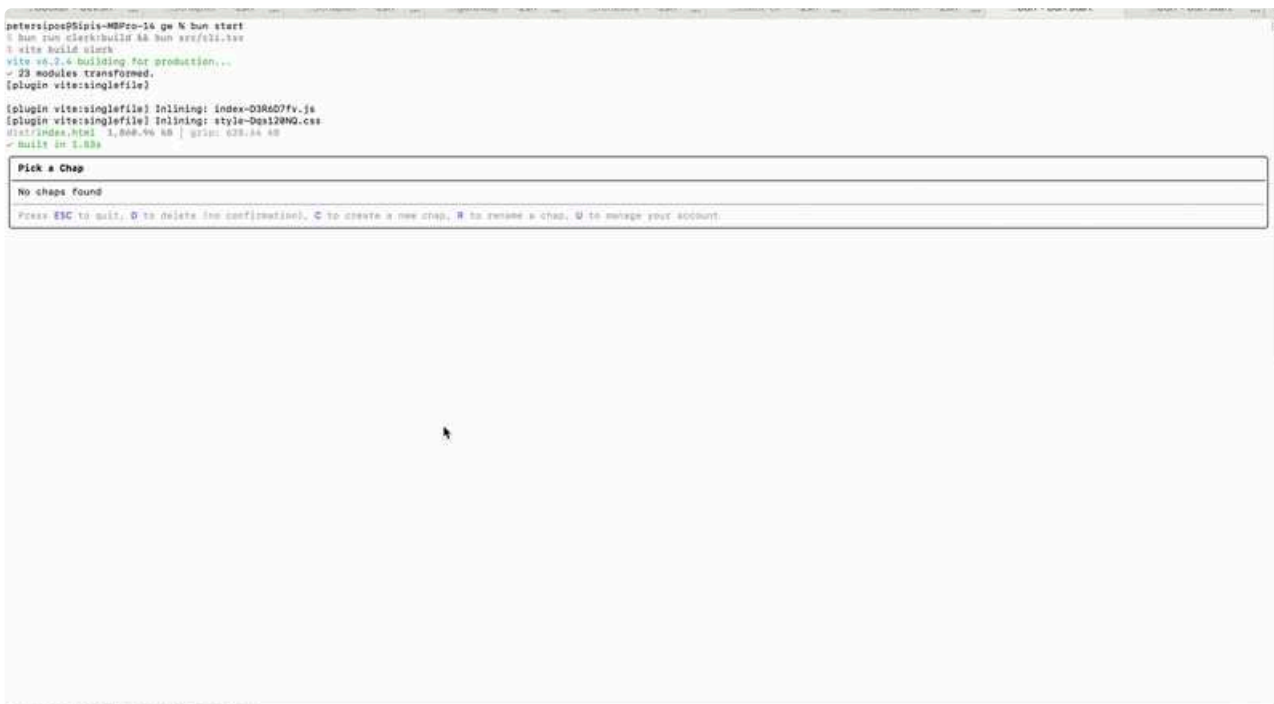
- Thread creation should not allow adding invalid participants
- Propagate as much errors as possible from chaplet to Auth Chap API response on creation
- API for deleting threads
- A more complicated one: creating a chap failed a few times (logs are in vercel and in slack)

Worked on OAuth flow and a Linear chap. OAuth as always is a quite deep hole, I'm building on a super battle tested library called `openid-client`, but had to pick up some domain knowledge as well. Also dissected what `remote-mcp` library is exactly doing - unfortunately it has a few things hardwired, and would not support eg Google OAuth. Considered forking it, but it is also doing a bunch of other stuff, and there's the above mention `openid-client` and similar libs, which seem to be exactly what we need (just have to configure and wire them properly).

Long story short:

- need 1-2 days more for these OAuth based MCP chaps.
- My goal for Monday is to have a successful Linear and Google Calendar conversation in Chaps.
- then we'll need to integrate it e2e eg to authorize in mobile browser of the user (or even magic link for desktop)
- Otherwise next thing after this (or maybe in parallel somewhat), is some fixup on widget styles/layout, so they show up decently inside chaps webviews as Balint is now actually rendering them. Also whatever blockers or serious stuff comes up in integration, can dynamically prioritize.

UPDATE: found a few extra hours over the weekend, and progressed until setup started working, for Linear as a first test subject



The screenshot shows a terminal window at the top with the following output:

```
petersipos@petersipos-MBP:~$ npm run start
npm run start
vite build --mode production
vite v4.2.4 building for production...
✓ 23 modules transformed.
[plugin vite:singlefile]
[plugin vite:singlefile] Inlining: index-D3R6D7fv.js
[plugin vite:singlefile] Inlining: style-Den12WNG.css
dist/index.html 3,868.94 kB | gzip: 628.84 kB
✓ built in 5.80s
```

Below the terminal, a web browser window displays a dialog titled "Pick a Chap". The dialog has a header bar with the title and a close button. The main content area says "No chaps found". At the bottom, there is a footer with instructions: "Press ESC to quit, D to delete (no confirmation), C to create a new chap, R to rename a chap, U to merge your account."